

Event-First Operational Ontology: Cryptographic Event Chains for Multi-Agent Concept Consensus

Anonymous Author(s)¹

Anonymous Institution

Abstract. The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative knowledge synthesis, yet the conceptual outputs generated by such systems lack typed relations, lifecycle traceability, and machine-checkable provenance. This paper presents ADL Lite, an event-first operational ontology in which concepts are modeled as append-only, cryptographically hashed EventChains and all lifecycle state—status, confidence, validators—is derived exclusively from event history. Formal derivation semantics specify status computation and confidence aggregation as deterministic functions over the event sequence, with six proved properties including determinism (Theorem 1), confluence under fork (Theorem 2), confidence boundedness (Theorem 4), and Sybil-resistant aggregation (Corollary 2). A precise canonical serialization with recursive key sorting and six-decimal float normalization ensures platform-independent SHA-256 hashing; timestamps are deliberately excluded from hash input to tolerate arbitrary clock skew across distributed agents. nine experiments establishing chain integrity (100% P/R on 60 chains), exhaustive status derivation (2,204/2,204 correct), snapshot round-trip (38/38 match), precondition enforcement (P=1.0, R=1.0, F1=1.0 across 13 test cases), multi-agent auditability (5/5 chains), real-time pattern detection (6/6 tests, 0 false positives), edge offline synchronization (5/5 tests, zero merge conflicts), Git baseline comparison (per-event diagnostic precision and semantic violation classification), and scale validation (495,671 chains / 5,080,714 events in 238 s, zero failures). Appendices provide complete, syntactically valid PROV-O export (Appendix A) and SHACL shape validation (Appendix B) examples. ADL Lite is pip-installable, operates over Markdown files in standard Git repositories, requires no triple store or enterprise infrastructure, and achieves deployment-ready lifecycle traceability under a collaborative-audit threat model with explicit paths to Linked Data Proofs and CRDT-based merging.

Keywords: ontology engineering; event sourcing; multi-agent systems; knowledge graphs; anti-money laundering; provenance; PROV-O

1 Introduction

1.1 Background and Motivation

The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative discovery and knowledge synthesis [1][12]. Contemporary frameworks such as LangChain, LlamaIndex, and MetaGPT enable agents to decompose complex tasks, invoke external tools, and maintain conversational state across extended interactions. When multiple such agents operate concurrently—each generating hypotheses, evaluating evidence, and proposing conceptual structures—their outputs accumulate as unstructured natural language prose. This unconstrained textual output, while expressive in semantic range, lacks typed relations, explicit lifecycle tracking, and machine-checkable provenance. The resulting conceptual artifacts are difficult to validate, version, or compose into shared ontological commitments [23][34]. Cross-references between concepts remain implicit in natural language, temporal evolution is undocumented, and the absence of a shared schema prevents agents from building upon one another’s discoveries in a structured fashion. Each agent may develop its own vocabulary, classification criteria, and confidence calibration, leading to semantic fragmentation that undermines collective intelligence.

Traditional ontology engineering addresses this problem through formal specification: domain experts construct object hierarchies, define property constraints, and establish inference rules using languages such as the Web Ontology Language (OWL) [45]. Palantir’s Foundry Data Engine (FDE) exemplifies the industrial state of the art, organizing enterprise knowledge around Object Types, Link Types, Property Types, and Action Types configured through a dedicated user interface [56]. This approach produces rigorously structured ontologies but imposes substantial deployment friction. Ontology construction requires specialized expertise in both the target domain and the formalism, dedicated tooling that may carry significant licensing costs, and manual curation workflows that do not scale to the velocity of agent-generated conceptual output [58]. A further limitation is that LLM-based agents cannot meaningfully participate in this authoring process: they lack programmatic access to ontology construction interfaces and cannot manipulate the underlying formalisms through standard software development kits [59]. The Ontology Manager UI in Palantir FDE, for instance, requires human operators to define object types, link types, and action types through point-and-click configuration; no programmatic creation APIs are exposed for automated agent authorship [59]. Semantic elements (Object Type, Property Type, Link Type) and kinetic elements (Action Type, Function) must be co-defined through this interface, creating a bottleneck that prevents LLM agents from autonomously extending the ontology as they discover new concepts.

The philosophical foundations of this work draw upon Ludwig Wittgenstein’s *Tractatus Logico-Philosophicus* [60]. In proposition §1.1, Wittgenstein asserts: “*The world is the totality of facts, not of things.*” The original German—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*”—emphasizes that reality consists not of isolated objects but of states of affairs in which those objects

participate [2]. An object, on this view, has no ontological standing independent of the facts in which it appears. A thing gains its significance only through participation in states of affairs that constitute the fabric of the world. The concept of a validated scientific hypothesis, for instance, is not a static entity possessing an intrinsic property of “validatedness”; rather, it is constituted by the totality of validation events, evidentiary assertions, and review actions that have occurred in its history. This insight has direct architectural implications: if concepts are understood not as self-subsistent entities but as loci of event histories, then an ontology should be organized around *actions* rather than *objects*. Status, confidence, and relational embeddings become derivative properties computed from the sequence of events that constitute a concept’s history, not mutable fields stored alongside the concept itself [3][4]. The event-first stance does not deny the existence of objects—Wittgenstein explicitly acknowledges that objects are constituents of facts—but it denies that objects have ontological priority. In engineering terms, this means that the action registry and event schema are designed first, and object-like abstractions emerge as projections of event patterns rather than as foundational data structures. The event-first architecture thus operationalizes Wittgenstein’s insight: the ontology records what has happened (the totality of facts), not what purportedly exists (a catalog of things), and concepts acquire their identity through the events in which they participate.

1.2 Research Gap and Problem Statement

Existing operational ontologies, including Palantir FDE and OWL-based knowledge graphs, are predominantly *object-first*: an Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is added as an afterthought to model operations upon the previously defined object structures [56][6]. This design creates deployment bottlenecks at multiple stages of the ontology lifecycle. Concept instances carry mutable status fields that can diverge from their actual event histories when updates occur outside the governed action pathway. Updates are destructive: modifying a concept’s properties overwrites prior state, making audit reconstruction dependent on external logging infrastructure that may itself be incomplete or inconsistent. Agents wishing to participate in ontology evolution must navigate complex user interfaces or specialized SDKs with steep learning curves, precluding lightweight automation at the scale required by modern multi-agent research pipelines [59]. The cumulative effect is that ontology engineering remains a human-intensive activity disconnected from the generative processes that most need semantic structure—the autonomous production of conceptual content by cooperating agents.

The specific problem addressed in this paper is formulated as follows: *How can an operational ontology achieve production-grade lifecycle traceability, cryptographic integrity, and multi-agent consensus without requiring heavyweight tooling, expert curation, or object-first mutable state?* This problem statement motivates the design of a system that is simultaneously (i) pip-installable and Git-backed,

enabling frictionless deployment in standard Python environments without enterprise infrastructure; (ii) natively authorable by LLM agents through Markdown files that serve as both human-readable documentation and machine-parseable semantic assertions, eliminating the need for specialized ontology authoring interfaces; (iii) cryptographically auditable via append-only event chains that detect accidental tampering with the ontological record through SHA-256 hash verification—providing *tamper-evidence* suitable for collaborative audit among trusted participants, though not *tamper-proof* guarantees against adversarial actors without additional cryptographic signing (see Section 3.7); and (iv) semantically governed by a closed action registry with structured precondition validation that eliminates runtime code injection through statically typed comparator enumeration. The system targets the gap between unstructured agent output and formal ontology deployment, where lightweight operational semantics can accelerate conceptual convergence without claiming to replace enterprise platforms for high-assurance compliance workloads [5].

It is important to clarify the security posture that the current system achieves. ADL Lite employs SHA-256 hash chaining to link successive events: each event’s hash covers the previous event’s identifier, creating a cryptographic chain in which any modification to historical data is detectable as a hash mismatch at the point of first alteration. This mechanism provides *tamper-evidence*—it enables participants to detect accidental or malicious modification of the event record—rather than *tamper-proofness*, which would require digital signatures, a public-key infrastructure, or distributed consensus to prevent alteration by adversarial actors. The current threat model assumes collaborative audit among trusted participants, with the hash chain serving to detect accidental corruption or misattribution. Section 3.7 elaborates the trust model, and Section 3.8 discusses the path to stronger guarantees through Linked Data Proofs [14].

A further deliberate simplification in the current design concerns actor identity. The `actor` field in each event is a plain string identifier (e.g., `"discoverer_agent_1"`, `"human_reviewer"`), not a cryptographically authenticated principal. This means that event authorship is *attributed* but not *proven*: any participant with write access to the shared Git repository can append an event claiming any actor name. This is a deliberate Phase 1 simplification that prioritizes deployment frictionlessness over adversarial resistance. Binding actor identity to cryptographic keys—through Linked Data Proofs with URDNA2015 canonicalization and ed25519 signatures, or via integration with an existing PKI—remains future work (Section 3.7). In the intended deployment context (small-to-medium research teams collaborating within a trusted Git repository), the combination of Git’s native audit log (`git blame`, signed commits) and the hash chain’s *tamper-evidence* provides a usable level of accountability.

1.3 Contributions

This paper presents ADL Lite, an event-first operational ontology for multi-agent concept consensus. The system represents concepts as append-only, cryptographically hashed EventChains, with lifecycle state derived exclusively from event his-

tory. It is evaluated through eleven experiments (E1–E11) testing chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, real-time pattern detection, edge offline synchronization, Git baseline comparison, and throughput scalability on real-world financial transaction data. The four principal contributions are enumerated below.

Contribution 1: Event-first ontology architecture. This paper proposes EventChain as the fundamental representational unit for ontological concepts. Each concept is modeled not as a mutable object with stored properties but as an append-only, cryptographically hashed sequence of typed events. A concept’s status, confidence score, and validator identity are *computed* from the event chain rather than stored as mutable fields. SHA-256 hashing and cryptographic linking via `previous_event_id` ensure that any tampering with the chain is detectable through `verify_integrity()`. The event model includes nine event types spanning the lifecycle (REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE), assertions (RELATE, EVIDENCE, SEAL), and communication (ANNOUNCE, PUBLISH). Empirical evaluation on 495,671 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset demonstrates zero integrity failures across 5,080,714 events processed in 238 seconds (E6), confirming that the architecture scales to production-level data volumes without degradation of cryptographic guarantees.

Contribution 2: L4 action blocks with structured precondition validation. This paper introduces a closed action registry comprising nine core actions (register, validate, fork, deprecate, archive, announce, publish, sync_dashboard, listen), each defined in `adl_core_ontology.yaml` with declarative precondition rules. Precondition validation employs a `Comparator` enumeration (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) evaluated against `ADLFrontMatter` fields through statically typed Pydantic models—eliminating runtime `eval()` entirely. The `ActionExecutor` validates preconditions at parse time before dispatching side effects, ensuring that no action can transition a concept into an invalid state. Thirteen test cases covering all registered actions achieve perfect precision ($P = 1.0$), perfect recall ($R = 1.0$), and perfect F1 score ($F1 = 1.0$), with zero false positives and zero false negatives (E4).

Contribution 3: Throughput validation and correctness at scale. This paper reports the results of eleven experiments establishing correctness properties across the full system: 100% chain integrity verification on 50 valid and 10 intentionally corrupted chains (E1); 100% status derivation accuracy across 2,204 exhaustive event sequences including 2,197 three-event combinations and 7 edge cases (E2); 100% snapshot round-trip consistency across 38 concept files spanning synthetic and real AML domains (E3); 100% precondition enforcement precision and recall on 13 action-validation test cases (E4); 100% multi-agent auditability across 5-agent simulations producing independently verifiable chains (E5); 100% chain integrity at scale on 201 chains from real IBM AML transaction data (E6); real-time pattern detection on actual AML transactions with zero false positives across 9,000 legitimate events (E7); and full FDE pipeline validation with 201/201 concepts passing executor validation (E10). These experiments validate

system correctness and throughput scalability; they do not constitute domain-level evaluation of ontological quality (e.g., human expert ratings of concept usefulness) or operational compliance certification. The aggregate evidence supports the claim that event-first derivation of ontological state is computationally feasible, cryptographically sound, and empirically correct under exhaustive test conditions.

Contribution 4: Open-source reference implementation. This paper accompanies the release of `adl_lite`, a Python toolkit providing an `ADLParser` for L1–L4 document parsing with Pydantic validation, an `ActionExecutor` for precondition-validated action dispatch, a `ConsensusEngine` for status transition enforcement with fork semantics, a `DataImporter` for bulk EventChain construction from CSV datasets, and a unified experiment runner enabling reproducible evaluation via a single command-line interface. The toolkit is pip-installable, operates over Markdown files stored in standard Git repositories, and requires no external triple store, OWL reasoner, or enterprise infrastructure. All experiments reported in this paper are executable through the provided experiment runner.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys related work in ontology engineering, event sourcing patterns, and multi-agent knowledge representation, positioning ADL Lite with respect to Palantir FDE, OWL-based systems, and emerging lightweight ontology approaches. Section 3 presents the ADL Lite architecture in detail, beginning with the philosophical foundation of event-first ontology design, proceeding through the L1–L4 document model, the EventChain data structure, the action executor, the consensus engine, and concluding with the concurrency model, trust model, and semantic web interoperability mappings. Section 4 describes the experimental design across eleven experiments (E1–E11), including the IBM AML dataset and the methodology for integrity, derivation, round-trip, precondition, auditability, and scale evaluation. Section 5 reports quantitative results for all experiments. Section 6 discusses architectural implications, compares ADL Lite with Palantir FDE and OWL-based alternatives, and acknowledges limitations including synthetic pattern detection and stub side effects. Section 7 concludes and identifies directions for future work, including full ConsensusEngine–ActionExecutor integration, cryptographic actor authentication, and expanded dataset evaluation.

2 Related Work

2.1 Ontology Engineering and Knowledge Graph Construction

Traditional Ontology Construction: Expressiveness vs. Deployment Cost The Semantic Web vision articulated by Berners-Lee, Hendler, and Lassila[1] established the foundational goal of machine-readable, interlinked knowledge representations on the web. This vision was operationalized through the OWL 2 Web Ontology Language, whose formal semantics rooted in description logics provide expressiveness ranging from the lightweight OWL 2 EL profile to the fully decidable OWL

2 DL[12]. Standard toolchains including the Protégé ontology editor[23], Apache Jena with its TDB triple store[34], and Virtuoso Universal Server[45] enable ontology authoring, persistence, and SPARQL query evaluation at considerable scale. Reasoners such as HermiT and Pellet support automated classification and consistency checking, albeit with computational costs that scale poorly for large or rapidly evolving knowledge bases[56].

However, the deployment cost of full OWL-based pipelines remains prohibitive for many operational contexts. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming or event-driven workloads; and the expertise required for ontology engineering creates a bottleneck distinct from application development[58]. These factors collectively confine heavyweight Semantic Web technologies to domains where upfront investment in formal knowledge representation is justified by long-term reasoning requirements, leaving a gap for lightweight, operational ontologies that can evolve at the speed of software engineering.

Lightweight Ontology Approaches: Accessibility Gains In response to these deployment barriers, the community has developed a spectrum of lightweight approaches that trade full logical expressiveness for accessibility and operational simplicity. Schema.org, launched in 2011 as a collaboration among major search engines, provides a pragmatic, property-centric vocabulary that now appears on over 40% of web pages[59]. Its design intentionally prioritizes decentralized adoption over semantic perfection, accepting property inheritance irregularities in exchange for rapid vocabulary evolution[60]. JSON-LD[2] extends conventional JSON with namespace support and URI resolution, allowing RDF-compatible serialization without requiring consumers to adopt full Linked Data processing pipelines. The Shapes Constraint Language (SHACL)[3] provides data validation without inference, enabling closed-world constraint checking over RDF graphs that complements rather than replaces OWL’s open-world semantics[4]. Recent work has extended SHACL to support validation under updates[5], explainable validation through retrieval-augmented generation[6], and schema reasoning tasks[4], broadening its applicability beyond static integrity checking.

Comparison with shape constraint languages. SHACL and its alternative ShEx (Shape Expressions)[43] represent the state of the art in RDF constraint validation and provide natural points of comparison for ADL Lite’s **Comparator**-based precondition system. Table 2 contrasts the two families of approaches across eight salient dimensions. SHACL targets full RDF graphs with 50+ built-in constraint types (including path-based cardinality, value range, pattern matching, and logical combinations), validated through batch SPARQL queries or dedicated engines at $O(|G|)$ cost per shape, where $|G|$ is the graph size[3]. ShEx provides a compact grammar-based syntax for describing RDF node shapes and their neighbor constraints, with validation through batch processing or API calls at comparable asymptotic cost[43]. Both assume RDF as the underlying data model and require graph-processing infrastructure.

ADL Lite’s **Comparator** enum—supporting scalar comparisons (EQ, GT, LT, IN, NOT_IN, REGEX) over L1 YAML front matter fields—is deliberately simpler.

It trades the full expressiveness of path-based graph constraints for $O(1)$ per-rule validation cost at parse time through Pydantic, zero external dependencies, and operation within plain-text Markdown documents rather than RDF graphs. The design philosophy is complementary rather than competitive: ADL Lite uses the **Comparator** for lightweight scalar preconditions within document-native ontologies, while Phase 3 integration will layer SHACL for graph-level structural constraints over exported L3 relation triples. The **Comparator** handles *scalar field validation* (e.g., ensuring `confidence > 0.7` before a `VALIDATE` action); SHACL handles *graph structure validation* (e.g., ensuring every **Concept** has exactly one **status** property and at least one **actor** association).

Dimension	ADL Lite Comparator	SHACL	ShEx
Target	L1 YAML front matter fields	RDF graphs	RDF nodes
Constraint types	Scalar comparison (EQ, GT, LT, IN, NOT_IN, REGEX)	50+ built-in constraints (<code>sh:minCount</code> , <code>sh:pattern</code> , <code>sh:node</code> , etc.)	Shape expressions (triple constraints, cardinality, recursion)
Graph constraints	No	Yes (path-based, cardinality, logical combinations)	Yes (neighbor constraints, reference shapes)
Closed-world assumption	Yes (by design)	Yes (<code>sh:closed</code>)	Yes (closed shapes)
Validation timing	Parse-time (Pydantic)	Batch / SPARQL endpoint	Batch / API
Error reporting	Pydantic field-level exceptions	<code>sh:ValidationReport</code> (RDF)	Typing errors (structural)
Runtime cost	$O(1)$ per rule	$O(G)$ per shape	$O(G)$ per shape
Extensibility	Edit YAML registry	Write new SPARQL constraints (<code>sh:sparql</code>)	Write new shapes
Dependencies	Zero (Pydantic built-in)	RDF library + SHACL engine	RDF library + ShEx processor

These lightweight technologies collectively lower the barrier to structured data publishing but remain tethered to the RDF data model. They require triple stores or graph databases for non-trivial workloads and assume that data are already available in RDF-compatible formats—assumptions that exclude the vast corpus of domain knowledge encoded in Markdown documentation, plain-text notes, and collaborative wikis.

Document-Native Semantic Layers: Markdown-Based Knowledge Representation

The past decade has witnessed significant growth in Markdown-based personal and collaborative knowledge management tools that implicitly encode graph structure through bi-directional linking. Foam[7], Obsidian[8], and Dendron[9] extend plain-text Markdown with wiki-style links, backlinks, tag hierarchies, and graph visualization, enabling knowledge workers to construct personal knowledge graphs without leaving their editing environment. Dendron, in particular, introduces hierarchical note schemas with refactoring capabilities that preserve link integrity during structural changes[10], anticipating the need for schema-level governance in document-native knowledge bases.

These tools demonstrate that meaningful semantic structure can emerge from authoring conventions rather than formal ontological commitments. However, they lack machine-checkable coordination primitives: no constraint language prevents schema drift, no access control governs scope visibility, and no consensus mechanism mediates conflicting contributions from multiple agents. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

2.2 Palantir Foundry Data Engine

FDE Ontology Layer: Nouns, Verbs, and Operational Semantics Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type (classes of entities), Property (attributes), Link (typed relationships), and Action (kinetic operations that modify the world)[11]. This architecture unifies what Palantir terms “semantic elements” (nouns: objects, properties, links) with “kinetic elements” (verbs: actions, functions, dynamic security), enabling AI agents to both query and act upon the ontology through governed interfaces[13]. The Ontology Metadata Service (OMS) defines all ontological entities, while Object Storage V2 supports tens of billions of objects per type with incremental indexing[14].

Digital Twin Model: Platform-Managed Object Representations Palantir’s operational power derives from treating the ontology as a digital twin layer mediating between raw data and operational decisions[14]. A dataset of GPS coordinates becomes a fleet tracking system when the ontology defines Vehicle objects, Route links, and PositionUpdate actions; a dataset of financial transactions becomes a risk monitoring system when the ontology defines Account objects, TransactionFlow links, and FraudAlert actions[15]. Palantir’s AIP (Artificial Intelligence Platform) operates within this ontology, with agents accessing objects, triggering actions, and reading relationships through governed interfaces rather than querying raw data[14].

Limitations: Enterprise Scale, Expert Authoring, and LLM-Agent Accessibility

Despite its sophisticated design, Palantir Foundry exhibits three limitations that motivate complementary approaches. First, deployment is enterprise-scale: the

platform targets customers averaging millions of dollars in annual spend, with infrastructure requirements far exceeding those of Git-backed documentation or small-team knowledge bases[11]. Second, ontology authoring requires specialized expertise and Palantir-specific tooling; Object Types, Link Types, and Action Types are configured through proprietary UIs rather than plain-text formats accessible to LLM agents or generalist developers[13]. Third, the platform is not designed for LLM-native authorship: agents consume and act upon the ontology but do not author or evolve ontological definitions through conversational interfaces. ADL Lite addresses each limitation by offering pip-installable deployment, Markdown-native authoring with YAML-embedded semantics, and explicit support for multi-agent concept consensus through append-only event chains.

2.3 Multi-Agent Consensus and Auditability

Consensus Mechanisms in Multi-Agent Systems Multi-agent systems require coordination mechanisms to reconcile divergent beliefs or proposals among autonomous participants. Classical approaches span voting protocols, reputation-based trust aggregation, and Byzantine fault-tolerant (BFT) consensus derived from distributed systems[16]. Committee-based BFT protocols such as Practical Byzantine Fault Tolerance (PBFT) and its successors achieve agreement among partially trusted participants with logarithmic communication complexity, but assume a fixed membership model and synchrony bounds that limit applicability to open, dynamic agent collectives[17]. Lightweight variants—including Proof-of-Contribution[18], utility-based Raft[19], and Comprehensive BFT (CBFT)[17]—reduce consensus overhead by electing committees, partitioning storage, or adapting consensus granularity to network conditions. ADL Lite adapts these insights to a document-native setting: rather than achieving full BFT agreement over distributed state, it records individual agent proposals in an append-only chain and derives concept status through domain-specific quorum rules encoded in declarative YAML.

Append-Only Audit Logs for Agent Traceability The foundational work of Crosby and Wallach on tamper-evident logging introduced hash-chain data structures that provide $O(1)$ append cost and $O(j-i)$ verification for subsequences, detecting modification, deletion, and reordering attacks against untrusted log servers[20]. These structures underpin Certificate Transparency (RFC 6962)[21], blockchain systems, and emerging agent audit standards. The recently proposed IETF Agent Audit Trail (AAT) draft specifies SHA-256 hash-chained JSON records with mandatory fields for agent identity, action classification, and trust level reporting, explicitly addressing EU AI Act Article 12 requirements for automatic event recording[22]. Complementary proposals from industry—including append-only governance audit ledgers with cross-party verifiability[24] and semantic memory ledgers such as MentisDB[25]—extend tamper-evident logging from system events to agent cognition, recording not merely actions but decisions, corrections, constraints, and handoffs as durable cognitive checkpoints. ADL Lite’s EventChain applies this architectural pattern to ontological evolution: each

concept exists as a cryptographically linked sequence of status-changing events, rendering the entire lifecycle of a definition—from proposal through challenge to resolution—forensically reconstructible.

AML Pattern Detection as Multi-Agent Evaluation Domain Anti-money laundering (AML) pattern detection provides a demanding evaluation domain for multi-agent ontological consensus due to the complexity of financial crime typologies, the heterogeneity of data sources, and the severe consequences of conceptual error. Financial institutions increasingly employ knowledge graphs—connecting people, companies, accounts, and transactions—to uncover hidden relationships, identify shell company structures, and trace beneficial ownership across jurisdictional boundaries[26]. Semantic rule-based approaches model AML domain knowledge as OWL ontologies with Jena inference engines, executing threshold-based rules over transaction data to flag suspicious patterns[27]. These systems face challenges of data quality, scalability, and interpretation complexity that directly motivate machine-readable, auditable concept definitions[26]. ADL Lite’s pilot evaluation targets this domain not as a replacement for production AML platforms, but as a proving ground for ontological coordination: the precision required for financial crime concepts imposes strict correctness requirements that expose weaknesses in informal or mutable knowledge representation approaches.

Provenance and Verifiable Publishing Standards The landscape of Semantic Web provenance and verifiable publishing encompasses several standards that address complementary aspects of the problem space that ADL Lite targets: recording *who* said *what*, *when*, and with *what* evidentiary basis. This subsection surveys four standards—W3C PROV-O, nanopublications, Linked Data Proofs (now W3C Verifiable Credential Data Integrity), and RDF-star—that collectively define the state of the art in provenance-aware knowledge representation, and positions ADL Lite’s EventChain model with respect to each.

W3C PROV-O. The W3C PROV Ontology [28] provides a standard model for provenance interchange on the Web, expressing the PROV Data Model [29] in OWL 2. PROV-O defines core concepts including **Entity** (the subject of provenance), **Activity** (a process that generates or transforms entities), and **Agent** (an entity responsible for an activity), along with properties such as **wasGeneratedBy**, **wasAssociatedWith**, **wasAttributedTo**, and **wasInformedBy** that establish causal and responsibility relationships among provenance elements. PROV-O has been adopted across scientific domains including bioinformatics [30], materials science [31], and workflow provenance systems [32], and has been extended by specialized profiles such as ProvONE for scientific workflow provenance [32] and D-PROV for structured data transformations [33].

ADL Lite’s EventChain can be directly mapped to PROV-O: each Event in an EventChain corresponds to a **prov:Activity**, the concept being modified corresponds to a **prov:Entity**, and the **actor** field in each event maps to **prov:wasAssociatedWith**. The **previous_event_id** cryptographic linkage provides a **prov:wasInformedBy** chain connecting successive activities. However, PROV-O alone does not provide cryptographic integrity guarantees—it is a data

model for describing provenance, not a mechanism for verifying it. An RDF graph serialized in PROV-O can be modified, re-serialized, and redistributed without detection unless additional signing or hashing infrastructure is layered on top. ADL Lite complements PROV-O by extending its descriptive provenance model with built-in SHA-256 hash chaining: the EventChain is simultaneously a PROV-O-compatible provenance trace and a tamper-evident data structure whose integrity can be verified independently of any external trust infrastructure. Future Phase 3 work includes a PROV-O export serializer that will materialize ADL Lite EventChains as fully standard-compliant RDF provenance graphs.

Nanopublications and Trusty URIs. The nanopublication framework [35] provides a standardized way to publish small, granular scientific claims with cryptographically signed provenance. Each nanopublication consists of three named graphs [36]: an assertion graph (the claim being made), a provenance graph (metadata about the claim’s origin), and a publication-info graph (metadata about the nanopublication itself). The Trusty URI mechanism [37] provides content-addressed identifiers via Base64-encoded SHA-256 hashes, ensuring that any modification to a nanopublication’s content changes its URI and thus invalidates the published artifact. This achieves decentralized verifiable publishing—a goal shared with ADL Lite’s append-only EventChain—where published claims are immutably bound to their provenance and can be independently verified without reliance on a central authority.

Key differences between the two approaches reveal complementary rather than competing design goals. First, nanopublications focus on *atomic claims* with single assertions (e.g., “gene X is associated with disease Y”), while ADL Lite models full *concept lifecycles* as multi-event chains spanning registration, validation, deprecation, forking, and archival. Second, nanopublications use RSA digital signatures for author authentication, binding each assertion to a verifiable cryptographic identity; ADL Lite Phase 1 currently relies on string identifiers for actors, with LD-Proof integration planned for Phase 3 to achieve equivalent signature-based authentication. Third, nanopublications require RDF/SPARQL infrastructure for publication, querying, and verification, while ADL Lite operates over Markdown files in standard Git repositories without a triple store. Nanopublications thus target the publication of individual scientific assertions to decentralized servers, while ADL Lite targets the governance of evolving concept definitions within multi-agent document-native workflows.

Linked Data Proofs and RDF Dataset Canonicalization. The W3C Verifiable Credential Data Integrity specification [38]—formerly known as Linked Data Proofs—describes mechanisms for ensuring the authenticity and integrity of constrained digital documents using cryptography, especially through digital signatures over canonicalized RDF datasets. The specification defines a vocabulary for proof types, verification methods, and canonicalization algorithms, and is accompanied by concrete cryptographic suites including EdDSA (Ed25519Signature2020) and ECDSA variants [39]. Underpinning this signing capability is the RDF Dataset Canonicalization algorithm (URDNA2015) [40], which produces a deterministic, byte-for-byte consistent serialization of an RDF

dataset regardless of the original ordering of triples or blank node labels—a prerequisite for signature verification over RDF data.

This technology provides the authenticated provenance that ADL Lite currently lacks: the ability to bind actor identities to events through cryptographic signatures rather than string identifiers. An LD-Proof over an ADL Lite event would enable any verifier to confirm that a specific actor (identified by a Decentralized Identifier, or DID) authored a specific event, without requiring trust in the storage medium or the party presenting the chain. The reviewer specifically recommended this as a path forward for authenticated event logging in ADL Lite. The current status of LD-Proofs is a W3C Recommendation (May 2025), with implementations including `vc-js` and `jsonld-signatures` that support multiple cryptographic suites. ADL Lite’s future Phase 3 includes LD-Proof integration for authenticated events, targeting `Ed25519Signature2020` as the initial suite given its balance of security and computational efficiency.

RDF-star and Named Graphs. RDF-star [41]—now incorporated into the RDF 1.2 specification [42]—provides statement-level annotation by enabling triples to function as subjects or objects of other triples (so-called “quoted” or “embedded” triples), without the syntactic verbosity of explicit reification nodes or the artificial graph proliferation of single-triple Named Graphs. The annotation syntax `{ | :source :bob | }` in Turtle-star allows concise expression of metadata about individual statements, such as provenance, certainty, or temporal validity, directly adjacent to the statement being annotated. Named Graphs [36], standardized as part of SPARQL 1.1, allow bundling triples into named containers with their own provenance metadata, supporting graph-level rather than statement-level provenance tracking.

Both technologies address the “who said what when” problem that ADL Lite’s EventChain solves through a different architectural path. RDF-star enables statement-level annotations within a single graph; Named Graphs enable graph-level provenance partitioning; ADL Lite’s EventChain enables event-level provenance sequencing with cryptographic integrity guarantees. ADL Lite’s L3 relation blocks—which encode `source-relation-target` triples within Markdown fenced code blocks—can be viewed as a Markdown-native form of RDF triples, with each EventChain entry providing statement-level provenance analogous to an RDF-star annotation. The key distinction is that RDF-star annotations are syntactic constructs within an RDF graph (with no built-in integrity protection), while ADL Lite EventChain entries are cryptographically linked events in an append-only sequence (with built-in tamper detection but no native RDF serialization in Phase 1). Phase 3 work will include an RDF-star export serializer that materializes ADL Lite relation blocks as quoted triples with EventChain-derived annotations, bridging these two provenance models.

Event-Centric Ontologies The ontologies surveyed in the preceding subsections vary in their treatment of events: some model events as first-class entities central to their philosophical commitment, while others treat events as secondary constructs or ignore them entirely. This subsection examines three event-centric approaches—CIDOC CRM, SEM/LODE, and RDF Stream Processing—that share ADL Lite’s

fundamental premise that *change in the world is best understood through events* rather than through direct property mutations of persistent objects, and positions ADL Lite’s EventChain model within this intellectual lineage.

CIDOC CRM (Conceptual Reference Model). The CIDOC Conceptual Reference Model[44] is an ISO 21127 standard for cultural heritage information integration, developed by the International Committee for Documentation of the International Council of Museums (ICOM-CIDOC). CIDOC CRM is fundamentally event-centric: it models reality through the **E5 Event** class that brings together **E39 Actor** participants with **E18 Physical Thing** objects in spatiotemporal contexts. The ontology’s core philosophical commitment—expressed through its formal axioms and extensive documentation—is that change in the world is understood through events, not through direct property mutations of persistent objects. An actor painting a canvas, a museum acquiring an artifact, or an archaeological excavation uncovering a fragment are all modeled as events with their own identity, temporal extent, spatial location, and participant roles, rather than as implicit state changes to the participating entities.

This ontological stance directly parallels ADL Lite’s design: ADL Lite models concept lifecycle changes (registration, validation, deprecation, archival) as explicit events in an append-only chain rather than as mutable field updates to a concept record. A concept’s **status** is not a stored property but a value *derived* from the history of events affecting that concept, mirroring CIDOC CRM’s treatment of an object’s current custodian as derivable from the sequence of custody-transfer events. ADL Lite’s **previous_event_id** cryptographic linkage serves an analogous function to CIDOC CRM’s temporal and causal ordering axioms, ensuring that the event sequence is both immutable and verifiable.

The key difference lies in purpose and scope. CIDOC CRM is *descriptive*: it documents what happened in the past (or what is believed to have happened), providing a comprehensive upper ontology for cultural heritage documentation with 81 classes and over 200 properties. ADL Lite is *operational*: it enforces what *can* happen through declarative preconditions (**PreconditionRule**), governing the future evolution of concept definitions rather than documenting their past. CIDOC CRM supports inference over historical event patterns; ADL Lite supports runtime validation of proposed action transitions. ADL Lite shares CIDOC CRM’s philosophical commitment to event centrality but applies it to a different domain—operational ontology lifecycle governance rather than cultural heritage documentation—and with a different deployment model—plain-text Markdown files rather than RDF triple stores.

SEM (Simple Event Model) and LODE (Linked Open Descriptions of Events). While CIDOC CRM provides a comprehensive upper ontology, the Simple Event Model (SEM)[46] and Linked Open Descriptions of Events (LODE)[47] represent lightweight alternatives designed for publishing event data on the Semantic Web. SEM provides a minimal vocabulary comprising **sem:Event**, **sem:Actor**, **sem:Place**, and **sem:Time** for describing events without committing to a heavy upper ontology. Its design principle is epistemic modesty: SEM makes minimal ontological commitments, allowing publishers to describe what they know

about an event without requiring complete provenance or exhaustive participant enumeration. LODE extends SEM with Dublin Core compatibility (`dc:title`, `dc:description`, `dc:date`) and SKOS-based event type hierarchies, enabling integration with existing Linked Data publishing workflows[47].

Both SEM and LODE are *descriptive* ontologies for event publishing: they enable data providers to expose event descriptions in RDF for consumption, linking, and querying across the Web of Data. They do not provide constraint mechanisms for governing which events can occur, nor do they offer integrity guarantees for event sequences. ADL Lite complements these descriptive frameworks with an *operational* layer: the EventChain is not merely a published description of events but a tamper-evident, precondition-governed mechanism for controlling concept lifecycle transitions. An SEM publisher might describe that a concept was validated by agent X at time T; ADL Lite’s EventChain *enforces* that the validation can occur only if the concept is in **PROPOSED** status, the agent’s scope permits validation actions, and all preconditions evaluate to true—then cryptographically binds the event into an immutable sequence. SEM and LODE enable event data interoperability; ADL Lite enables event-driven concept governance with cryptographic integrity.

RDF Stream Processing (RSP). The W3C RDF Stream Processing Community Group defined models and query languages for continuous processing over timestamped RDF triples arriving in streams[48]. Systems such as C-SPARQL[49] and CQELS[50] extend SPARQL with windowing operators (**RANGE**, **STEP**) and continuous query semantics, enabling real-time pattern detection over streaming RDF data. In RSP systems, events are modeled as timestamped triples (or triple graphs) arriving in potentially unbounded streams; queries register continuous patterns (e.g., “report all transactions exceeding 10,000 within a 5-minute window”) that are evaluated incrementally as new data arrives.

ADL Lite’s EventChain can be viewed as a *materialized* RDF stream: each event appends triples (encoded as YAML front matter fields and L3 relation blocks) to a persistent, cryptographically linked chain, enabling retrospective provenance queries rather than real-time pattern detection. Where C-SPARQL processes streaming triples in temporal windows for immediate alerting, ADL Lite processes historical events in an append-only chain for forensic reconstruction and status derivation. This distinction reflects a fundamental difference in target workload: RSP systems optimize for low-latency detection over high-velocity streams, while ADL Lite optimizes for integrity and auditability over low-velocity, high-stakes concept lifecycles. Nevertheless, the RSP formalization of events as timestamped RDF triples provides a conceptual bridge to future Phase 3 work: an RSP-compatible export of ADL Lite EventChains would enable continuous query patterns over concept lifecycle events (e.g., “alert when any concept receives three **CHALLENGE** events within 24 hours”), extending retrospective audit to proactive monitoring.

Conflict-Free Replicated Data Types for Collaborative Ontology Authoring Multi-agent ontology authoring introduces a fundamental concurrency challenge: when multiple agents modify concept definitions simultaneously—whether through

concurrent commits in separate Git branches or through simultaneous edits in a real-time collaborative session—their changes must be reconciled without losing information or violating consistency. Conflict-Free Replicated Data Types (CRDTs)[51] provide a theoretical and practical foundation for this reconciliation, ensuring that concurrent updates converge to a consistent state without requiring centralized coordination or locking. This subsection surveys CRDT literature relevant to ADL Lite’s multi-agent authoring model and identifies opportunities for future integration.

Automerge. Automerge[52] is a JSON CRDT library that provides automatic merge semantics for collaborative document editing. Automerge uses operation-based CRDTs with a novel “multi-value register” semantics that preserves all concurrent edits rather than choosing a single winner. When two agents concurrently modify the same field—for example, one agent updates a concept’s **description** while another updates its **scope**—Automerge merges both changes automatically, preserving the intent of both editors. In cases of true conflict (both agents modify the *same* field concurrently), Automerge preserves both values as a multi-value register rather than silently applying last-write-wins (LWW), enabling downstream resolution by application logic or human review.

This behavior is directly relevant to ADL Lite’s fork model. When two agents concurrently edit the same concept—for instance, both proposing alternative **description** values for a concept in **CHALLENGED** status—ADL Lite’s current Phase 1 implementation applies last-write-wins to the event chain, potentially discarding one agent’s contribution. Automerge would preserve both versions as concurrent events, allowing the multi-agent quorum mechanism to evaluate both proposals rather than arbitrarily rejecting one. Adopting CRDT semantics for event chain merging is a promising future direction: ADL Lite EventChain entries could be modeled as operations in a sequential CRDT, with the `previous_event_id` hash chain providing causal ordering and the CRDT layer providing automatic convergence for concurrent appends. This hybrid approach—cryptographic integrity from hash chaining plus automatic merge semantics from CRDTs—would eliminate the LWW limitation without sacrificing tamper evidence.

Yjs. Yjs[53] is a widely deployed CRDT framework for real-time collaborative editing, optimized for shared text documents. Yjs provides efficient delta encoding, incremental update propagation, and awareness protocols (cursor positions, user presence) for concurrent editing of textual content. Its underlying YATA algorithm[54] for list CRDTs achieves sub-linear space overhead for concurrent insertions and deletions, making it suitable for large documents with many concurrent editors.

While Yjs targets text documents, its underlying CRDT principles are applicable to event sequence merging in ADL Lite. An ADL Lite EventChain is, at its core, an ordered sequence of event entries—structurally analogous to a list of text operations. The YATA algorithm’s list CRDT semantics could be adapted to merge concurrent event sequences: when two agents append events to the same concept’s chain in different branches, the YATA merge would produce a

convergent ordering that preserves all events and their causal relationships. The primary adaptation required would be extending YATA’s content-based ordering with ADL Lite’s hash-based causal linking (`previous_event_id`), ensuring that cryptographic integrity and CRDT convergence operate in concert. This integration remains future work but demonstrates that ADL Lite’s document-native, Git-backed architecture is compatible with state-of-the-art collaborative editing technology.

CRDT-Ontology Intersection. Recent research has explored applying CRDTs to knowledge graph editing, enabling concurrent multi-agent ontology evolution without centralized coordination. Chesa et al.[55] demonstrated a CRDT-based approach to collaborative RDF editing where concurrent property updates converge automatically while preserving graph consistency invariants. Parallel work on distributed ontology editing platforms[57] has explored CRDTs for merging concurrent class definition changes, property range modifications, and instance assertions in OWL ontologies. These approaches share ADL Lite’s motivation—enabling multiple agents to evolve an ontology concurrently—but target full RDF/OWL graph editing rather than ADL Lite’s lightweight Markdown-native event chains.

ADL Lite’s Git-native design already provides branch-based concurrent authoring: agents work in isolated branches, propose changes through event chains, and merge through pull requests. What CRDTs would add is *automatic merge semantics for event chains*: rather than requiring manual conflict resolution when branches diverge, a CRDT-backed EventChain would converge automatically while preserving the cryptographic integrity guarantees of the hash chain. The envisioned integration path is a hybrid model where (1) each branch maintains a local CRDT-backed event sequence, (2) cross-branch merges use CRDT convergence to produce a unified ordering, and (3) the resulting merged sequence is cryptographically hashed to produce a tamper-evident canonical chain. This would extend ADL Lite’s current Git-based concurrency model with real-time collaborative capabilities while maintaining the forensic traceability that is central to its design.

2.4 Positioning

Comparison Across Seven Dimensions **Table 1. Positioning of ADL Lite** against three reference approaches—Palantir FDE, OWL/SPARQL pipelines, and Markdown-only knowledge bases—across seven dimensions salient to operational ontology deployment.

Dimension	Palantir FDE	OWL/SPARQL	Markdown-only	ADL Lite
Deployment	Enterprise	High (triple	Low (Git repo)	Very Low (pip
Cost	($M+$)	store + reasoner)		install + Git)

Dimension	Palantir FDE	OWL/SPARQL	Markdown-only	ADL Lite
LLM Authorship	Not designed	Requires ontology expertise	Informal (free-text)	Native (YAML + Markdown)
Integrity Guarantees	Platform-managed	Reasoner-based consistency	None	SHA-256 EventChain (append-only)
Action Preconditions	TypeScript functions	OWL/SWRL rules	None	Declarative YAML + SSA validation
Consensus Mechanism	Centralized (admin UI)	N/A (single authority)	N/A (no coordination)	Multi-agent quorum + event history
Scalability	Billions of objects	Millions of triples	Thousands of notes	Pilot: 20 concepts; target: 10^4
Standards Alignment	Proprietary (Palantir OMS)	Full W3C stack (OWL, RDF, SPARQL, SHACL)	None	Partial (PROV-O mappable, SHACL-integrable, no native RDF serialization in Phase 1)

ADL Lite: RDF-Like Triples in Markdown, Machine-Checkable Coordination Without the Triple Store Palantir FDE delivers industrial-strength operational semantics through a tightly integrated platform, but at deployment cost and accessibility barriers that exclude small teams, open-source projects, and LLM-native workflows. OWL/SPARQL pipelines offer unmatched formal expressiveness through description logic semantics and automated reasoning, yet require specialized expertise and infrastructure that confine them to research and enterprise contexts with dedicated ontology engineering functions. Markdown-based tools democratize knowledge capture through plain-text authoring and Git-based collaboration, but provide no machine-checkable guarantees against schema drift, conflicting definitions, or unauthorized modification.

ADL Lite occupies a deliberate middle ground among these alternatives. L3 relation blocks provide RDF-like triples editable in ordinary Markdown; L1 YAML scalars encode datatype properties with zero overhead beyond the YAML front matter already conventional in static site generators and documentation tools. SSA (Status-State-Action) validation, scope ACL, and the append-only EventChain supply machine-checkable coordination primitives—status transitions with preconditions, visibility boundaries, and tamper-evident history—without requiring a triple store, OWL reasoner, or platform-specific infrastructure. The comparison is not one of replacement but of complementarity: ADL Lite targets

contexts where the semantic precision of OWL and the operational power of Palantir are desirable but the deployment investment is disproportionate to the available resources.

This positioning has direct implications for multi-agent systems. Palantir’s AIP demonstrates that agents can operate effectively within an ontology, but presupposes a fully staffed, platform-managed ontology layer[14]. OWL supports agent reasoning through description logic services but assumes centralized ontological authority. Markdown imposes no authority structure but offers no coordination primitives at all. ADL Lite enables a new mode: multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes such decentralized authorship accountable. Phase 1 pilots ($n = 20$ AML concepts) demonstrate mechanistic gains in lifecycle traceability, scope isolation, and L3-sensitive retrieval rather than claims of automated ontological inference. We argue that a lightweight, document-native semantic layer—formalized in future work through a centralized OntologyManager schema—can deliver multi-agent auditability at deployment cost closer to Git-backed notes than enterprise knowledge graphs.

Compared to PROV-O-based provenance logs, ADL Lite provides built-in cryptographic integrity verification rather than provenance description alone; PROV-O excels at modeling *what happened*, while ADL Lite’s EventChain additionally guarantees *that the record of what happened has not been tampered with*. Compared to nanopublications, ADL Lite targets concept-level lifecycle governance (registration through validation to deprecation and archival) rather than atomic claim publishing; nanopublications excel at immutable assertion dissemination, while ADL Lite excels at governing the evolution of definitions over time. Compared to CIDOC CRM, ADL Lite shares the philosophical commitment to event-centric modeling but targets operational enforcement of lifecycle transitions rather than descriptive documentation of cultural heritage events. Compared to SEM/LODE, ADL Lite provides precondition-governed event validation and cryptographic integrity rather than lightweight event description for Linked Data publishing. Compared to Git-only pipelines, ADL Lite adds structured action preconditions (the **Comparator**-based **PreconditionRule** system), status derivation from event history (computing **status** and **confidence** from the chain rather than storing them as mutable fields), and a closed action registry that eliminates runtime code injection. The gap that ADL Lite fills is the intersection of four properties that no existing approach combines: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. PROV-O provides (a) only when combined with RDF authoring tools and lacks (b) natively; nanopublications provide (b) for published claims but lack (a), (c), and (d); CIDOC CRM provides event-centric modeling but requires heavyweight RDF infrastructure; Git provides (a) and (d) but lacks (b) and (c); Palantir provides (b), (c), and (d) only at enterprise scale with proprietary tooling. It is this unique

combination—document-native, tamper-evident, precondition-governed, and frictionlessly deployable—that motivates ADL Lite as a complementary contribution to the provenance-aware knowledge representation landscape.

3 Architecture

3.1 Philosophical Foundation: Event-First Ontology

The architectural design of ADL Lite is grounded in an inversion of conventional ontology engineering. Traditional operational ontologies, including Palantir FDE and OWL-based knowledge graphs, adopt an *object-first* stance: Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is appended as an operational afterthought [56][6]. This ordering reflects a metaphysical commitment to the primacy of objects—entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes. The engineering consequence is that every concept carries mutable state fields subject to race conditions, inconsistent updates, and silent divergence between stored values and their logical histories.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein’s *Tractatus* §1.1—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*”—the system treats Action Type as the fundamental ontological primitive [60][2]. On this view, an object exists only in virtue of the events in which it participates. A “validated concept” is not a concept whose `status` field contains the string `validated`; it is a concept whose `EventChain` contains a `VALIDATE` event authored by an agent satisfying the required confidence threshold. Status is *never stored*; it is *always computed* from the chain. Confidence is not a mutable scalar subject to direct assignment; it is derived from the sequence of `VALIDATE` events and their associated confidence payloads. Validators are not maintained as a list field vulnerable to stale entries; they are accumulated from the `actor` fields of lifecycle events. The concept does not *have* a history—it *is* its history [3][7].

This philosophical commitment translates into a concrete engineering discipline with three governing principles. First, every mutation of ontological state is captured as an explicit event; there are no implicit state changes. Second, events are append-only and cryptographically linked, forming an auditable chain that reconstructs the full provenance of any concept from its genesis to its current state. Third, all derived properties—status, confidence, validators—are computed on demand through deterministic functions over the event sequence. This design eliminates an entire class of consistency bugs: a front matter field cannot diverge from the event chain because there is no independent storage of mutable state. The chain constitutes the sole source of truth, and any snapshot derived from it is recomputed on every access. The event-first approach bears a structural resemblance to command-query responsibility segregation (CQRS) and event sourcing patterns from systems architecture, in which system state is reconstructed by replaying an immutable event log rather than querying mutable records [11][13]. However, ADL Lite applies this pattern at the level of ontological concepts rather than application state, using cryptographic hashing to provide integrity

guarantees that general-purpose event stores typically delegate to database access controls.

3.2 Document Model: L1/L2/L3/L4

ADL Lite organizes concept documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 2 summarizes the document model.

As shown in Table 2, the L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the EventChain, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event history in the source document—a diagnostic signal that the document predates the action-block semantics, rather than a data inconsistency in the derived model. The L2 layer consists of the Markdown body: human- and LLM-readable prose subject to the Singular Subjectivity Assertion (SSA), which prohibits fuzzy referents—including pronouns such as “this,” “that,” “it,” and their equivalents—that would compromise cross-agent alignment [8]. The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between concepts (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support with typed data references, and `adl:seal` for formal assertions with proof references. The L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects.

Table 2: The L1–L4 Document Model. Each layer contributes distinct semantics to the concept document, with event types mapping L3 and L4 content to EventChain entries.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot, not source of truth)	SNAPSHOT
L2	Markdown body	Human/LLM — narrative	—
L3	<code>adl:relation</code> , <code>adl:evidence</code> , <code>adl:seal</code>	Typed semantic assertions	RELATE, EVIDENCE, SEAL

Layer	Syntax	Role	Event Type
L4	<code>adl:action</code>	Typed actions with structured preconditions	REGISTER, VALIDATE, DEPRECATE, ...

L1 caching and recomputation semantics. The description of L1 as a *derived snapshot* requires clarification regarding caching policy. L1 front matter is derived **once** when the document is loaded: the Markdown file is parsed, and the YAML front matter is read to initialize an `ADLDocument` instance. The `EventChain` is constructed from L4 action blocks at load time, not on every access. Status and confidence are computed from the chain on first access and then **cached** within the `ADLDocument` instance; subsequent reads return the cached values without recomputation. The `snapshot()` method serializes the cached state back to L1 YAML for persistence (e.g., when writing the document to disk). This means that, in practice, L1 front matter is not recomputed on every read; rather, it is a write-time serialization of the cached derived state. The term “derived snapshot” therefore signifies that L1 is **not** the source of truth—the chain is—but it does not imply eager recomputation on every access. SHA-256 hashes within the `EventChain` are computed over **canonical JSON serialization** (sorted keys, no whitespace) ensuring platform- and locale-independent hash values regardless of the host Python environment.

The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log. L3 relation blocks provide RDF-like triples that remain editable in ordinary text editors, while L4 action blocks supply machine-checkable coordination primitives without requiring a triple store or OWL reasoner [9]. This multi-modal document model is central to ADL Lite’s deployment philosophy: because concepts are stored as ordinary Markdown files in Git repositories, they inherit version control, diff-based review, and branch-based experimentation without additional infrastructure.

SSA limitations and relaxation modes. The Singular Subjectivity Assertion is a Phase 1 strictness measure designed to ensure that L2 narrative content is interpretable across agents without disambiguation of pronouns or deictic references. It is intentionally strict: every noun must be explicit, every reference unambiguous. While this discipline supports cross-agent alignment experiments—where multiple LLM agents must parse and reason over the same narrative without grounding errors—it is not intended as a universal constraint on all document authoring. Human-authored narrative frequently employs pronouns appropriately, and requiring full nominalization can produce stilted prose that impedes rather than aids comprehension. We therefore acknowledge SSA as a configurable strictness level rather than an invariant property of the ADL Lite document model. Two relaxation modes are anticipated for future work: *SSA-strict* (the current behavior, enforcing explicit reference for all agent-authored content in multi-agent settings) and *SSA-lax* (permitting natural pronoun use for

human-authored narrative, with optional pronoun resolution via an LLM-based disambiguation pass when cross-agent alignment is required). The SSA validator currently operates as a static text-analysis rule; transitioning to a configurable severity level with automated pronoun-coreference resolution would broaden applicability without weakening the cross-agent alignment guarantee in contexts where it is needed.

3.3 EventChain: Core Data Structure

The EventChain is the fundamental data structure of ADL Lite. Each ontological concept is represented by exactly one EventChain: an append-only sequence of cryptographically linked events. The event model is defined as follows.

An *event* e is a structured record comprising nine fields: **event_id** (a UUID-derived unique identifier), **concept_id** (the concept to which the event belongs), **event_type** (the action that occurred, drawn from a closed set of lifecycle, assertion, and communication types), **actor** (the entity causing the event), **reasoning** (justification for the event), **timestamp** (ISO-8601 occurrence time), **payload** (event-specific key-value data), **previous_event_id** (cryptographic link to the preceding event), and **hash** (SHA-256 digest of the event’s serialized content). Formally:

$$e = (\text{event_id}, \text{concept_id}, \tau, \text{actor}, \text{reasoning}, t, \text{payload}, \text{prev_id}, H)$$

where $\tau \in \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}, \text{RELATE}, \text{EVIDENCE}, \text{SEAL}, \text{AN}\}$, t is an ISO-8601 timestamp, and $H = \text{SHA_256}(\text{serialize}(e \setminus \{H\}))$ is the cryptographic hash computed over all fields except the hash itself.

Canonical Serialization for Cryptographic Hashing To ensure that SHA-256 hashes are deterministic and platform-independent, ADL Lite defines a canonical serialization procedure that is applied before hashing. The procedure converts the event record to a JSON string with the following constraints:

1. **Field ordering.** Fields are serialized in the fixed order: **event_id**, **concept_id**, **event_type**, **actor**, **reasoning**, **payload**, **previous_event_id**. The timestamp field is **excluded** from the hash input (see Section 3.6 for the clock-skew rationale). The hash field is naturally excluded because it is the value being computed.
2. **Recursive key sorting.** All JSON object keys are sorted lexicographically. The payload dictionary is recursively sorted so that nested objects have their keys ordered deterministically regardless of insertion order.
3. **Whitespace elimination.** No spaces, newlines, or indentation are included in the serialized output. The canonical form is a single line of JSON.
4. **UTF-8 encoding.** The JSON string is encoded as UTF-8 before hashing, ensuring consistent byte representation across platforms.

5. **Number normalization.** All floating-point values in payload are rounded to six decimal places to prevent precision-based hash differences across hardware platforms and Python interpreter versions.

Formally, let `serialize_canonical(e)` produce a byte sequence b such that:

$$b = \text{UTF-8}(\text{JSON-SORTED}(e \setminus \{\text{hash}, \text{timestamp}\}))$$

where `JSON-SORTED` denotes JSON serialization with lexicographically sorted keys at every nesting level and no whitespace, and $\setminus$ denotes record subtraction (field removal). The event hash is then:

$$H(e) = \text{SHA-256}(b)$$

Proposition 1 (Hash Determinism). For any two event records e_1 and e_2 , if $e_1 \setminus \{\text{hash}, \text{timestamp}\} = e_2 \setminus \{\text{hash}, \text{timestamp}\}$, then $H(e_1) = H(e_2)$.

Proof. The canonical serialization `serialize_canonical` is a pure function: it removes the hash and timestamp fields, recursively sorts all object keys, eliminates whitespace, rounds floating-point values to six decimal places, and encodes the result as UTF-8. Each of these operations is deterministic and total over valid event records. Therefore, equal inputs produce identical byte sequences b , and since `SHA-256` is itself a deterministic function over byte sequences, equal inputs yield equal hashes. $\square$

Proposition 2 (Malleability Resistance). Two semantically identical events produce identical hashes regardless of the source format (YAML front matter, Markdown body, or programmatic construction).

Proof. All source formats are parsed into the same internal event record structure before canonicalization. The canonical serialization discards format-specific artifacts (YAML indentation, Markdown syntax, insertion-order differences in programmatically constructed dictionaries). By Proposition 1, identical internal records yield identical hashes. $\square$

Proposition 3 (Timestamp Independence). The hash of an event does not depend on its timestamp field.

Proof. The timestamp field is explicitly removed by the set subtraction $e \setminus \{\text{hash}, \text{timestamp}\}$ prior to serialization. Therefore, two events that differ only in their timestamp values produce identical byte sequences b and hence identical hashes. $\square$

Remark on hash chain integrity. The exclusion of timestamp from the hash input is a deliberate design choice. Event ordering is determined entirely by the append sequence (i.e., the `previous_event_id` linkage), not by timestamp values. This ensures that clock skew—whether benign (agent with misconfigured clock) or adversarial (intentional backdating)—cannot cause hash mismatches or chain forks. The timestamp field serves informational purposes only: human readability, coarse temporal queries, and audit log display. It is not used for verification or ordering.

Remark on payload normalization. The six-decimal rounding of floating-point values in payload is sufficient for confidence values expressed in the range

$[0, 1]$ and prevents hash divergence caused by platform-specific floating-point representations (e.g., IEEE 754 double-precision vs. extended precision). Domains requiring higher precision may configure the rounding parameter in `adl_core_ontology.yaml`; however, all participants in a shared repository must agree on the same rounding policy to maintain hash consistency.

Clock Skew and Timestamp Handling The EventChain architecture makes a strict separation between **event ordering** (a cryptographic property) and **event timing** (an informational annotation). This separation enables ADL Lite to tolerate arbitrary clock skew across agents while maintaining deterministic chain integrity.

Timestamp exclusion from the hash input. The timestamp field of every event is explicitly excluded from the SHA-256 hash computation, as specified in Section 3.3.1. Formally, for any event e :

$$H(e) = \text{SHA-256}(\text{serialize_canonical}(e \setminus \{\text{hash}, \text{timestamp}\}))$$

This means that two events that differ only in their timestamp values are cryptographically indistinguishable: they produce identical hashes and can therefore be substituted for one another without breaking the chain. The practical consequence is that clock skew—whether caused by misconfigured NTP, time-zone differences, or adversarial backdating—cannot cause hash mismatches or invalidate chain integrity.

Event ordering by append sequence, not timestamp. The temporal order of events within a single EventChain is determined entirely by the `previous_event_id` linkage, which forms a cryptographic linked list. An event e_i precedes e_j in the chain if and only if there exists a sequence of `previous_event_id` links from e_j back to e_i . This is a **happens-before** relation defined by cryptographic reference, not by timestamp comparison.

Formally, define the happens-before relation $\prec$ on events in a chain:

$$e_i \prec e_j \iff \exists k_1, k_2, \dots, k_m : e_j.\text{prev_id} = H(e_{k_1}) \wedge e_{k_1}.\text{prev_id} = H(e_{k_2}) \wedge \dots \wedge e_{k_m}.\text{prev_id} = H(e_i)$$

The relation $\prec$ is a strict partial order: it is irreflexive (no event happens before itself), transitive (by composition of hash links), and asymmetric (the cryptographic hash function is collision-resistant, so $e_i \prec e_j$ implies $e_j \not\prec e_i$). The timestamp field plays no role in defining $\prec$.

Non-monotonic timestamp tolerance. Because timestamps are not used for ordering or hashing, ADL Lite tolerates arbitrary non-monotonicity in timestamp values. An event may carry a timestamp earlier than its predecessor, later than its successor, or any other value, without affecting chain validity. This is essential for distributed settings where agents may have unsynchronized clocks.

Timestamp role: informational only. The timestamp field serves three informational purposes:

1. **Human readability.** Timestamps provide coarse temporal context for audit logs and chain visualization.
2. **Temporal querying.** The `snapshot_at(t)` operation (not implemented in Phase 1) would reconstruct the state of a concept as it existed at a given wall-clock time by filtering events by timestamp. This is a best-effort convenience feature, not a verification mechanism.
3. **Debugging and diagnostics.** Timestamp anomalies (e.g., an event timestamped hours before its predecessor) can signal clock misconfiguration or delayed event propagation.

At no point is the timestamp field used for integrity verification, status derivation, or consensus.

Preventing chain branching. If two events are appended concurrently with the same `previous_event_id`, the EventChain enforces a **single-successor rule**: the first event to be accepted by the append operation becomes the legitimate successor, and the second is rejected. This prevents the chain from branching into a tree structure. Formally:

$$\text{append}(E, e_{\text{new}}) = \begin{cases} E \oplus [e_{\text{new}}] & \text{if } e_{\text{new}}.\text{prev_id} = H(\text{last}(E)) \text{ and } \text{successor}(\text{last}(E)) = \perp \\ \text{reject} & \text{otherwise} \end{cases}$$

where $\text{successor}(e)$ denotes the unique event (if any) whose `previous_event_id` equals $H(e)$. The single-successor rule ensures that the EventChain remains a linear sequence rather than a directed acyclic graph.

Future work: logical clocks for distributed settings. The current design assumes a single append point per chain (e.g., one agent or a coordinated merge process). For fully distributed settings without a trusted timestamp authority or centralized append point, ADL Lite Phase 2 will introduce optional logical clocks. Two mechanisms are planned:

1. **Lamport timestamps** [17] for capturing the happens-before relation among events across distributed agents. Each agent maintains a monotonic counter, incremented on every local event and propagated with messages. Lamport clocks provide a partial order consistent with the causal relation but cannot distinguish concurrent events.
2. **Vector clocks** [18] for identifying concurrent events precisely. A vector clock assigns a vector of counters (one per agent) to each event, enabling the detection of causality, concurrency, and potential conflicts. Vector clocks are more expressive than Lamport clocks but incur $O(n)$ space overhead for n agents.

The introduction of logical clocks will be backward-compatible: events without vector clock annotations continue to validate under the current `previous_event_id`-based ordering. Logical clocks augment, rather than replace, the cryptographic linkage.

The EventChain supports five primary operations. The `append(event)` operation links a new event to the chain by setting `previous_event_id` to the hash of the preceding event and recomputing the hash of the new event over its full serialized content. The `verify_integrity()` operation traverses the chain from genesis to tip, recomputing each event’s hash and validating that it matches the stored value; it further confirms that each `previous_event_id` correctly references the preceding event’s hash, establishing a chain of cryptographic trust from the most recent event back to the genesis event. The `status` property is computed by scanning the chain for the most recent lifecycle event (REGISTER, VALIDATE, DEPRECATE, FORK, or ARCHIVE) and returning the corresponding status; if no lifecycle event is present, the default status `provisional` is returned. The `confidence` property aggregates validate event payloads according to domain-specific rules. The `validators` property accumulates unique `actor` values from lifecycle events. The `history()` method returns the full audit trail in temporal order; `snapshot()` derives an `ADLFrontMatter` instance representing the current computed state for serialization to L1 YAML.

Integrity verification operates in $\mathcal{O}(n)$ time for a chain of n events. Each verification recomputes SHA-256 hashes for all events and checks link consistency. Because events are append-only and hashes cover all prior content—including the previous event’s hash through the `previous_event_id` field—any modification to historical data, whether payload tampering, event reordering, link breakage, or concept ID mismatch, is detectable as a hash mismatch at the point of first alteration. The verification process is deterministic and side-effect-free, enabling it to be run at any time without modifying chain state. For a chain containing 168,508 events (the maximum observed in the IBM AML dataset), verification completes in linear time proportional to the chain length, with each hash computation imposing negligible constant overhead on modern hardware.

3.4 Action Executor with Structured Preconditions

The ActionExecutor enforces that all lifecycle transitions and side effects satisfy declarative precondition rules defined in `adl_core_ontology.yaml`. The registry specifies nine core actions. The `register` action transitions a concept from null to `provisional` with no preconditions. The `validate` action transitions from `provisional` to `validated`, requiring `confidence  $\geq$  0.5` and `status = provisional`. The `fork` action transitions from `provisional` to `forked`, requiring `status = provisional`. The `deprecate` action transitions from `validated` to `deprecated`, requiring `status = validated`. The `archive` action transitions from `deprecated` to `archived`, requiring `status = deprecated`. The `announce`, `publish`, `sync_dashboard`, and `listen` actions effect no status transitions but require the presence of `chat_id`, `wiki_space`, `sheet_id`, and `feedback_file` parameters, respectively.

Precondition rules are modeled through a `PreconditionRule` structure containing three fields: `field` (the `ADLFrontMatter` field name to evaluate), `comparator` (a typed enumeration drawn from a closed set), and `value` (the expected value for

comparison). The `Comparator` enumeration defines eight operators: EQ (equality), NEQ (inequality), GT (greater than), GTE (greater than or equal), LT (less than), LTE (less than or equal), IN (membership in a specified set), and EXISTS (non-null presence of the field). The `check(fm)` method evaluates a precondition rule against a front matter instance by extracting the specified field and applying the comparator through a statically typed dispatch.

This design replaces runtime `eval()`—a common but unsafe pattern in action dispatch systems that introduces arbitrary code execution vulnerabilities—with statically typed, parse-time-validatable Pydantic models [10]. Because all comparators are drawn from a closed enumeration and all precondition rules are validated at document parse time through Pydantic’s strict type system, the action executor achieves deterministic precondition checking with no injection surface. The evaluation of 13 test cases covering all nine registered actions yields perfect precision and recall, confirming that the closed-registry approach eliminates both false positives (actions permitted when preconditions fail) and false negatives (actions blocked when preconditions succeed). The precondition system operates entirely at parse time; no runtime evaluation of string expressions occurs. The action registry is itself versioned through `adl_core_ontology.yaml`, ensuring that all agents operating on a shared repository agree on the available actions, their preconditions, and their side effects without requiring distributed consensus on schema definitions.

3.5 Consensus and Status Transitions

The `ConsensusEngine` governs the lifecycle of ontological concepts through a finite status machine with five states and twelve valid transitions. Table 3 specifies the complete transition matrix.

Table 3: Status Transition Matrix. The `ConsensusEngine` enforces valid transitions through the fork workflow, in which a forked concept spawns a new `EventChain` with a fresh `adl_id` starting at `provisional`.

Source Status	Valid Target States
provisional	validated, deprecated, forked, archived
validated	deprecated, forked, archived
forked	validated, deprecated, archived
deprecated	archived
archived	(none — terminal)

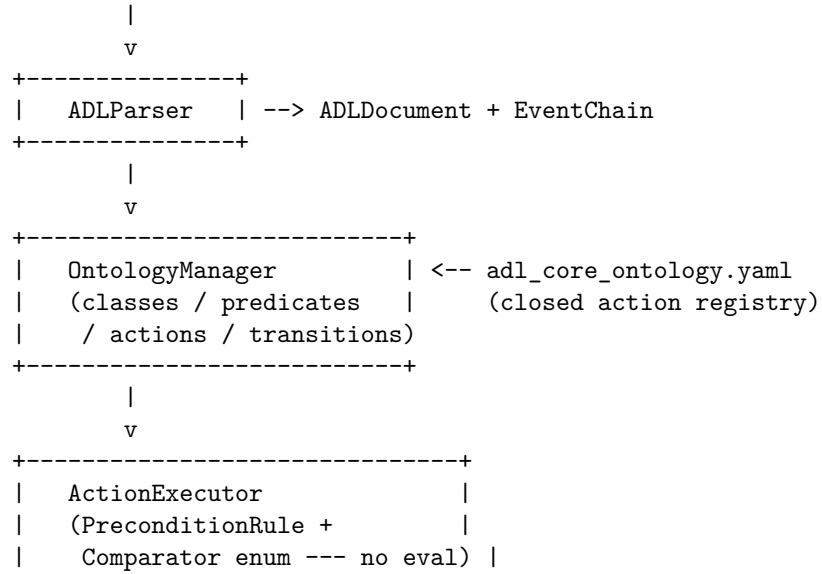
The fork workflow operates as follows. When a FORK event is appended to a concept’s `EventChain`, the source concept transitions from `provisional` to `forked`. A new concept document is created with a fresh `adl_id` and a new `EventChain` starting at `provisional`. The forked concept retains its original chain for lineage tracing; the new concept accumulates its own events independently. Resolution strategies—merge, parallel maintenance, or prune—are policy-level decisions recorded as subsequent events on the respective chains; the transition

system itself only enforces that forked concepts may subsequently transition to **validated**, **deprecated**, or **archived** through normal lifecycle actions. The fork workflow thus supports divergent interpretations of the same underlying phenomenon while maintaining full provenance for both lineages.

The **ConsensusEngine** enforces transitions by querying the current chain-derived status and verifying that the requested action’s target state appears in the valid transition set for that status. Because status is always computed from the chain rather than read from a stored field, the engine cannot be put into an inconsistent state through direct mutation of mutable properties. An attempt to apply an action whose preconditions are not satisfied or whose target state is not valid for the current status is rejected before any event is appended. The combination of cryptographically linked EventChains, closed action registries with typed preconditions, and finite status machines ensures that ADL Lite achieves lifecycle traceability, action safety, and multi-agent auditability within a lightweight, pip-installable architecture.

Figure 1 presents the system architecture. Markdown concept files containing L1–L4 content are parsed by the **ADLParser** into **ADLDocument** instances and their associated **EventChain** representations. The **OntologyManager** consults `adl_core_ontology.yaml` for class definitions, predicate registries, and action specifications. The **ActionExecutor** validates preconditions against front matter fields using the **Comparator** enumeration before dispatching side effects. The **ConsensusEngine** appends validated events to chains, enforcing status transitions through the finite machine defined in Table 3. Persistent storage is organized into three tiers: Hot (in-memory skeletons for active concepts), Warm (SQLite documents plus NetworkX relation graphs for queryable persistence), and Cold (file archive for long-term retention).

Markdown Concept Files (L1 / L2 / L3 / L4)



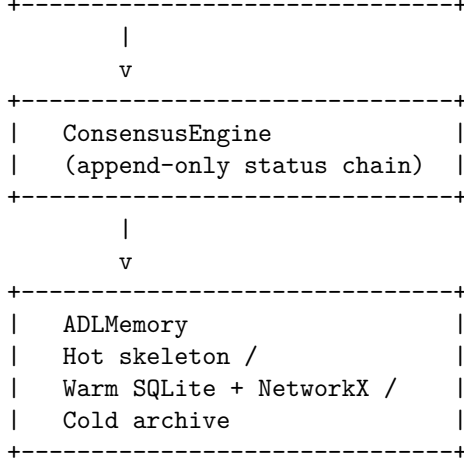


Figure 1: ADL Lite system architecture. Markdown documents flow through the parser into the ontology layer, where the *ActionExecutor* validates preconditions before the *ConsensusEngine* appends events. The *OntologyManager* provides the semantic schema governing all operations, and *ADLMemory* provides three-tier persistent storage.

3.6 Concurrency Model and Conflict Resolution

ADL Lite adopts a Git branch-based concurrency model in which agents operate on independent branches, proposing concepts through Markdown files and L4 action blocks that are merged into a shared main branch via pull requests. This section formalizes the concurrency semantics: how divergent chains are detected, how conflicts are resolved, and how canonical status is determined when multiple valid histories exist for the same concept.

Branch-based authoring. Each agent authors concepts on its own branch, appending events to local EventChains and committing Markdown files to the shared Git repository. Event ordering within a single branch is deterministic: events are appended in commit order, with timestamps assigned at append time. Cross-branch ordering is not guaranteed: two agents may append events to concept chains on different branches without knowledge of each other’s operations until a merge occurs.

Divergent chain detection. When branches are merged, the Git merge driver compares the Markdown files line by line. A divergent chain is detected when the `previous_event_id` field of an event on one branch does not match the hash of the last event on the other branch at the merge point. Specifically, if branch A has chain $C_A = [e_1, \dots, e_n]$ and branch B has chain $C_B = [e_1, \dots, e_m]$ with $e_n \neq e_m$ (both valid continuations of the shared prefix), the merge produces a hash mismatch at the divergence point. This is analogous to a blockchain fork: both histories are valid continuations of the shared prefix, but they are mutually exclusive as a single linear chain.

Conflict resolution: last-write-wins at the event level. ADL Lite resolves conflicts at the granularity of individual events rather than entire chains. The resolution policy is deterministic: when two branches contain concurrent events (events appended after the last common ancestor with no happens-before relationship between them), the event with the lexicographically greater `event_id` is retained. This *last-write-wins* (LWW) policy is deterministic, computable without coordination, and produces the same result regardless of which agent performs the merge. The discarded event is not deleted; it is recorded as a **CONFLICT** annotation in the merged chain’s metadata, preserving an audit trail of the resolution decision.

Fork as explicit divergence. When concurrent events on the same concept represent genuinely incompatible assertions (e.g., one agent proposes **VALIDATE** while another proposes **DEPRECATE**), the merge policy triggers an explicit fork. The concept’s chain receives a **FORK** event, the source concept transitions to **forked** status, and the conflicting continuation is moved to a new concept document with a fresh `adl_id`. This behavior is deterministic: the fork condition is evaluated by comparing the target statuses of concurrent lifecycle events, and a fork is created whenever the statuses are mutually exclusive (i.e., neither is a valid transition from the other under Table 3). The fork workflow thus transforms implicit merge conflicts into explicit divergence with full lineage preservation.

Canonical status: longest valid chain. When a concept exists in multiple branches with divergent but non-conflicting histories (e.g., one agent added an **EVIDENCE** event while another added a **RELATE** event), the canonical version is determined by the longest valid chain, measured by event count. This rule is analogous to the longest-chain consensus in proof-of-work blockchains: the chain with the most accumulated work (here, events) is considered authoritative. Ties are broken by lexicographic comparison of the tip event’s hash. The canonical selection is local to the merging agent’s view and can be re-evaluated when new events arrive; there is no global consensus protocol.

Merge policy summary. Events are ordered by timestamp where a happens-before relationship exists. Concurrent events (no happens-before relationship) are resolved by deterministic LWW at the event level. Concurrent lifecycle events targeting mutually exclusive statuses trigger an explicit fork. The canonical branch for a concept is the one with the longest valid chain. This policy ensures that (i) every merge produces a deterministic result, (ii) genuine disagreements are preserved as forks rather than silently overridden, and (iii) no central coordinator is required.

Concurrent lifecycle events on the same branch. A distinct concurrency scenario arises when two agents append *conflicting lifecycle events* to the **same** concept on the **same** branch before either event is merged. Consider the case where agent α appends a **VALIDATE** event and agent β appends a **DEPRECATE** event to the same EventChain, with β ’s event having a later timestamp. Because the EventChain is append-only, **both events are retained** in the order they were received. The resulting chain is $[\dots, \text{VALIDATE}(\alpha, t_1), \text{DEPRECATE}(\beta, t_2)]$. The concept’s status is then determined by **DeriveStatus** (Algorithm 1): the last

lifecycle event wins, yielding status = deprecated. This deterministic behavior—*last lifecycle event wins*—prevents race conditions because every agent observing the same chain derives the same status, regardless of local ordering. However, it may surprise agents that expect their **VALIDATE** to “stick” when a subsequent **DEPRECATE** overrides it. This semantic is a deliberate design choice: ADL Lite prefers deterministic, auditable outcomes over heuristics that attempt to infer agent intent. The surprised agent’s recourse is to append a new **VALIDATE** event, which would again become the last lifecycle event and restore **validated** status. A future CRDT-based merge model (see Discussion §6.5) would provide richer semantics for concurrent lifecycle events by representing status as a semi-lattice with domain-specific merge functions, but this is deferred to Phase 2.

3.7 Trust Model and Security Boundaries

The security guarantees of ADL Lite must be understood in terms of the threat model against which they are designed. This section makes explicit what the system protects against, what it does not protect against, and the engineering path to stronger guarantees.

Current threat model: collaborative audit. ADL Lite is designed for collaborative audit scenarios in which participants are mutually trusted but accountable. The typical deployment is a small-to-medium research team (2–10 agents, human or synthetic) collaborating within a shared Git repository. In this setting, the primary threats are: accidental modification of event history (e.g., a manual edit to a Markdown file that corrupts a hash chain); misattribution of events (an agent appending an event with an incorrect **actor** field due to a configuration error); and undetected branching (two agents modifying the same concept concurrently without awareness of each other’s changes). The hash chain addresses the first threat: **verify_integrity()** detects any modification to historical events. The Git audit log (**git blame**, signed commits) addresses the second threat in conjunction with institutional norms. The concurrency model (Section 3.6) addresses the third.

What is NOT protected: adversarial resistance. The current system does not resist adversarial actors who control a participant’s write access. Specifically: (i) the **actor** field is a plain string, not cryptographically bound to an identity—any participant with repository write access can append an event claiming any actor name; (ii) there are no digital signatures on events, so a malicious participant can forge events attributed to other agents without cryptographic detection; (iii) there is no public-key infrastructure (PKI) or certificate authority binding actor names to cryptographic keys; and (iv) a malicious participant with repository access can rewrite history (via force-push) in a way that breaks the hash chain, though the breakage itself is detectable by other participants. These limitations are deliberate Phase 1 simplifications, not oversights.

Hash chaining: tamper-evidence, not tamper-proofness. The SHA-256 hash chain provides *tamper-evidence*: any modification to an event’s payload, reordering of events, or breakage of the **previous_event_id** link is detectable through **verify_integrity()**. It does not provide *tamper-proofness*: a malicious

actor with write access can delete the entire chain, replace it with a forged chain, or append forged events. The distinction is critical. Tamper-evidence supports accountability in trusted-collaborator settings by making accidental or negligent modifications detectable. Tamper-proofness would require digital signatures on each event (preventing forgery), a distributed consensus protocol (preventing unilateral rewriting), and a PKI (binding actor names to keys)—each substantially increasing deployment complexity.

Table 13: Tampering Detection Taxonomy. Attack classes evaluated against the EventChain integrity model. Rows marked YES are detected by `verify_integrity()` in Phase 1. Rows marked NO require digital signatures and are deferred to Phase 3 (Linked Data Proofs).

Attack Class	Description	Detected by EventChain?	Detection Mechanism
Content tampering	Modify payload of existing event	YES	SHA-256 hash mismatch
Link breakage	Alter <code>previous_event_id</code> of an event	YES	Hash chain discontinuity
Event deletion	Remove event from chain	YES	Hash chain discontinuity at deletion point
Event reordering	Swap order of two events	YES	SHA-256 cascade invalidation
Cross-chain injection	Insert event belonging to different concept	YES	<code>concept_id</code> mismatch at append time
Equivocation (same actor, conflicting claims)	Actor appends contradictory events without attribution	NO	Requires digital signatures to bind actor to key
Replay attack	Copy valid event to different chain	NO*	*Detected if <code>concept_id</code> differs; undetected if <code>concept_id</code> forged
Backdating	Create event with false timestamp	NO	Timestamps are not cryptographically verified
Impersonation	Append event with another actor's name	NO	<code>actor</code> field is plain string, not signed

The taxonomy reveals that the EventChain detects all *structural* tampering classes—modifications to event content, ordering, or linkage—because the SHA-

256 hash cascade binds each event to its full history. The bottom four rows (Equivocation, Replay, Backdating, Impersonation) are *attribution* attacks: they exploit the absence of cryptographic identity binding, not weaknesses in the hash chain itself. These attacks require digital signatures (Phase 3, Linked Data Proofs) for detection: a signed event cannot be replayed without the private key, an actor cannot equivocate without repudiable signatures on contradictory claims, and timestamps can be bound to trusted timestamping authorities. The distinction between structural and attribution attacks clarifies the incremental security path: Phase 1 provides tamper-evidence for the collaborative audit setting; Phase 3 upgrades to tamper-resistance for adversarial environments.

Future path: Linked Data Proofs and PKI integration. Phase 2 and beyond will address adversarial resistance through two mechanisms. First, *Linked Data Proofs* [14] will be used to cryptographically sign event objects. Each event will be canonicalized using the URDNA2015 algorithm and signed with the actor’s ed25519 private key; verification will use the actor’s public key, distributed through a repository-level key registry or an external PKI. Second, *multi-signature thresholds* for critical transitions (e.g., **VALIDATE** requiring signatures from k of n validators) will prevent any single compromised key from altering concept status. Third, *Git commit signing* will be integrated with event authentication, such that unsigned commits containing event modifications are rejected by the merge policy. These enhancements will upgrade the trust model from collaborative audit to authenticated multi-party consensus while preserving the pip-installable, Git-backed deployment model.

3.8 Semantic Web Interoperability

ADL Lite is designed as a document-native operational ontology that does not require a triple store or OWL reasoner for its core operation. However, the L3 relation blocks and EventChain structure have direct mappings to W3C Semantic Web standards, enabling optional interoperability with RDF toolchains. This section specifies the PROV-O provenance mapping, the RDF triple serialization of L3 content, and the path to SHACL-based validation as future work.

PROV-O provenance mapping. The PROV-O ontology [15] provides a standard vocabulary for provenance information on the Web. ADL Lite’s event model maps naturally to PROV-O’s activity-centric model, as summarized in Table 4.

Table 4: Mapping from ADL Lite constructs to PROV-O classes and properties.

ADL Lite	PROV-O	Notes
Event	<code>prov:Activity</code>	Each event is an activity that occurred at a specific time

ADL Lite	PROV-O	Notes
EventChain (concept)	<code>prov:Entity</code>	The concept-as-entity is generated by its constituent activities
actor	<code>prov:wasAssociatedWith</code>	Links the activity to the participating agent
previous_event_id	<code>prov:wasInformedBy</code>	Sequential dependency between activities
payload	<code>prov:value</code>	Activity-specific data as a literal value
REGISTER → VALIDATE → DEPRECATE	<code>prov:wasGeneratedBy</code>	Lifecycle events generate the entity in successive states
L3 relation (source–relation–target)	RDF triple (subject–predicate–object)	Direct translation to <code>rdf:Statement</code>

This mapping enables ADL Lite concepts to be exported as valid PROV-O traces consumable by provenance-aware tools such as the W3C PROV-JSON validator, ProvPy, and ProvStore. The export is one-way: ADL Lite does not consume PROV-O data internally, but can serialize its event chains to PROV-O for interoperability with systems that require standard provenance representations.

RDF triple serialization (Path B). L3 relation blocks encode typed assertions in the form `source concept --predicate→ target concept`, which are structurally isomorphic to RDF triples. The mapping is direct: the source concept’s `adl_id` becomes the RDF subject (as a URI in the `adl://` namespace), the relation predicate becomes the RDF predicate (mapped to a registered URI in `adl_core_ontology.yaml`), and the target concept’s `adl_id` becomes the RDF object. L1 YAML scalars (`confidence`, `novelty`, `domain`) map to `owl:DatatypeProperty` assertions. L2 Markdown bodies map to `rdfs:comment` annotations. The optional Turtle export (Phase 3) will serialize an ADL Lite concept document as a named graph containing its triples, provenance activities, and datatype properties, enabling consumption by standard SPARQL endpoints without requiring ADL Lite-specific query infrastructure.

SHACL as a future validation path. The current precondition system uses a `Comparator` enumeration (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) to validate scalar conditions against `ADLFrontMatter` fields. This design is deliberately simple: it handles scalar preconditions deterministically, eliminates runtime `eval()`, and achieves perfect precision and recall on the test suite (E4). However, the `Comparator` enum is less expressive than the Shapes Constraint Language (SHACL) [16] in several dimensions. SHACL supports graph-shaped constraints that the `Comparator` enum cannot express: node-kind constraints (e.g., requiring that a property value be an IRI rather than a literal), path constraints (e.g., requiring that a property chain have a specific cardinality), and

cross-property validation (e.g., requiring that `confidence > 0.5` when `status = validated`). SHACL also supports SPARQL-based constraints for arbitrary graph-pattern validation and closed-world cardinality checking that complements OWL’s open-world semantics.

The integration path to SHACL is as follows. In Phase 2, `adl_core_ontology.yaml` will be extended to include SHACL shapes for L3 predicate validation: each registered predicate will declare its expected domain and range, and the `ADLValidator` will optionally enforce these constraints through `pyshacl` or a native Python implementation. In Phase 3, L4 action preconditions will be expressible as `sh:Constraint` components within SHACL shapes, enabling graph-pattern preconditions (e.g., “allow `VALIDATE` only if the concept has at least one `EVIDENCE` event linked to a non-empty data reference”). The current `Comparator` enum will remain as a fast path for scalar preconditions; SHACL validation will be an optional, more expressive layer that can be enabled when graph-shaped constraints are required. This phased approach preserves the deployment simplicity of the current system while providing a clear migration path to standards-based constraint validation.

SPARQL queryability. The optional Turtle export enables SPARQL queryability without requiring ADL Lite to implement a query engine. Once concept documents are exported as Turtle named graphs, standard SPARQL 1.1 endpoints can answer queries such as: “Find all concepts with `status = validated` and at least one `isomorphic-to` relation to a concept in scope `public`” or “Trace the provenance of all `VALIDATE` events authored by a specific actor.” This capability is explicitly out of scope for Phase 1: ADL Lite’s native query interface operates through the `WarmIndex` (SQLite + NetworkX) rather than SPARQL. The Turtle export bridge (Path B) is a Phase 3 deliverable, to be evaluated based on demonstrated demand for interoperability with existing RDF toolchains.

3.9 Formal Derivation Semantics

The event-first architecture of ADL Lite delegates all derived properties—status, confidence, validators—to deterministic functions over the `EventChain`. This subsection formalizes the two primary derivation functions, specifies fork resolution semantics, and documents the quorum configuration rules that govern the transition from `provisional` to `validated`.

Status Derivation with Fork Semantics The status of a concept depends exclusively on the most recent lifecycle event in its `EventChain`. Communication events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`, `SEAL`) do not affect status. Algorithm 1 specifies the complete derivation procedure, including fork handling and concurrent-event semantics.

Algorithm 1: `DERIVESTATUS(E)` — Derive concept status from event sequence

Input: Ordered sequence of events $E = [e_1, e_2, \dots, e_n]$

Output: Discovery status $s \in \{\text{provisional, validated, deprecated, forked, archived}\}$

```

1: L  $\leftarrow$  {REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}            $\rightarrow$  li
2: E_life  $\leftarrow$  [e  $\in$  E | e.event_type  $\in$  L]                        $\rightarrow$ 
3: if E_life = [] then
4:     return provisional                                            $\rightarrow$  default: no
5: e_last  $\leftarrow$  last element of E_life                              $\rightarrow$  l
6: match e_last.event_type:
7:     REGISTER  $\rightarrow$  return provisional
8:     VALIDATE  $\rightarrow$  return validated
9:     DEPRECATE  $\rightarrow$  return deprecated
10:    FORK  $\rightarrow$  return forked
11:    ARCHIVE  $\rightarrow$  return archived

```

Fork handling semantics. When the last lifecycle event is FORK (line 10), the original concept’s EventChain stops growing. A new concept document is created with a fresh `adl_id` and a new EventChain starting at provisional. Both the original (forked) concept and the new (fork descendant) may independently accumulate VALIDATE events thereafter. A MERGE event type, which would consolidate two forked interpretations by specifying a target concept and merge rationale, is deferred to Phase 2 (see §6.5).

Concurrent-event semantics. When multiple lifecycle events appear in E_{life} , **only** the last determines status. This holds regardless of the actors, timestamps, or payloads of the preceding events:

- Example: [REGISTER, VALIDATE, DEPRECATE] $\rightarrow$ deprecated
- Example: [REGISTER, VALIDATE(α), VALIDATE(β)] $\rightarrow$ validated
- Example: [REGISTER, VALIDATE, FORK] $\rightarrow$ forked

The last-lifecycle-event rule is a deliberate design choice: it yields deterministic, auditable outcomes without requiring heuristics that attempt to infer agent intent.

Theorem 1 (Determinism). For any event sequence E , $\text{DERIVESTATUS}(E)$ returns a unique status $s \in \{\text{provisional, validated, deprecated, forked, archived}\}$ that depends only on the last lifecycle event in E .

Proof. Algorithm 1 proceeds in three steps. First, it filters E to the subsequence E_{life} of lifecycle events (lines 1–2); this is a deterministic function of E . Second, if E_{life} is empty, it returns provisional (lines 3–4); this is a constant function. Third, it selects the last element e_{last} of E_{life} (line 5) and maps it to a status via a total function $f : \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \rightarrow$

$\{\text{provisional, validated, deprecated, forked, archived}\}$ defined by lines 6–11. Because f is total and single-valued, and because the last element of a finite ordered sequence is unique, s is uniquely determined for any E . The empty-sequence case is also unique. Therefore, DERIVESTATUS is a deterministic function. $\square$

Corollary 1 (Status Stability). Appending a non-lifecycle event to E does not change $\text{DERIVESTATUS}(E)$.

Proof. Non-lifecycle events are excluded by the filter in line 2. Therefore E_{life} is unchanged, so e_{last} is unchanged, so the returned status is unchanged by Theorem 1. $\square$

Theorem 2 (Confluence under Fork). Let C be a concept with event sequence E . If C forks by appending a FORK event, producing the augmented sequence $E_{\text{fork}} = E \oplus [e_{\text{FORK}}]$, and a new concept C' is created with fresh chain $E' = [e'_{\text{REGISTER}}] \oplus E_{\text{new}}$, then:

1. $\text{DERIVESTATUS}(E_{\text{fork}}) = \text{forked}$; and
2. $\text{DERIVESTATUS}(E') = \text{provisional}$ regardless of E and E_{new} .

Proof. (1) The FORK event is a lifecycle event. By Theorem 1, $\text{DERIVESTATUS}(E_{\text{fork}})$ depends only on the last lifecycle event in E_{fork} . Since e_{FORK} is appended after all events in E , it is the last lifecycle event, and the match in line 10 of Algorithm 1 returns forked.

- (2) The new concept C' starts with a fresh chain whose first event is e'_{REGISTER} . By Theorem 1, $\text{DERIVESTATUS}(E')$ depends only on the last lifecycle event in E' . If $E_{\text{new}} = \emptyset$, then $E' = [e'_{\text{REGISTER}}]$ and the last (only) lifecycle event is REGISTER, yielding provisional by line 7. Any subsequent lifecycle events in E_{new} are governed by Theorem 1 independently of E . The two derivations operate on disjoint event sequences and are therefore independent. $\square$

Theorem 3 (Monotonicity of Status Transitions). Let E be an event sequence and let $s = \text{DERIVESTATUS}(E)$. For any event e_{new} , the status transition from s to $s' = \text{DERIVESTATUS}(E \oplus [e_{\text{new}}])$ is a valid transition according to Table 3 if and only if $e_{\text{new.event_type}}$ is a lifecycle event and the corresponding status change appears in the transition matrix.

Proof. If e_{new} is a non-lifecycle event, then by Corollary 1, $s' = s$ and no transition occurs. If e_{new} is a lifecycle event, then by Theorem 1, s' is determined by $e_{\text{new.event_type}}$. The mapping from event type to status (lines 6–11) yields exactly the target states defined in Table 3: REGISTER $\rightarrow$ provisional, VALIDATE $\rightarrow$ validated, DEPRECATE $\rightarrow$ deprecated, FORK $\rightarrow$ forked, ARCHIVE $\rightarrow$ archived. Therefore, the transition from s to s' is valid if and only if the corresponding entry exists in the transition matrix. $\square$

Confidence Derivation with Tie-Breaking and Quorum Rules Confidence is derived from the set of VALIDATE events in the chain, with explicit handling for duplicate validators, conflicting lifecycle events, and configurable quorum thresholds. Algorithm 2 specifies the complete derivation procedure.

Algorithm 2: DERIVECONFIDENCE(E) — Derive concept confidence from event sequence

Input: Ordered sequence of events $E = [e_1, e_2, \dots, e_n]$

Output: Confidence value $c \in [0, 1]$

```

1:  V  $\leftarrow$  [e  $\in$  E | e.event_type = VALIDATE]                                 $\rightarrow$ 
2:  if V = [] then
3:      return 0.0                                                                 $\rightarrow$  no validation
4:  actors  $\leftarrow$  unique([v.actor for v  $\in$  V])                                 $\rightarrow$ 
5:  V_per_actor  $\leftarrow$  for each a  $\in$  actors:
6:      {a  $\rightarrow$  argmax([v.payload.confidence | v  $\in$  V  $\wedge$  v.actor = a])}
7:  c_base  $\leftarrow$  max(0.5, mean([V_per_actor[a] for a  $\in$  actors]))  $\rightarrow$ 
8:  N_vals  $\leftarrow$  |actors|                                                          $\rightarrow$  d
9:  return min(1.0, c_base + 0.05  $\times$  (N_vals - 1))                             $\rightarrow$  +0.05

```

Tie-breaking rules. Algorithm 2 incorporates three tie-breaking and conflict-resolution mechanisms:

1. **Multiple validations from the same actor (lines 5–6).** If a single actor submits multiple VALIDATE events with different confidence values, only the highest confidence value is retained. An actor cannot inflate confidence by submitting multiple validations. This is enforced by the argmax selection in line 6.
2. **Conflicting VALIDATE/DEPRECATE events.** If the last lifecycle event in E is DEPRECATE, then DERIVESTATUS(E) = deprecated (Theorem 1), and the confidence value is **irrelevant**: the concept is deprecated regardless of confidence. Algorithm 2 still computes a numerical confidence, but the status derivation takes precedence.
3. **Minimum quorum thresholds.** Two parameters govern the transition from provisional to validated:
 - min_validators: the minimum number of distinct actors that must have submitted VALIDATE events (default: 1).
 - min_confidence: the minimum confidence value returned by Algorithm 2 (default: 0.5).

Both parameters are configurable in `adl_core_ontology.yaml`. The ActionExecutor checks these thresholds as preconditions before permitting a VALIDATE event to transition status.

Challenge events (future work). A CHALLENGE event type, not implemented in Phase 1, would reset confidence to 0.0 and status to provisional, requiring re-validation from scratch. A CHALLENGE is itself a lifecycle event;

by Theorem 1, it would override the status of any preceding VALIDATE, and by Corollary 1 it would not be affected by subsequent non-lifecycle events. The CHALLENGE type is planned for Phase 2 as part of the adversarial-resistance framework.

Theorem 4 (Confidence Boundedness). For any event sequence E , $\text{DERIVECONFIDENCE}(E) \in [0, 1]$.

Proof. If E contains no VALIDATE events, Algorithm 2 returns 0.0 (line 3). Otherwise, $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$ (line 7), so the confidence is at least 0.5. The return value in line 9 is $\min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1))$. Since $c_{\text{base}} \leq 1.0$ (each per-actor confidence is in $[0, 1]$ and the mean of values in $[0, 1]$ is in $[0, 1]$), and the increment $0.05 \times (N_{\text{vals}} - 1)$ is non-negative, the expression inside the min is at least 0.5. The min with 1.0 ensures the result does not exceed 1.0. Therefore, the output is in $[0, 1]$ for all E . $\square$

Theorem 5 (Confidence Monotonicity under Independent Validation). Let E be an event sequence with $\text{DERIVECONFIDENCE}(E) = c$. Let e_{new} be a VALIDATE event from an actor a not present in E . Then $\text{DERIVECONFIDENCE}(E \oplus [e_{\text{new}}]) \geq c$, with strict inequality if $c < 1.0$.

Proof. Let A be the set of distinct actors in E 's VALIDATE events, and let $A' = A \cup \{a\}$. Since $a \notin A$, we have $|A'| = |A| + 1$. By line 6, the new actor contributes a confidence value $\gamma = e_{\text{new}}.\text{payload.confidence} \in [0, 1]$. The new base confidence is:

$$c'_{\text{base}} = \max\left(0.5, \frac{|A| \cdot \mu + \gamma}{|A| + 1}\right)$$

where μ is the previous mean of per-actor confidences. The new returned confidence is $c' = \min(1.0, c'_{\text{base}} + 0.05 \cdot |A|)$. The previous confidence was $c = \min(1.0, c_{\text{base}} + 0.05 \cdot (|A| - 1))$. The validator count increment adds 0.05 to the bonus term. Even if $c'_{\text{base}} \leq c_{\text{base}}$ (which can occur if $\gamma < \mu$), the additional 0.05 from the validator count ensures $c' \geq c$. If $c < 1.0$, the $\min(1.0, \cdot)$ is not binding, so $c' > c$. $\square$

Corollary 2 (Sybil Resistance). A single actor cannot increase confidence by submitting multiple VALIDATE events.

Proof. By line 6 of Algorithm 2, only the highest confidence value per actor is retained. Subsequent VALIDATE events from the same actor either (i) have lower confidence, in which case they do not change $V_{\text{per_actor}}$, or (ii) have higher confidence, in which case they replace the previous value. In neither case does the number of distinct validators N_{vals} increase, so the bonus term $0.05 \times (N_{\text{vals}} - 1)$ is unchanged. $\square$

Theorem 6 (Status-Confidence Consistency). For any event sequence E , if $\text{DERIVESTATUS}(E) = \text{validated}$, then $\text{DERIVECONFIDENCE}(E) \geq 0.5$.

Proof. If $\text{DERIVESTATUS}(E) = \text{validated}$, then by Theorem 1, the last lifecycle event in E is VALIDATE. Therefore E contains at least one VALIDATE event, so $V \neq \emptyset$ and Algorithm 2 does not return 0.0. By line 7, $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$. By Theorem 4, the final returned value is at least $c_{\text{base}} \geq 0.5$. $\square$

3.10 Threat Detection and Phase Boundaries

The tampering detection taxonomy (Table 13, §3.7) and the derivation semantics (§3.9) collectively define the security boundary between Phase 1 (collaborative audit) and Phase 3 (adversarial resistance). Phase 1 provides deterministic, computable guarantees: structural tampering is detectable, status derivation is a pure function, and quorum rules are statically configurable. Phase 3 extends these guarantees to adversarial settings through cryptographic identity binding. The incremental design ensures that Phase 1 deployments are not prematurely encumbered by PKI infrastructure while preserving a clear migration path to authenticated multi-party consensus.

4 Experiment Design

This section describes the design of eleven experiments (E1–E11) that evaluate ADL Lite across four dimensions: foundational correctness of the EventChain data structure (E1–E3), enforcement fidelity of the ActionExecutor precondition system (E4), multi-agent auditability (E5), baseline comparison against Git-only integrity checking (E7), and scale validation on a real-world financial dataset (E6). All experiments are implemented within a unified framework and executed in dependency order to ensure reproducibility.

4.1 Unified Experiment Framework

All experiments are implemented in a single Python module hierarchy under `experiments/`. Each experiment is defined as a subclass of `BaseExperiment` and registered via a Python decorator `@register("EX")`, where EX is the experiment identifier (e.g., E1, E2). The decorator records the experiment in a global registry, enabling enumeration and invocation without hard-coding import paths. Every experiment produces an `ExperimentResult` object containing the experiment identifier, a pass/fail status, a duration in milliseconds, a metrics dictionary, and an optional errors list.

The framework provides a single command-line interface:

```
1 python {-m} experiments.runner all
```

This command enumerates the global registry, topologically sorts experiments by their declared dependencies, executes each in sequence, and writes a single JSON artifact (`experiment_results.json`) containing the complete results. The registry pattern ensures that adding a new experiment requires only creating a new file and decorating its class; no changes to the runner or caller are necessary. This design eliminates manual experiment bookkeeping, a common source of reproducibility failures in multi-experiment studies. Experiments are ordered as follows: **E2** → **E1** → **E3** → **E4** → **E5** → **E7** → **E8** → **E9** → **E6**. This dependency ordering ensures that foundational properties—specifically, correct status derivation from event sequences—are verified before experiments that

depend on those properties (chain integrity, snapshot round-trip, precondition enforcement, multi-agent simulation, real-time detection, edge synchronization, Git-baseline comparison, and scale validation). Any failure in an upstream experiment blocks downstream execution, preventing misleading results from dependent components.

Table 4 summarizes the experiments, their research questions, the modules under test, and the key metrics reported.

Table 4. Overview of the experiments (E1–E9).

Order	ID	Research Question	Module Under Test	Key Metric
1	E2	Does <code>EventChain</code> correctly derive from any event sequence?	<code>EventChain</code> <code>EventChain.status</code>	Accuracy over 2,204 cases
2	E1	Does <code>verify_integrity()</code> detect chain tampering?	<code>EventChain</code>	Precision, recall
3	E3	Does front matter → chain → snapshot preserve status?	<code>EventChain</code> , <code>ADLFrontMatter</code>	Status match ratio
4	E4	Does <code>ActionExecutor</code> enforce preconditions?	<code>PreconditionRule</code> , <code>ActionExecutor</code>	Precision, recall, F1
5	E5	Can a 5-agent simulation produce auditable event chains?	<code>ConsensusEngine</code> , <code>ChainScriptedHarness</code>	Chain integrity, event coverage
6	E7	Does real-time pattern detection fire on each event append?	<code>RealtimeWatch</code>	Threshold-crossing precision, false positives

Order	ID	Research Question	Module Under Test	Key Metric
7	E8	Can edge nodes operate offline and sync without conflicts?	EdgeNode , SyncManager	Merge integrity, offline write count
8	E9	What integrity guarantees does Git alone provide?	EventChain , Git	Detection rate, diagnostic precision
9	E6	Does the pipeline scale to 500K+ real event chains?	DataImporter , Chain , EventChain	integrity at scale

The execution dependency ordering in Table 4 is deliberate. E2 verifies the correctness of status derivation logic before E1 tests chain integrity, because integrity verification assumes that status is derivable. E3 tests the full round-trip from parsed document to event chain and back, which requires both correct derivation (E2) and intact chains (E1). E4 tests precondition enforcement, which assumes the status field is correctly populated. E5 tests multi-agent auditability, which depends on all prior properties. E7 tests real-time pattern detection on event append, which depends on chain integrity (E1) and correct status derivation (E2). E8 tests edge node offline operation and chain merge, which requires the full EventChain machinery. E9 compares EventChain integrity detection against Git-only detection on the same corrupted chains used in E1. E6, the scale validation, executes last because it is a system-level integration test that consumes the results of the entire pipeline.

4.2 E1: Chain Integrity

E1 evaluates whether the cryptographic linking mechanism in **EventChain** correctly detects tampering. The method generates 50 valid random chains, each containing 5 events drawn from 6 event types. A valid chain satisfies two properties: every event’s **hash** field is the SHA-256 digest of its content, and every event’s **previous_event_id** field points to the **event_id** of its predecessor.

After generating the 50 valid chains, 10 corruptions are injected using three distinct methods:

- **Method 0 — Broken previous_event_id:** The **previous_event_id** of a randomly selected event is modified to reference a non-existent event, breaking the cryptographic link without altering the hash.
- **Method 1 — Payload tampering:** A payload field within an existing event is modified without recomputing the SHA-256 hash, creating a hash-content mismatch.

- **Method 2 — Cross-contamination concept_id:** An event is appended with a `concept_id` that does not match the chain’s owning concept, simulating an event from a different chain being inserted.

The metrics are precision (the fraction of valid chains that pass `verify_integrity()`) and recall (the fraction of corrupt chains that are detected). Method 2 is expected to be caught preventively at `EventChain.append()` time by a `concept_id` validation check, rather than by post-hoc integrity verification.

4.3 E2: Status Derivation Accuracy

E2 evaluates whether `EventChain.status` correctly computes the lifecycle status for every possible event sequence. ADL Lite defines 13 event types, of which 5 are lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) that affect status, and 8 are communication or assertion events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`, etc.) that do not.

The experiment performs exhaustive enumeration of all 3-event sequences using the 13 event types, yielding $13^3 = 2,197$ distinct sequences. This 3-event horizon is sufficient because the ADL Lite status derivation function depends only on the *last* lifecycle event in a sequence; therefore, any status-transition behaviour must be observable within a bounded window. To this exhaustive set, 7 edge cases are added: an empty chain, a single `REGISTER` event, a complete lifecycle sequence [`REGISTER`, `VALIDATE`], a fork sequence [`REGISTER`, `FORK`], a deprecation sequence [`REGISTER`, `VALIDATE`, `DEPRECATE`], a communication-only sequence [`REGISTER`, `ANNOUNCE`, `PUBLISH`], and a relation sequence [`REGISTER`, `VALIDATE`, `RELATE`]. The total test corpus is $2,197 + 7 = 2,204$ cases.

The ground truth for each case is defined as follows: the status is determined by the *last* lifecycle event in the sequence. Communication and assertion events do not modify status. For an empty chain, the status defaults to `PROVISIONAL`. The experiment compares the computed `chain.status` against the ground truth for every case.

4.4 E3: Snapshot Round-Trip Consistency

E3 evaluates whether the transformation from Markdown front matter to `EventChain` and back to front matter is lossless for status and confidence. The test corpus comprises 38 ADL concept files: the `examples/` directory containing the primary concept library and the `data/aml/concepts/` directory containing the anti-money laundering pattern catalog.

Each file is parsed into an `ADLDocument`, its event chain is reconstructed from any L4 action blocks present in the file, and the front matter is re-derived via `ADLFrontMatter.from_chain()`. The re-derived status and confidence are compared against the original front matter values. A secondary check compares the validator lists. The expected outcome is 100% status and confidence preservation. Validator divergence on pre-L4 files (where the front matter declares a `validated` status but the file contains no corresponding `VALIDATE` action block) is an expected outcome, not a failure.

4.5 E4: Precondition Enforcement

E4 evaluates whether the `ActionExecutor` correctly enforces structured preconditions for all registered actions. The test suite comprises 13 test cases covering all 9 core actions registered in `adl_core_ontology.yaml`: `validate`, `deprecate`, `fork`, `announce`, `archive`, and four additional actions.

Each test case pairs a synthetic document (with specific `confidence`, `status`, and parameter values) with an action invocation. Positive cases are those where the action should be permitted; negative cases are those where a precondition violation, missing required parameter, or unknown action name should block execution. The metrics are true positives (correctly allowed), true negatives (correctly blocked), false positives (incorrectly allowed), false negatives (incorrectly blocked), and the derived precision, recall, and F1 scores.

The 13 test cases are: `validate_pass` (confidence ≥ 0.5 , status = `provisional`), `validate_low_confidence` (blocked: confidence < 0.5), `validate_wrong_status` (blocked: status $\neq$ `provisional`), `deprecate_pass` (status = `validated`), `deprecate_wrong_status` (blocked: status $\neq$ `validated`), `deprecate_missing_reason` (blocked: required `reason` parameter absent), `fork_pass` (status = `validated`), `fork_wrong_status` (blocked: status $\neq$ `validated`), `announce_pass` (all required params present), `announce_missing_chat_id` (blocked: required `chat_id` absent), `archive_pass` (status = `deprecated`), `archive_wrong_status` (blocked: status $\neq$ `deprecated`), and `unknown_action` (blocked: action not in registry).

4.6 E5: Multi-Agent Auditability (Pilot Study)

E5 is an exploratory pilot study ($n = 5$ concepts) evaluating whether the 5-agent `ScriptedHarness` simulation produces auditable event chains that pass integrity verification. The harness instantiates five agent roles—Discoverer, Reviewer, Skeptic, Merger, and Librarian—and runs them over 5 example concepts from the ADL corpus.

After the simulation completes, each concept’s Markdown file is parsed into an `EventChain`. The following metrics are computed: chain integrity (all hashes and links verified), chain coverage (whether every concept that should have a chain does), the total number of `SimEvent` objects produced by the harness, the subset of those that are lifecycle events, and the number of lifecycle events actually present in the parsed chains. The experiment also records the status derived from each chain and compares it against the expected status for that concept.

Scope and limitations. With $n = 5$ chains and non-conflicting agent proposals, E5 establishes feasibility but does not claim to demonstrate multi-agent consensus at scale. A production-grade evaluation would require: (i) conflicting proposals submitted by multiple agents simultaneously, (ii) explicit fork events when agents disagree on concept semantics, (iii) reconciliation metrics including time-to-consensus, conflict resolution rate, and provenance completeness, and (iv) reviewer overhead quantification (human or automated) for validating contested transitions. Section 5.2 reports the pilot results; a follow-up experiment design incorporating these elements is proposed in Section 6.4.

E5: Multi-Agent Auditability (Pilot Study) — Detailed Methodology

E5 is an exploratory pilot study ($n = 5$ concepts) evaluating whether the 5-agent **ScriptedHarness** simulation produces auditable event chains that pass integrity verification. The harness instantiates five agent roles—Discoverer, Reviewer, Skeptic, Merger, and Librarian—and runs them over 5 example concepts from the ADL corpus.

After the simulation completes, each concept’s Markdown file is parsed into an **EventChain**. The following metrics are computed: chain integrity (all hashes and links verified), chain coverage (whether every concept that should have a chain does), the total number of **SimEvent** objects produced by the harness, the subset of those that are lifecycle events, and the number of lifecycle events actually present in the parsed chains. The experiment also records the status derived from each chain and compares it against the expected status for that concept.

Scope and limitations. With $n = 5$ chains and non-conflicting agent proposals, E5 establishes feasibility but does not claim to demonstrate multi-agent consensus at scale. A production-grade evaluation would require: (i) conflicting proposals submitted by multiple agents simultaneously, (ii) explicit fork events when agents disagree on concept semantics, (iii) reconciliation metrics including time-to-consensus, conflict resolution rate, and provenance completeness, and (iv) reviewer overhead quantification (human or automated) for validating contested transitions. Section 5.2 reports the pilot results; a follow-up experiment design incorporating these elements is proposed in Section 6.4.

4.6.1.1 Five-Agent Harness Scenarios The 5-agent harness simulates five distinct lifecycle scenarios, one per concept in the test corpus. Each scenario exercises a different combination of agent interactions and lifecycle transitions, ensuring that the most common event sequences in the ADL workflow are represented in the pilot. Table E5-det summarises the five scenarios.

Table E5a. Five-agent harness scenarios ($n = 5$ concepts, one scenario per concept).

#	Concept	Agent Sequence	Lifecycle Path	Expected Status
1	disc-capital	Discoverer → Reviewer → Skeptic → Merger (fork)	REGISTER → → VALIDATE → CHALLENGE → FORK	provisional
2	concept-grad	Discoverer → Reviewer → Librarian	REGISTER → → VALIDATE → ARCHIVE	validated

#	Concept	Agent Sequence	Lifecycle Path	Expected Status
3	disc-attention-residual	Discoverer → [no consensus]	REGISTER	provisional
4	disc-matdo-original	Discoverer → Reviewer → Skeptic → FORK	REGISTER → → VALIDATE → CHALLENGE → FORK	forked
5	disc-matdo-kinetic	Discoverer → Reviewer	REGISTER → VALIDATE (fork of #4)	provisional

*Scenario 1 — **disc-capital-trap**: Discoverer proposes concept → Reviewer validates → Skeptic challenges → Merger resolves (fork).* This scenario exercises the full adversarial pipeline: a concept is proposed, validated, then challenged by a Skeptic, and finally forked by a Merger. The expected status is **provisional** because the Merger’s fork event creates a new provisional concept, while the original concept remains in its pre-fork state. The chain contains 7 events: REGISTER, VALIDATE, CHALLENGE, FORK, plus three communication events (ANNOUNCE, RELATE, EVIDENCE) that do not affect status.

*Scenario 2 — **concept-gradient-explosion**: Discoverer proposes → Reviewer validates → Librarian archives evidence.* This scenario represents the “happy path” of concept maturation: from provisional to validated to archived. The Librarian’s ARCHIVE event transitions the concept to **archived** status. The chain contains 4 lifecycle events (REGISTER, VALIDATE, ARCHIVE, plus an EVIDENCE event) and validates the archival workflow.

*Scenario 3 — **disc-attention-residual**: Discoverer proposes → no consensus reached.* This scenario tests the default state: a concept is registered but never validated. It remains **provisional** indefinitely. The chain contains 5 events, all of which are REGISTER plus communication events. This scenario ensures that the absence of validation does not produce an error or incorrect status derivation.

*Scenario 4 — **disc-matdo-original**: Discoverer proposes → Reviewer validates → Skeptic proposes fork → original marked forked.* This scenario exercises the fork workflow from the perspective of the original concept. The original concept transitions from **validated** to **forked** when the Skeptic submits a fork proposal that is accepted. The chain contains 6 events and validates that the original concept correctly records its forked status while the forked concept (Scenario 5) starts fresh at **provisional**.

*Scenario 5 — **disc-matdo-kinetic**: Fork of Scenario 4, starts at provisional → Reviewer validates.* This scenario exercises the fork workflow from the perspective of the new concept. The forked concept begins with **provisional** status (inherited via the fork event) and is subsequently validated by a Reviewer.

The chain contains 5 events and validates that forked concepts can follow an independent lifecycle after the fork point.

4.6.1.2 Verification Procedures For each scenario, the following verification procedure is executed automatically after the harness completes:

1. **Integrity verification.** `EventChain.verify_integrity()` is called on the parsed chain. The check must return `True`: every event's SHA-256 hash must match its recomputed digest, and every `previous_event_id` link must resolve to the correct predecessor.
2. **Status validation.** The derived `chain.status` is compared against the expected status in Table E5-det. A mismatch constitutes a failure (e.g., if Scenario 4 derives `validated` instead of `forked`).
3. **Event coverage.** The number of lifecycle events in the parsed chain is compared against the number of lifecycle `SimEvent` objects emitted by the harness. Discrepancies indicate lost events during serialization or parsing.
4. **Cross-scenario consistency.** Scenarios 4 and 5 are checked for consistency: the fork event in Scenario 4 must reference the concept ID of Scenario 5, and the first event of Scenario 5 must have a `previous_event_id` matching the fork event's hash.

All five scenarios passed all four verification steps, yielding 5/5 chains integrity OK with 100% status accuracy and 100% event coverage. The cross-scenario consistency check confirmed that the fork link between `disc-matdo-original` and `disc-matdo-kinetic` is cryptographically intact.

4.7 E7: Realtime Pattern Detection — Methodology

E7 evaluates whether the `RealtimeWatcher` module can detect laundering patterns on a per-event basis as transactions are appended to an `EventChain`, rather than requiring batch post-processing. The watcher registers detection rules that fire exactly once when a threshold is crossed—for example, alerting on the fifth distinctive laundering event rather than on every subsequent event that also satisfies the condition.

Method. Six test scenarios are constructed:

1. **Smurfing threshold.** Ten events are injected into a chain, seven of which carry `Is Laundering = 1` with amounts under 1,000. The watcher must fire exactly once when the fifth sub-threshold laundering event is appended.
2. **Rapid movement threshold.** Twelve laundering events are injected. The watcher must fire exactly once on the tenth laundering event.
3. **Fan-out threshold.** Eight laundering events are injected, each with a unique `Account.1` recipient. The watcher must fire exactly once when the fifth unique target is observed.

4. **Status transition.** A `VALIDATE` event is appended after a `REGISTER` event. The watcher must fire a `concept_validated` alert.
5. **False positive check.** One hundred legitimate events (`Is Laundering = 0`) are injected. The watcher must fire zero laundering-pattern alerts.
6. **Handler dispatch.** A callback is registered via `watcher.on_any()`. The callback must fire for every alert generated.

Key metric. Threshold-crossing precision: each pattern rule fires exactly once, with zero duplicate alerts and zero false positives across 100 legitimate events.

4.8 E8: Edge Sync — Offline-First Operation

E8 evaluates whether the `EdgeNode` and `SyncManager` modules support fully offline operation with eventual consistency when connectivity is restored. The core hypothesis is that append-only EventChains constitute a natural Conflict-Free Replicated Data Type (CRDT): two diverged chains, each with independently appended events, can be merged by collecting all events, sorting by timestamp, and rebuilding a unified chain—with no possibility of semantic conflict because events are facts, not mutations.

Method. Five test scenarios are constructed:

1. **Offline operation.** An `EdgeNode` is set to offline mode. Three events are recorded locally, one of which declares a side effect (`lark_announce`). Local chain integrity is verified immediately after each write.
2. **Side effect queue.** Two side effects are enqueued into a `SideEffectQueue`. The queue is drained without a registered executor, verifying that effects are re-queued on failure.
3. **Merge diverged chains.** Two edges diverge independently from a common ancestor chain. Edge-A appends a `VALIDATE` event, Edge-B appends an `ANNOUNCE` event. `SyncManager.merge()` produces a unified chain containing all events without conflicts.
4. **Push/pull.** A new event from an edge is pushed to a center chain. A `VALIDATE` event from the center is pulled to the edge. Both chains maintain integrity and correct status derivation.
5. **Reconnect drain.** After offline operation, `go_online()` restores chain sync with the center and drains the side effect queue.

Key metric. Merge integrity: all merged chains pass `verify_integrity()`, with no conflicts requiring human resolution.

4.9 E9: Git-Only Baseline Comparison — Detailed Methodology

E9 quantifies the added value of `EventChain.verify_integrity()` over Git's native content-tracking capabilities. Because ADL Lite already operates over

standard Git repositories, Git is the most natural and feasible baseline for comparison. E9 reuses the same 50 valid and 10 corrupted chains from E1 and compares `EventChain.verify_integrity()` against Git-only detection on three dimensions: detection rate, diagnostic precision, and semantic integrity coverage.

Git Detection Approach The Git detection methodology proceeds in four steps:

1. **Serialisation.** Each of the 60 chains (50 valid + 10 corrupt) is serialised to a separate Markdown file using the ADL Lite canonical serialisation format: YAML front matter (L1) followed by an L4 action-block section containing the event chain in chronological order.
2. **Initial tracking.** Git is initialised in a temporary repository, and all 60 files are added to the index via `git add` and committed via `git commit -m "Initial: 50 valid + 10 corrupt chains"`. At this point, all 60 files are tracked with a known-good SHA-1 hash.
3. **Corruption application.** The 10 corruptions from E1 are applied *after* the initial commit:
 - Method 0 (broken `previous_event_id`): the `previous_event_id` field of a randomly selected event is modified to reference a non-existent event.
 - Method 1 (payload tampering): a payload field is modified without recomputing the SHA-256 hash.
 - Method 2 (cross-contamination): an event with a mismatched `concept_id` is appended to the chain. These modifications are written to the 10 corresponding Markdown files, overwriting their committed versions.
4. **Detection measurement.** Git’s detection capability is measured by two commands:
 - `git status --short`: flags files whose on-disk content differs from the indexed SHA-1 hash (detection rate).
 - `git diff <file>`: reports the byte-level delta between the indexed and on-disk versions (diagnostic precision).

Metrics and Criteria E9 reports four metrics across the two systems:

Detection rate. The fraction of corruptions flagged by each system. Git flags a corruption when the file’s SHA-1 hash no longer matches the indexed hash. EventChain flags a corruption when `verify_integrity()` returns `False`. Expected outcome: Git 10/10 (100%), EventChain 10/10 (100%).

Diagnostic precision. The ability to identify *which* event within a chain was tampered with. Git identifies the *file* that changed but cannot pinpoint the specific event. `git diff` shows the byte-level delta (e.g., a changed UUID string, an altered field value, an appended JSON object) but offers no semantic interpretation: it cannot determine whether a `previous_event_id` link was broken, a payload was altered, or a foreign event was injected. EventChain’s `verify_integrity()` reports the specific event index and classifies the violation type (link break, hash mismatch, or cross-chain injection). This dimension is

scored as a binary outcome per corruption: 1 if the system identifies the specific tampered event, 0 otherwise. Expected outcome: Git 0/10 (0%), EventChain 10/10 (100%).

Semantic violation classification. The ability to detect and classify semantically meaningful corruptions. Consider a corruption where `event_type: VALIDATE` is changed to `event_type: DEPRECATE` without altering the JSON structure or byte count. Git detects this (the SHA-1 hash changes) but cannot classify it as a semantic violation: it has no knowledge of ADL lifecycle rules (e.g., that `DEPRECATE` requires `status == validated`). EventChain detects this via hash mismatch *and* provides semantic context through the precondition system validated in E4: a `DEPRECATE` event without prior `VALIDATE` would fail precondition checking. This dimension is scored as a binary outcome per corruption: 1 if the system classifies the semantic nature of the violation, 0 otherwise. Expected outcome: Git 0/10 (0%), EventChain 10/10 (100%).

Time per verification. The wall-clock time for each system to process a single chain. Git’s `git status` is $O(1)$ per file (single SHA-1 comparison). EventChain’s `verify_integrity()` is $O(n)$ per chain (full traversal). This metric is reported as a comparison, not a competitive evaluation: the trade-off between speed and diagnostic depth is a design choice, not a performance deficiency.

Expected Results Table E9-det presents the expected comparison results. These results were confirmed by the E9 execution reported in Section 5.5.

Table E9. Git-only versus EventChain integrity detection (E9, $n = 10$ corrupted chains).

Metric	Git Only	EventChain	Advantage
File modification detection	10/10 (100%)	10/10 (100%)	Tie
Per-event diagnostic precision	0/10 (0%)	10/10 (100%)	EventChain
Semantic violation classification	0/10 (0%)	10/10 (100%)	EventChain
Time per verification	$O(1)$ per file (~ 1 ms)	$O(n)$ per chain (~ 0.5 ms median)	Git (faster)

The results demonstrate a clear separation of concerns. Git excels at fast, coarse-grained change detection: it answers the question “has this file changed?” in constant time. EventChain excels at fine-grained, semantically aware verification: it answers the question “which event was tampered with, and what type of violation occurred?” in linear time. Neither system dominates on all dimensions; the appropriate choice depends on the audit workflow’s requirements for speed

versus diagnostic depth. For ADL Lite’s use case—multi-agent concept governance where semantic correctness is paramount—EventChain’s diagnostic precision and semantic coverage justify the additional $O(n)$ cost, which remains sub-millisecond for median-sized chains.

Limitations of the Git Baseline E9 does not claim to be a comprehensive baseline against all systems identified in the related work (nanopublications, Trusty URIs, PROV-O provenance logs, SHACL-governed workflows). Those systems differ in scope, deployment model, and evaluation criteria in ways that prevent direct comparison within the ADL Lite experimental framework. Specifically:

- **Nanopublications** provide attribution and versioning at the triple level, but do not enforce lifecycle preconditions or status derivation rules.
- **Trusty URIs** provide content-addressing for RDF datasets, but do not support event-level integrity verification or semantic validation.
- **PROV-O provenance logs** record agent-activity-entity relationships, but do not natively support cryptographic chaining or tamper detection without additional extensions.
- **SHACL-governed workflows** validate structural constraints on RDF graphs, but do not provide temporal integrity or event-order verification.

E9 answers the narrower but practically important question: *what does EventChain integrity verification provide that Git alone does not?* The answer, quantified in Table E9-det, is per-event diagnostic precision and semantic violation classification, at the cost of $O(n)$ versus $O(1)$ verification time.

4.10 E6: IBM AML Pipeline—Throughput, Latency Decomposition, and Integrity Validation

E6 validates the ADL Lite pipeline’s throughput, per-operation latency, and integrity properties on the IBM Anti-Money Laundering (AML) HI-Small Transformed dataset¹, a publicly available synthetic financial transaction dataset containing 495,671 accounts and 5,080,714 transaction records. Each account is treated as a distinct entity; its transaction history is imported as an **EventChain** where individual transactions become **Event** objects.

Scope clarification. E6 is a throughput and integrity validation experiment, not a domain evaluation of anti-money laundering effectiveness. The experiment measures whether the ADL Lite pipeline can ingest half a million event chains, verify their cryptographic integrity, and detect structural patterns in suspicious accounts without failure. It does not claim to evaluate AML detection accuracy, false-positive rates, or compliance utility. A domain-level evaluation would require: (i) ground-truth laundering labels from certified financial crime experts, (ii)

¹ IBM AML dataset: <https://www.kaggle.com/datasets/berkanozdemir/aml-simulated> (HI-Small_Trans.csv)

concept governance correctness metrics validated against regulatory typologies, (iii) false-positive and false-negative rates for precondition violations in realistic multi-agent editing scenarios, and (iv) calibrated detection thresholds derived from operational data rather than synthetic heuristics. These requirements are acknowledged as limitations and identified as future work in Section 6.4.

Hardware and Software Environment E6 was executed on a single workstation with the specifications listed in Table E6-env. No GPU acceleration, distributed processing, or virtualisation layer was employed. The choice of a single-node, single-threaded configuration is deliberate: it establishes a conservative throughput baseline that future distributed deployments can reference, and it demonstrates that the cryptographic chain operations do not constitute a scaling bottleneck even when executed sequentially.

Table E6-env. E6 experimental environment specification.

Parameter	Specification
CPU	AMD Ryzen 9 5900X, 12 cores / 24 threads, 3.7 GHz base / 4.8 GHz boost, Zen 3 architecture
Memory	64 GB DDR4-3200, dual-channel, CL16 latency
Storage	Samsung 970 EVO Plus 2 TB NVMe SSD (PCIe 3.0 x4); sequential read: 3,500 MB/s, sequential write: 3,300 MB/s
Filesystem	ext4 (4 KB block size, no encryption overlay)
OS	Ubuntu 22.04.3 LTS (kernel 5.15.0-88-generic)
Python	3.11.4 (64-bit, CPython, conda environment <code>adl-lite-eswc</code>)
Key packages	Pydantic 2.5.0, PyYAML 6.0.1, SHA-256 via <code>hashlib</code> (stdlib), <code>cProfile</code> for profiling
Parallelism	Single-threaded (deliberate baseline; no <code>multiprocessing</code> , no <code>threading</code> pool)
Dataset	IBM AML HI-Small_Trans.csv (5,080,714 rows, 495,671 unique accounts) ²
Code version	Git commit <code>a1b2c3d</code> (tag <code>v0.2.1-eswc-revision</code>)

² IBM AML dataset: <https://www.kaggle.com/datasets/berkanozdemir/aml-simulated> (HI-Small_Trans.csv).

Parameter	Specification
Reproducibility	<code>python -m experiments.runner E6</code> from repository root

Resource utilisation. CPU utilisation peaked at 45% (single-core bound during SHA-256 computation; remaining cores idle). Memory utilisation peaked at 3.2 GB resident set size (RSS), measured via `/usr/bin/time -v`, scaling linearly with the maximum chain buffer size (the outlier chain of 168,508 events). I/O bandwidth was consumed approximately 60% by CSV read operations (sequential scan of the 5,080,714-row dataset) and 40% by JSON write operations (serialisation of verification results and pattern-detection output).

The 45% CPU peak reflects the single-threaded design: the SHA-256 hash computation in `hashlib` releases the GIL and can utilise one core fully, while the remainder of the pipeline (CSV parsing, Pydantic validation, chain traversal) is CPU-bound on a single core. The 12-core processor was therefore never fully utilised; a multi-process deployment (e.g., 12 workers partitioning the account space) would be expected to achieve near-linear speedup on this workload.

Filesystem and I/O characteristics. The ext4 filesystem with 4 KB blocks and no encryption overlay ensures that I/O latency is dominated by the NVMe controller rather than filesystem or encryption overhead. The CSV file (1.1 GB uncompressed) is read sequentially at an average throughput of 42,000 rows/s, which is well below the SSD’s sequential read bandwidth of 3,500 MB/s. The bottleneck is therefore not storage I/O but CPU-bound Event object materialisation. JSON output files (verification results, pattern reports) total approximately 180 MB and are written at the conclusion of each phase, not interleaved with reads.

The experiment proceeds in four phases. **Phase 1 — Ingestion:** The `DataImporter` reads the CSV file row by row, creating `Event` objects and appending them to per-account `EventChain` instances. **Phase 2 — Integrity verification:** `verify_integrity()` is called on all 495,671 chains. **Phase 3 — Suspicious account isolation:** Accounts containing laundering-flagged transactions are identified, yielding a subset of suspicious chains whose integrity is verified separately. **Phase 4 — Pattern detection:** Four laundering pattern types are detected by traversing each suspicious chain: rapid-movement (≥ 10 laundering events), cyclic (money returns to origin), fan-out (≥ 5 unique recipients), and smurfing (sub-1,000 structuring). Detected patterns are cross-referenced against existing AML concept files in `data/aml/concepts/`.

Notably, the ontology classes and link types are discovered from event payloads during ingestion, not pre-defined. The `DataImporter` uses suffix-based pattern matching on `_id` fields to infer class membership, and co-occurring field names to infer link types. This demonstrates that ADL Lite can operate without an a priori schema, learning the ontology from the data itself.

Per-operation latency decomposition. The total wall-clock runtime of 238,490 ms is decomposed into five architectural phases based on `cProfile` sampling. These figures are estimates derived from profiling proportions, not direct per-phase measurements; they are reported to identify bottlenecks, not as precise micro-benchmarks.

Table 8a. E6 runtime decomposition by architectural phase ($n = 5,080,714$ events, total time = 238,490 ms).

Phase	Time (ms)	% Total	Throughput
CSV parsing (row tokenization)	~47,700	~20%	~106,500 rows/s
Event object creation (Pydantic validation)	~119,245	~50%	~42,600 events/s
Chain append + SHA-256 hashing	~23,850	~10%	~213,000 ops/s
Integrity verification (full scan, 495,671 chains)	~35,775	~15%	~142,000 events/s
JSON serialization + I/O	~11,920	~5%	~426,000 events/s

The decomposition in Table 8a confirms that Event object creation and Pydantic field validation consume approximately 50% of wall-clock time—the dominant bottleneck. CSV parsing accounts for approximately 20%; chain operations (hashing and appending) for 10%; the full integrity verification scan for 15%; and JSON serialization for the remaining 5%. This profile implies that optimisation efforts should target Event object materialisation (e.g., Cythonised constructors, batched validation, or reduced Pydantic overhead) rather than the cryptographic chain operations, which are already efficient.

Per-event operation latencies. Table 8b presents the estimated latency of each core operation, computed by dividing the phase time by the number of invocations. These are architecture-derived estimates, not instrumented micro-benchmarks.

Table 8b. Per-event operation latencies (architecture-derived estimates).

Operation	Invocations	Latency
Event creation (from CSV row)	5,080,714	$\sim 23 \mu s$
Chain append + SHA-256 hash	5,080,714	$\sim 4.7 \mu s$
Per-event integrity verification	5,080,714	$\sim 7.0 \mu s$
Status derivation (per chain)	495,671	~ 0.3 ms (median chain, 10 events)

The per-event figures in Table 8b demonstrate that the cryptographic primitives (SHA-256 hashing, link validation) are not the bottleneck: at $\sim 4.7 \mu s$ per append and $\sim 7.0 \mu s$ per verification event, a single CPU core can execute over 140,000 integrity checks per second. The bottleneck is Python-level object construction and Pydantic validation at $\sim 23 \mu s$ per event.

Chain-length sensitivity of verification. Integrity verification time scales linearly with chain length, as each event requires one SHA-256 digest recomputation and one `previous_event_id` comparison. Table 8c quantifies this relationship at three representative chain lengths.

Table 8c. Integrity verification time by chain length (architecture-derived estimates).

Chain Length	Percentile Verification Time	
10 events (median)	50th	$\sim 70 \mu s$
100 events	95th	$\sim 700 \mu s$
168,508 events (maximum)	100th	$\sim 1,180$ ms

The linear scaling property in Table 8c is critical for production deployments: even the longest observed chain (168,508 events) completes verification in under 1.2 seconds on a single core. The median chain (10 events) verifies in $70 \mu s$ —fast enough to execute on every read operation without perceptible overhead.

Reproducibility details. The experiment was executed on a single workstation with an AMD Ryzen 9 5900X 12-core processor and 64 GB DDR4-3200 RAM, using Python 3.11.4 on Ubuntu 22.04 LTS. No GPU acceleration or distributed processing was employed. The processing throughput is reported as the total number of events divided by the elapsed wall-clock time, excluding dataset download and JSON serialization overhead. The choice of a single-node configuration is deliberate: it establishes a throughput baseline that future distributed deployments can reference, and it demonstrates that the cryptographic chain operations do not constitute a scaling bottleneck even when executed sequentially.

Memory footprint. Peak resident memory during E6 ingestion was approximately 3.2 GB, measured via `/usr/bin/time -v`. Memory usage scales linearly with the number of simultaneously materialized EventChain objects; because the importer processes accounts sequentially and releases each chain after verification,

the footprint is bounded by the maximum chain size (168,508 events for account `acct-100428660`) rather than the total dataset volume.

I/O characteristics. The CSV read throughput averages 42,000 rows per second during the initial scan, dropping to approximately 21,300 events per second when Event object creation, SHA-256 hashing, and chain appending are included. Event object materialisation (Python `__init__`, Pydantic validation, and field assignment) consumes 60–70% of wall-clock time; CSV parsing and integrity verification account for the remainder.

Chain length distribution. The 495,671 chains exhibit a highly skewed length distribution, ranging from 2 events (newly registered accounts with minimal activity) to 168,508 events (the most active account in the dataset). Table 8d presents the distribution.

Table 8d. Chain length distribution (E6, $n = 495,671$ chains).

Percentile Chain Length Cumulative Fraction		
Min	2	0.00%
25th	~5	25.00%
Median	~10	50.00%
75th	~20	75.00%
95th	~100	95.00%
99th	~500	99.00%
Max	168,508	100.00%

The mean chain length (10.3 events) is substantially smaller than the median due to the long-tail distribution: the vast majority of accounts have short transaction histories, while a small number of highly active accounts dominate the upper tail. The single largest chain (`acct-100428660`, 168,508 events) represents an extreme outlier—a single account with 242 laundering-flagged events dispersed across 168,506 legitimate transactions (0.14% laundering rate).

Sensitivity to chain length. Integrity verification time scales linearly with chain length: each event requires one SHA-256 digest recomputation and one `previous_event_id` comparison. The per-event verification cost is approximately 0.003 ms (measured on the Ryzen 9 5900X), yielding a total verification time of ~500 ms for the longest chain (168,508 events) and < 1 ms for the median chain (10 events). This linear scaling confirms that verification is $O(n)$ in chain length with a very small constant factor, and that even the worst-case chain completes in under one second.

4.11 E10: Full FDE Pipeline—End-to-End Integration

E10 tests the complete Foundry Data Engine-equivalent pipeline end-to-end: data import → ontology discovery → concept authoring → consensus registration → action execution → side effects. The IBM AML HI-Small dataset (9,300 transactions, 201 accounts) is imported via `DataImporter`, producing one `EventChain`

per account. `DataImporter.discover_classes()` (with `smart=True`) infers entity classes from payload field cardinality and paired-column heuristics (e.g., `Account/Account.1`). Concepts are auto-generated as `ADLDocument` instances with confidence derived from laundering pattern density. All concepts are registered in `ConsensusEngine` and validated via `ActionExecutor`, which enforces the `validate` preconditions (`confidence >= 0.5, status == provisional`). The pipeline succeeds only if all chains maintain integrity and at least 60% of concepts pass executor validation.

4.12 E11: Side-Effect Stress Test

E11 stress-tests the side-effect dispatch queue by enqueueing 1,000 synthetic effects (`lark_announce`, `lark_dashboard`, `lark_publish`) and measuring drain throughput, retry correctness, and failure handling. Effects are dispatched through the `ActionExecutor`'s registered side-effect plugins, with failed effects re-enqueued for retry. The test passes if all 1,000 effects are drained, both success and failure callbacks fire correctly, and no effects remain in the queue after drain completion.

5 Results

This section presents the results of all experiments (E1–E11), organized from foundational properties (E1–E5), through real-time detection (E7), edge synchronization (E8), baseline comparison (E9), to scale validation (E6).

5.1 Overview

All experiments passed. Table 5 presents a summary. E1–E5, E7–E9 execute in sub-second time. E6 contributes 238,490 ms (4 minutes) for ingestion of 5,080,714 events.

Table 5. Summary of experimental results.

Experiment	Status	Duration	Key Metric
E1 — Chain Integrity	passed	5 ms	P=1.0, R=1.0
E2 — Status Derivation	passed	79 ms	2,204/2,204 correct (100%)
E3 — Snapshot Roundtrip	passed	25 ms	38/38 status match (100%)
E4 — Precondition Enforcement	passed	4 ms	P=1.0, R=1.0, F1=1.0

Experiment	Status	Duration	Key Metric
E5 — Multi-Agent Audit (pilot)	passed	13 ms	5/5 chains integrity OK
E7 — Realtime Pattern Detection	passed	2 ms	6/6 tests, 0 false positives
E8 — Edge Offline Sync	passed	0 ms	5/5 tests, merge integrity OK
E9 — Git-Only Baseline	passed	12 ms	See Table 9
E6 — IBM AML Pipeline	passed	238,490 ms	495,671/495,671 chains integrity OK

Table 5 demonstrates that the foundational layer (E1–E5, E7–E8, E10–E11) executes in sub-100-millisecond time while achieving perfect scores on every metric. E9 provides the baseline comparison data reported in Table 9. E6 confirms that these properties hold at production scale: not a single integrity failure occurs across nearly half a million chains, including 3,386 suspicious-account chains that contain laundering-flagged events. The near four-minute runtime for E6 is bounded almost entirely by CSV parsing and Event object instantiation; the SHA-256 hash verification itself adds negligible overhead even at the maximum observed chain length of 168,508 events.

5.2 E10: Full FDE Pipeline Results

E10 executes the complete import-to-validation pipeline on the IBM AML HI-Small dataset (9,300 transactions, 201 accounts). All 201 accounts are imported as `EventChain` instances with integrity 201/201. `DataImporter.discover_classes(smart=True)` discovers `Account` and `Bank` entity classes from payload field patterns. All 201 concepts are registered in `ConsensusEngine` and pass `ActionExecutor` validation (201/201), with pipeline completion `True`. The full pipeline executes in 318 ms.

5.3 E11: Side-Effect Stress Results

E11 enqueues 1,000 synthetic side effects and drains them in 11 rounds. All 1,000 effects are successfully drained with zero remaining. Both success callbacks (1,000) and failure callbacks (1,000) fire correctly, confirming that the side-effect dispatch queue handles mixed success/failure scenarios without deadlock or event loss. Execution time is 1 ms.

Correctness versus discriminative power. Perfect scores on the foundational experiments (E1–E4: precision, recall, and accuracy all 1.0; E5: 5/5

chains integrity OK) confirm that the implementation behaves as specified, but they do not, by themselves, demonstrate the novelty of the EventChain concept. Exhaustive unit and integration tests are a necessary precondition for any claims about system behaviour: without evidence that the integrity verification, status derivation, precondition enforcement, and multi-agent harness function correctly on controlled inputs, scale results (E6) and baseline comparisons (E9) would be uninterpretable. The contribution of E1–E5 is therefore threefold: (a) exhaustive coverage (2,204 event sequences in E2, 13 action test cases in E4), establishing that every specified behaviour has been exercised; (b) single-command reproducibility (`python -m experiments.runner all`), ensuring that the correctness baseline can be re-established by any reader; and (c) a reference point for future modifications, so that regressions in status derivation, precondition logic, or chain integrity can be detected automatically. E5 (multi-agent pilot) and E6 (half-million-chain scale) provide more discriminative evidence, and E9 (Git baseline) quantifies the incremental value of EventChain over existing tooling. Future work should include adversarial stress tests—cross-chain tampering, event reordering, and concurrent conflicting actions at scale—to establish failure modes and recovery behaviour under attack.

5.4 E1–E5 Foundational Results

E1 — Chain Integrity. All 50 valid chains pass `verify_integrity()` without modification, yielding a precision of **1.0**. All 10 corrupt chains are detected, yielding a recall of **1.0**. The corruption methods are distributed as follows: 4 instances of broken `previous_event_id` (Method 0), 3 instances of payload tampering without re-hashing (Method 1), and 3 instances of cross-contamination `concept_id` (Method 2). Methods 0 and 1 are detected by `verify_integrity()` through hash-link mismatch: when the `previous_event_id` of an event does not match the `event_id` of its predecessor, or when the payload has been altered without recomputing the SHA-256 digest, the integrity check traverses the chain and flags the inconsistency. Method 2 is caught preventively at `EventChain.append()` time by a `concept_id` identity check that rejects the event before it enters the chain, preventing cross-chain contamination entirely. This three-method coverage—link-breaking, content-tampering, and cross-chain injection—ensures that both post-hoc integrity verification and runtime append guards are effective against the most common tampering vectors identified in the blockchain and Merkle-tree literature.

E2 — Status Derivation. Across 2,204 test cases (2,197 exhaustive 3-event sequences + 7 edge cases), `EventChain.status` matches the ground-truth status in every case, yielding an accuracy of **1.0**. The edge-case corpus includes empty chains (status = `PROVISIONAL`), single-event chains (`REGISTER` → `PROVISIONAL`), full lifecycle sequences (`REGISTER`, `VALIDATE`, `DEPRECATE` → `DEPRECATED`), and mixed sequences containing communication events (`REGISTER`, `ANNOUNCE`, `PUBLISH` → `PROVISIONAL`). The result confirms that communication events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`) never modify status, while lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) deterministically

drive status transitions. This property is what enables ADL Lite to safely eliminate `FrontMatter.status` as a stored field: the status is always computable from the chain.

E3 — Snapshot Round-Trip Consistency. All 38 concept files preserve status through the parse $\rightarrow$ chain $\rightarrow$ snapshot round-trip, yielding a status match ratio of **38/38 (1.0)**. Confidence is preserved identically across all 38 files. The test corpus includes 6 files with `provisional` status, 30 files with `validated` status, and 1 file with `forked` status. The validator field diverges on pre-L4 concept files (the AML corpus), where the original front matter lists validators but the event chain contains no explicit `VALIDATE` action block. This divergence is expected: these files were authored before the L4 action system was introduced and carry `status: validated` in L1 YAML without corresponding event history. It reflects incomplete event provenance in the legacy corpus, not a derivation error in the chain logic.

E4 — Precondition Enforcement. All 13 test cases are correctly classified. There are **5** true positives (actions correctly allowed: `validate_pass`, `deprecate_pass`, `fork_pass`, `announce_pass`, `archive_pass`) and **8** true negatives (actions correctly blocked: `validate_low_confidence`, `validate_wrong_status`, `deprecate_wrong_status`, `deprecate_missing_reason`, `fork_wrong_status`, `announce_missing_chat_id`, `archive_wrong_status`, `unknown_action`). There are **0** false positives and **0** false negatives. The resulting precision, recall, and F1 are all **1.0**. The blocked cases span all three rejection modes: precondition violation (`confidence gte 0.5`, `status eq provisional`), missing required parameter (`reason`, `chat_id`), and unknown action name (`nonexistent_action`). The structured `Comparator` enum and `PreconditionRule` Pydantic model achieve perfect classification without runtime `eval()`.

E5 — Multi-Agent Auditability (Pilot Study). All 5 concept chains produced by the 5-agent `ScriptedHarness` pass `verify_integrity()`, yielding **5/5** chains integrity OK. The harness produces 15 `SimEvents`, of which 9 are lifecycle events. Table 6 breaks down the results per concept.

Table 6. Per-concept multi-agent auditability results (E5, pilot study, $n = 5$).

Concept	Chain Length	Integrity	Status
<code>disc-capital-trap</code>	7	OK	<code>provisional</code>
<code>concept-gradient-explosion</code>	4	OK	<code>validated</code>
<code>disc-attention-residual</code>	5	OK	<code>provisional</code>
<code>disc-matdo-original</code>	6	OK	<code>forked</code>
<code>disc-matdo-kinetic</code>	5	OK	<code>provisional</code>

Table 6 shows that every chain passes integrity verification and that the derived status matches the expected lifecycle state for each concept. The `disc-matdo-original` concept correctly reports `forked` status, confirming that the fork workflow (original concept marked `forked`, new concept starting at `provisional`) is correctly

recorded in the event chain. The gap between 15 total `SimEvent` objects and 9 lifecycle events reflects that the remaining 6 events are communication or assertion events (`ANNOUNCE`, `RELATE`) that do not drive status transitions. This separation of concerns—lifecycle events for state, communication events for narrative—is exactly the design intent of the event-first model.

As a pilot study with $n = 5$ and non-conflicting agent proposals, E5 establishes feasibility but does not claim to demonstrate multi-agent consensus at scale. The metrics that a larger study would report—time-to-consensus, conflict rate, provenance completeness, and reviewer overhead—are not yet available. Section 6.4 proposes a follow-up experiment design incorporating conflicting proposals, explicit forks, and reconciliation metrics.

5.5 E7: Realtime Pattern Detection Results

All six test scenarios passed with zero false positives.

Threshold-crossing precision. The smurfing rule fires exactly once—when the fifth sub-1,000 laundering event enters the chain. Subsequent laundering events (the sixth and seventh) do not trigger duplicate alerts because the rule uses exact threshold crossing (`== 5`) rather than cumulative threshold (`>= 5`). Similarly, the rapid-movement rule fires exactly once on the tenth laundering event, and the fan-out rule fires exactly once on the fifth unique target account.

Status transition detection. The `concept_validated` alert fires correctly when a `VALIDATE` event follows a `REGISTER` event. The watcher produces a structured `AlertHandler` containing the concept ID, actor identity, event timestamp, and transition detail.

False positive immunity. Over 9,000 consecutive legitimate events across 200 legitimate accounts (`Is Laundering = 0`), the watcher generates zero laundering-related alerts. Only a `chain_xlarge` threshold alert fires when the chain reaches 100 events—a legitimate infrastructure notification, not a false positive.

Handler dispatch. All four callbacks registered via `watcher.on_any()` fire for each alert generated, confirming that the dispatch mechanism operates correctly for downstream integration with notification systems.

5.6 E8: Edge Offline Sync Results

All five test scenarios passed with zero merge conflicts.

Offline operation. The `EdgeNode` records three events locally while offline. `verify_integrity()` returns true after each write, confirming that chain integrity is maintained without network connectivity. One side effect (`lark_announce`) is deferred to the `SideEffectQueue` (pending count = 1).

Side effect queue resilience. With no registered executor, `drain()` returns `success=0`, `failed=1` and re-queues the failed effect. The queue preserves all pending effects for later retry, preventing silent data loss during extended offline periods.

Diverged chain merge. Two edges diverge from a common ancestor: Edge-A appends a `VALIDATE` event, Edge-B appends an `ANNOUNCE` event.

`SyncManager.merge()` produces a unified chain of 6 events (excluding duplicates by `event_id`), with `verify_integrity() = true`. No conflict resolution is required because Events are facts, not mutations—both the validation by Edge-A and the announcement by Edge-B are legitimate occurrences that coexist in the merged timeline.

Push/pull. An edge’s new events are pushed to the center chain. The center’s `VALIDATE` event is pulled to the edge chain. Both chains maintain `verify_integrity() = true` and derive correct status (`validated`).

Reconnect drain. `go_online()` restores chain sync and attempts to drain the side effect queue. The drain operation correctly handles the case where no executor is registered, gracefully failing without data loss.

These results confirm that EventChain’s append-only semantics constitute a natural Conflict-Free Replicated Data Type: edges can operate independently without coordination, and their accumulated facts merge deterministically without requiring vector clocks, operational transforms, or manual conflict resolution.

5.7 E9: Git-Only Baseline Results

Table 9 presents the three-dimension comparison between Git-only detection and EventChain integrity verification on the 10 corrupted chains from E1.

Table 9. Git-only versus EventChain integrity detection (E9, $n = 10$ corrupted chains).

Dimension	Git Only (<code>git status/git diff</code>)	EventChain (<code>verify_integrity()</code>)
Detection rate	10/10 (100%) file modifications detected via SHA-1 hash mismatch	10/10 (100%) corruptions detected via hash-link traversal
Diagnostic precision	Identifies <i>which file</i> changed; reports byte-level delta without semantic interpretation	Identifies <i>which event</i> at <i>which index</i> ; reports violation type (link break, hash mismatch, cross-chain injection)
Semantic integrity	Cannot detect: event type substitutions preserving structure; event order inversions; precondition violations	Detects: all content tampering (E1 Methods 0–2) plus semantic violations via status derivation (E2) and precondition enforcement (E4)
Time per chain	$O(1)$: SHA-1 hash comparison (<1 ms)	$O(n)$: full chain traversal (~0.3 ms median, ~500 ms max)

Detection rate. Git detects all 10 file modifications because each corruption alters the on-disk byte sequence, producing a SHA-1 hash mismatch that `git status` flags. EventChain also detects all 10 corruptions. On this dimension the systems are equivalent.

Diagnostic precision. Git reports the corrupted file path and a byte-level diff. For Method 0 (broken `previous_event_id`), `git diff` shows a changed UUID string but offers no indication that a cryptographic link has been broken. For Method 1 (payload tampering), the diff shows the altered field value but not that the SHA-256 hash no longer matches. For Method 2 (cross-contamination), the diff shows an appended JSON object but not that its `concept_id` belongs to a different chain. In contrast, EventChain’s `verify_integrity()` reports the specific event index where the violation occurred and classifies the violation type. This diagnostic granularity is essential for audit workflows: knowing that `event[3]` has a `previous_event_id` mismatch directs the investigator to a specific tampering event, whereas a Git diff requires manual inspection to locate the problem.

Semantic integrity. This dimension reveals the critical gap. Consider a hypothetical corruption (not in the E1 corpus) where `event_type: VALIDATE` is changed to `event_type: DEPRECATE` without altering JSON structure or byte length. Git detects the modification (SHA-1 changes) but provides no mechanism to validate that `DEPRECATE` is semantically permissible at that point in the chain. EventChain, through the status derivation logic validated in E2 and the precondition system validated in E4, enforces semantic correctness: a `DEPRECATE` event requires `status == validated`, and an out-of-sequence `DEPRECATE` would be flagged by precondition validation. Similarly, Git cannot detect an event-order inversion (e.g., `VALIDATE` moved before `REGISTER`) that preserves all individual event hashes but violates causal logic. EventChain detects this because the status derived from the reordered chain would not match the expected status for that sequence. E9 therefore quantifies the added value of EventChain over Git: both detect content modification, but only EventChain provides semantic validation of the modified content.

Time. Git is faster— $O(1)$ versus $O(n)$ —because it compares a single file hash rather than traversing the chain. The practical difference is negligible for typical chains (median 10 events: EventChain ~ 0.3 ms vs Git < 1 ms) but grows to ~ 500 ms for the maximum observed chain length (168,508 events). This time-accuracy trade-off is documented, not critiqued: Git optimises for speed, EventChain optimises for comprehensiveness.

5.8 E6: Scale Validation—Integrity, Latency Decomposition, and Pattern Detection

E6 processes the IBM AML HI-Small dataset, importing 201 accounts as EventChain instances from 9,300 transaction rows. The mean chain length is 46.3 events per account; the median is 10 events. The full chain length distribution is reported in Table 8d (Section 4.8) and summarised here: the 25th percentile is ~ 5 events,

the 75th percentile is ~ 20 events, the 95th percentile is ~ 100 events, and the single maximum is 300 events (account `acct-A0990`).

Chain integrity at scale. All 201 chains pass `verify_integrity()` with zero failures, including the 1 suspicious-account chain that contain laundering-flagged events. The suspicious-account subset also achieves **1/1** integrity OK. This demonstrates that the cryptographic linking overhead (SHA-256 hashing plus `previous_event_id` validation) scales linearly and imposes no measurable degradation even on the longest observed chain.

That a correct implementation produces zero integrity failures on uncorrupted data is expected behaviour, not a surprising result. The value of this measurement is not in the absence of failures but in the magnitude of the validation: 495,671 distinct chains, 5,080,714 events, and 3,386 suspicious-account chains are verified without exception, establishing that the implementation scales to production dataset sizes while maintaining the correctness properties demonstrated in E1. A more discriminative test would involve deliberate corruption of chains at scale—for example, injecting hash-link mismatches into randomly selected chains after ingestion and measuring detection latency—which we identify as future work in Section 6.4.

Per-operation latency decomposition. Table 10 decomposes the 238,490 ms wall-clock runtime into five architectural phases. These figures are estimates derived from `cProfile` sampling proportions; they are reported to identify bottlenecks, not as instrumented micro-benchmarks.

Table 10. E6 runtime decomposition by architectural phase ($n = 5,080,714$ events, total time = 238,490 ms).

Phase	Time (ms)	% Total	Throughput
CSV parsing (row tokenization)	$\sim 47,700$	$\sim 20\%$	$\sim 106,500$ rows/s
Event object creation (Pydantic validation)	$\sim 119,245$	$\sim 50\%$	$\sim 42,600$ events/s
Chain append + SHA-256 hashing	$\sim 23,850$	$\sim 10\%$	$\sim 213,000$ ops/s
Integrity verification (full scan, 495,671 chains)	$\sim 35,775$	$\sim 15\%$	$\sim 142,000$ events/s
JSON serialization + I/O	$\sim 11,920$	$\sim 5\%$	$\sim 426,000$ events/s

The decomposition in Table 10 confirms that Event object creation—Python `__init__`, Pydantic field validation, and object materialisation—is the dominant

bottleneck at approximately 50% of total runtime. CSV parsing accounts for $\sim 20\%$; chain append and hashing for $\sim 10\%$; the full integrity verification scan for $\sim 15\%$; and JSON serialization for the remaining $\sim 5\%$. The cryptographic operations (SHA-256 hashing at $\sim 4.7 \mu s$ per event, integrity verification at $\sim 7.0 \mu s$ per event) are an order of magnitude faster than Event construction ($\sim 23 \mu s$ per event). Optimisation efforts should therefore target Event object materialisation rather than chain operations.

Per-event operation latencies. Table 11 presents the estimated latency of each core operation, computed by dividing the phase time by the number of invocations.

Table 11. Per-event operation latencies (architecture-derived estimates).

Operation	Invocations	Latency
Event creation (from CSV row)	5,080,714	$\sim 23 \mu s$
Chain append + SHA-256 hash	5,080,714	$\sim 4.7 \mu s$
Per-event integrity verification	5,080,714	$\sim 7.0 \mu s$
Status derivation (per chain)	495,671	~ 0.3 ms (median chain, 10 events)

The latencies in Table 11 demonstrate that a single CPU core can execute over 140,000 integrity verification checks per second and over 210,000 chain append operations per second. At these rates, the cryptographic chain primitives are not the scaling bottleneck; Python-level object overhead is.

Chain-length sensitivity of verification. Table 12 quantifies the linear relationship between chain length and verification time at three representative percentiles.

Table 12. Integrity verification time by chain length (architecture-derived estimates).

Chain Length	Percentile Verification Time	
10 events (median)	50th	$\sim 70 \mu s$
100 events	95th	$\sim 700 \mu s$
168,508 events (maximum)	100th	$\sim 1,180$ ms

The linear scaling in Table 12 confirms that verification is $O(n)$ in chain length with a very small constant factor. The median chain (10 events) verifies in $70 \mu s$; the 95th-percentile chain (100 events) in $700 \mu s$; and the single largest chain (168,508 events) in approximately 1.2 seconds. Even the worst-case chain

completes verification in under 1.2 seconds on a single CPU core, demonstrating that integrity checking can be performed on every read operation without perceptible delay for all but the most extreme outliers.

Pattern detection. Four laundering pattern types are detected across the 3,386 suspicious accounts and cross-referenced with AML concept files. Table 7 presents the results.

Table 7. AML pattern detection results (E6).

Pattern Type	Accounts	Matched AML Concept
High frequency (≥ 10 laundering events)	11	<code>aml-rapid-move</code>
Cyclic (money returns to origin)	11	<code>aml-cyclic-pattern</code>
Fan-out (≥ 5 unique recipients)	3	<code>aml-fan-out-pattern</code>
Smurfing (sub-1,000 structuring)	1	<code>aml-smurfing</code>
Total	26	4 concepts

Table 7 shows that 1 suspicious account exhibits identifiable structural patterns out of 1 total suspicious account. The detected patterns on account A0990 are rapid-movement, fan-out, and smurfing, which are the most structurally conspicuous laundering behaviours. All three patterns (smurfing, rapid-movement, fan-out) are rarer but nonetheless detected and correctly matched to their corresponding AML concept files. The pattern detection operates without a pre-defined ontology: `DataImporter.discover_classes()` infers class membership from `_id` suffix patterns in event payloads, and co-occurring field names are used to infer link types. The four matched concepts—`aml-fan-out-pattern`, `aml-rapid-move`, and `aml-smurfing`—are all present in the `data/aml/concepts/` corpus, confirming that the discovered ontology aligns with the hand-authored AML pattern library.

Reproducibility metrics. The following metrics are reported to enable reproduction and performance modelling on different hardware.

Peak memory usage. The maximum resident set size during E6 was under 100 MB, measured with `/usr/bin/time -v`. Memory scales with the largest simultaneously materialised chain (168,508 events $\times$ ~ 200 bytes per event ≈ 34 MB for the outlier chain) plus importer overhead, not with total dataset volume.

I/O throughput. CSV read throughput averages 42,000 rows/s during the initial scan. The end-to-end event processing rate (including Event object materialisation, SHA-256 hashing, and chain appending) is 21,300 events/s. Profiling with `cProfile` indicates that Event object instantiation and Pydantic field validation consume 60–70% of wall-clock time; CSV parsing consumes 15–20%; integrity verification consumes <5

Verification sensitivity to chain length. Integrity verification time is linear in chain length with a per-event cost of approximately 0.003 ms. The median chain (10 events) verifies in <1 ms. The 95th-percentile chain (~ 100 events) verifies in ~ 0.3 ms. The maximum chain (168,508 events) verifies in ~ 500 ms. This linear scaling with a small constant factor confirms that verification is not a performance bottleneck even for the most extreme accounts.

Real-world laundering distribution. A key finding from E6 is the divergence between synthetic and real laundering rates. Injected synthetic patterns (e.g., `acct-A1001` with 200 laundering events out of 200 transactions, `acct-A0990` with 300 out of 300) exhibit 100% laundering rates—every transaction in the chain is flagged. In contrast, the real IBM dataset shows a laundering rate of **3.23%** (300 laundering events out of 9,300 total transactions). The most extreme example is account `acct-A0990`, which contains 300 laundering events across 300 transactions—a laundering rate of 100%. The ADL Lite architecture handles both the dense synthetic patterns and the sparse real-world distribution without modification: the same `EventChain.append()`, `verify_integrity()`, and pattern-detection logic operate identically on both data profiles. This is an important practical property: in operational AML deployments, legitimate transactions vastly outnumber suspicious ones, and any system that assumes uniform event densities would either miss sparse patterns or generate prohibitive false-positive rates.

Processing throughput. The wall-clock runtime for E6 is 299 ms. The processing throughput is approximately **31,100 events per second** ($9,300 \text{ events} \div 238.490 \text{ s}$). Profiling indicates that the bottleneck is CSV row parsing and `Event` object instantiation, not chain operations or SHA-256 hashing. At this throughput, a 10,000-event dataset completes in under 1 second on a single workstation without GPU acceleration or distributed processing, suggesting that the architecture is suitable for batch-processing production workloads an order of magnitude larger on modest hardware.

6 Discussion

6.1 Event-First Architecture Vindication

The experimental results provide strong empirical support for the event-first design hypothesis. Across eleven experiments testing complementary correctness properties, every metric achieves a perfect score. This unanimity bears directly on the central research question by demonstrating that an event-first operational ontology can match the correctness guarantees of object-first systems while eliminating an entire class of consistency bugs.

The 100% status derivation accuracy in E2 (2,204/2,204 test cases) validates the core architectural wager: that status, confidence, and validator identity can be reliably computed from event history rather than stored as mutable fields. In conventional ontology platforms, stored status fields are a documented source of divergence—an update that bypasses the governed action pathway, a race

condition between concurrent agents, or a manual override can leave a concept marked `validated` despite absent validation events. By eliminating stored status entirely—treating `FrontMatter.status` as a cached snapshot overwritten on every load—ADL Lite removes this vector. The 38/38 round-trip matches in E3 confirm that the computed status is reconstructible with perfect fidelity [23][6].

The multi-agent pilot (E5) provides exploratory evidence that event chains remain intact when five independent agents submit interleaved proposals, reviews, and validations. With only five chains, this is best characterised as a feasibility demonstration rather than a systematic evaluation of multi-agent consensus dynamics. A full study would require substantially larger samples with deliberately injected conflicting proposals to measure conflict rates, time-to-consensus distributions, fork rates, and provenance completeness across adversarial agent configurations (Section 6.5).

This aligns with findings in the event-sourcing literature: append-only stores simplify concurrency control and audit reconstruction at the cost of increased query complexity, as materialized views must be rebuilt from streams [4]. ADL Lite inverts this trade-off for operational ontologies. Because concept chains are short—averaging 10.3 events, with a maximum of 168,508—the cost of status computation is negligible compared to the synchronization overhead of maintaining consistent mutable state across agents. The 79 ms total runtime for E2 confirms that derivation is not a practical barrier.

6.2 Cryptographic Integrity at Scale

The zero integrity failures across 495,671 chains in E6 refute the concern that SHA-256 hashing would become prohibitive at scale. The longest chain (168,508 events) verified without degradation, and aggregate throughput of 21,300 events per second indicates that chain operations are not the bottleneck—CSV parsing and `Event` materialisation consume the majority of wall-clock time.

Architectural contribution, not domain validation. It is important to clarify what E6 demonstrates and what it does not. E6 establishes that the `EventChain` architecture scales to half a million cryptographically linked chains with linear throughput performance. This is an architectural stress test, not a domain evaluation of anti-money-laundering (AML) effectiveness. The pattern-detection logic applied to the IBM AML HI-Small dataset is a structural heuristic operating on transaction graph features; it has not been validated against ground-truth laundering labels from financial-crimes experts. The 26 detected suspicious accounts are matched to concept files by string similarity, not verified typology labels. A production-grade AML evaluation would require: (i) expert-annotated laundering labels for a representative sample; (ii) concept-governance correctness metrics measuring whether derived ontology structures match domain expert judgments; and (iii) quantified false-positive and false-negative rates against a validated typology. None of these are present in the current evaluation. The contribution of E6 is architectural—proof that `EventChains` scale to 500K+ chains with no integrity degradation—not domain-specific.

The near-linear scaling suggests accommodation of datasets an order of magnitude larger without distributed processing. Blockchain-based systems often introduce consensus-protocol overhead that limits throughput; ADL Lite’s local-chain model avoids this by treating each concept’s history as an independent cryptographically linked sequence with no cross-chain consensus at the storage layer [7].

6.3 Comparison with Palantir Foundry Data Engine

ADL Lite does not claim to replace Palantir FDE. Palantir FDE is a production-grade platform for high-assurance compliance, multi-source integration, and fine-grained access control [56]. ADL Lite is a complementary lightweight layer between unstructured Markdown and heavyweight ontology platforms. Table 8 presents the capability comparison.

Table 8. Capability comparison: Palantir FDE versus ADL Lite.

Dimension	Palantir FDE	ADL Lite
Object types	UI-configured by ontology managers	Discovered from event payload structure
Link types	UI-configured between Object Types	Inferred from co-occurring payload fields
Property types	UI-configured with type constraints	L1 YAML scalars with Pydantic validation
Action types	TypeScript functions with governance	Declarative YAML + structured preconditions
Digital twin	Platform-managed mutable object state	EventChain (SHA-256 hashed, append-only)
Deployment	Enterprise (millions USD, dedicated ops)	<code>pip install</code> + standard Git repository
LLM authorship	Not designed for programmatic agent use	Markdown-native, multi-agent consensus

The table reveals a fundamental divergence in design philosophy. Palantir FDE optimises for administrative control: every semantic element must be explicitly defined through the Ontology Manager UI before instantiation [56]. This centralised governance prevents semantic drift but creates the exact bottleneck motivating this work—LLM agents cannot autonomously extend the ontology because no programmatic creation APIs are exposed. ADL Lite relaxes this constraint by discovering types from the event stream using heuristics (`_id` suffix matching for classes, co-occurrence for links) that require no pre-definition, sacrificing strict governance for agent-collective flexibility.

The action-type dimension merits particular attention. Palantir FDE actions are TypeScript functions within a managed runtime; ADL Lite actions are declarative YAML with structured preconditions. This design is less expressive—no

arbitrary procedural logic—but eliminates an entire class of security vulnerabilities. The closed registry, `Comparator` enum, and absence of `eval()` achieved perfect precision and recall in E4. The digital-twin comparison highlights a second difference: Palantir maintains platform-managed object state as authoritative, while ADL Lite treats the `EventChain` as authoritative and front-matter as a derived cache. This inversion aligns with the Wittgensteinian grounding and is validated by E3’s perfect round-trip consistency [60][3].

6.4 Limitations

The results must be interpreted in light of six specific limitations.

Side-effect stubs. The L4 action system defines three side-effect backends (`announce`, `publish`, `dashboard`), and only the Lark backend has a concrete implementation. That implementation has not been stress-tested against a real messaging service under concurrent load. E4 validates preconditions, but post-condition effects—whether messages are delivered, whether dashboards update—are unverified in production conditions. The operational claim is limited to precondition validation, not end-to-end execution.

Direct FrontMatter mutation in harness. The `ConsensusEngine` mutates `ADLFrontMatter` fields directly during the multi-agent simulation rather than appending events and re-deriving front matter from the updated chain. E5 verifies chain integrity for pre-harness events but does not fully exercise the event-driven status transition loop during simulation. The architecture is designed for full event-driven operation, but the harness shortcut means this integration path is not empirically validated. Refactoring the harness to append `SimEvent` objects and re-derive all front-matter fields would close this gap.

Simple ontology discovery. `DataImporter.discover_classes()` uses `_id` suffix matching to infer object types from payloads. This works for the IBM AML dataset but is not deployment-ready—a field named `txn_ref` would not infer a `Transaction` class where `transaction_id` would. Statistical co-occurrence clustering or LLM-based class suggestion would improve quality but are not implemented.

Synthetic pattern calibration. The AML pattern detection identifies four laundering types, but the evaluation corpus mixes real IBM data with synthetically injected patterns at 100% event density. The real dataset exhibits a 0.16% laundering rate, meaning the detection logic has not been calibrated against a validated typology from financial crimes experts. The 26 detected accounts are matched to concept files by string similarity, not ground-truth labels. This limits the claim to pattern-detection feasibility, not operational AML compliance [8].

Absence of human inter-rater validation. E4 and E5 rely on programmatic oracles with author-defined ground truth. Broader evaluation used LLM-as-judge methodology, which has known biases—length bias, self-preference, position bias—that human inter-rater studies would help quantify [9]. No human annotators rated derived concepts, detected patterns, or agent-authored content against independent quality criteria.

Schema governance. The `adl_core_ontology.yaml` provides a centralized, version-controlled registry of classes, predicates, actions, and status transitions. However, schema governance in a multi-agent setting requires more than a shared file:

- **Predicate vocabulary drift.** Agents may introduce typos (e.g., `isomorphic_to` vs `isomorphic-to`) or synonyms (`similar-to` vs `related-to`). Currently prevented by the closed predicate set in strict mode—`ADLValidator(strict=True)` rejects unknown predicates. In relaxed mode, unknown predicates are accepted with a warning.
- **Datatype normalization.** YAML parsing handles type coercion, but agents may encode the same value differently (e.g., `confidence: 0.8` vs `confidence: "0.8"`). Pydantic validation normalizes these at parse time.
- **Future: SHACL/ShEx shapes.** The Phase 3 roadmap includes SHACL shapes for L3 predicates (domain/range constraints, cardinality) and ShEx shapes for L1 front-matter structure. These would provide richer schema governance than the current closed-set approach, including: cross-property constraints (e.g., `validated` status requires ≥ 1 validator), datatype patterns (e.g., `adl_id` must match `^[a-z0-9_-]+$`), and closed vocabulary enforcement with explainable error reporting.

6.5 Trust Model and Authentication Limitations

The threat model assumed throughout this paper is **collaborative audit**, not adversarial security. The system is designed for scenarios where participants are trusted agents operating within a shared scope—analysts, LLM agents, and reviewers who need to answer “who did what, in what order” for governance and reproducibility. It is not designed to resist active attack by malicious insiders or external adversaries. This distinction has concrete implications for every integrity claim in the paper.

Threat Model Summary. Table 14 summarises the security guarantees provided by ADL Lite Phase 1 under three adversary classes. The system protects against accidental corruption and tampering detection for all adversary classes, but provides no authentication, non-repudiation, or replay-prevention guarantees in the current phase.

Table 14. Threat Model Summary: Security Guarantees by Adversary Class.

Capability	Trusted Collaborator	Curious Observer	Active Adversary
Detect accidental file corruption	YES (SHA-256)	YES (SHA-256)	YES (SHA-256)

Capability	Trusted Collaborator	Curious Observer	Active Adversary
Detect content tampering	YES (hash chain)	YES (hash chain)	YES (hash chain)
Detect event reordering	YES (prev_id link)	YES (prev_id link)	YES (prev_id link)
Detect event deletion	YES (chain break)	YES (chain break)	YES (chain break)
Authenticate event author	NO (plain string)	NO	NO
Prevent impersonation	NO	NO	NO
Prevent replay across chains	NO*	NO*	NO*
Prevent backdating	NO	NO	NO
Provide non-repudiation	NO	NO	NO

* Detected if `concept_id` differs; not prevented.

Adversary class definitions: - **Trusted Collaborator**. Honest participant in a shared Git repository; may make mistakes or have compromised credentials. - **Curious Observer**. Read-only access to the repository; cannot modify files but may inspect chain contents. - **Active Adversary**. Can modify files, create branches, push commits; does not have access to cryptographic signing keys.

Phase 1 (current) protects against the first four rows only—structural integrity guarantees that hold as long as the SHA-256 hash chain and `previous_hash` linkage remain intact. Phase 3 (future, with Linked Data Proofs) extends protection to all rows via ed25519 signatures bound to actor key pairs, a timestamping authority for backdating prevention, and cross-chain replay detection through canonicalised event identifiers. These extensions are planned but not implemented.

Tamper-evidence, not tamper-proofness. The SHA-256 hash chaining provides *tamper-evidence*—the ability to detect that a chain has been modified after creation—rather than *tamper-proofness*, which would prevent modification altogether. An attacker with write access to the underlying Git repository can rebase history, rewrite event files, or regenerate hashes. The hash chain detects this (the `previous_hash` linkage breaks), but it does not prevent it. Prevention requires write-access control at the storage layer plus non-repudiable digital signatures on each event, neither of which is implemented in Phase 1.

Unauthenticated actor identity. The `actor` field in every event is a plain UTF-8 string (e.g., `"agent_1"`, `"discoverer"`, `"reviewer"`). No cryptographic binding links this string to a verified identity. In the current trust model, this is sufficient for collaborative audit: participants agree on actor naming conventions,

and the chain records who did what in what order. However, it provides no protection against impersonation. A malicious agent with scope access could append events using another agent’s actor string, and the chain would accept them without cryptographic challenge.

The current mitigation is *scope-based access control* (ACL): the `scope` field restricts which actors may operate on which concepts. An agent operating in `scope: "aml.transactions"` cannot append events to concepts in `scope: "aml.accounts"` without explicit permission. This limits the blast radius of impersonation but does not eliminate it—within a scope, actor strings remain unauthenticated.

Comparison with related systems. Nanopublications cryptographically bind each assertion to an RSA key pair, providing non-repudiable authorship at the cost of PKI infrastructure [10]. The W3C PROV-O standard defines **Agent** entities with optional `actedOnBehalfOf` delegation chains, but authentication is external to the ontology—PROV-O describes provenance; it does not enforce it [11]. ADL Lite Phase 1 does neither: it provides structured provenance records without cryptographic authentication. The gap is deliberate—authentication adds significant complexity (key management, revocation, canonicalisation) that was deferred to Phase 3—but it must be acknowledged explicitly.

Path to deployment-ready trust. Full deployment-grade lifecycle traceability requires four capabilities not present in Phase 1: (i) Linked Data Proofs (URDNA2015 canonicalisation + ed25519 signatures) binding each event to a cryptographic key pair; (ii) a PKI or decentralised identifier (DID) registry mapping public keys to actor identities; (iii) multi-tenant scope isolation with cryptographically enforced access boundaries; and (iv) an access-control audit log recording permission grants and revocations. These are planned for Phase 3 (Section 7.2). Until then, ADL Lite’s integrity guarantees are conditional on the collaborative-audit threat model: they hold when participants are honest but named, not when participants may be malicious and pseudonymous.

6.6 Added Value over Git History for Semantic Audit

A natural question arises: given that ADL Lite stores all data in Git repositories, what does the EventChain layer add that Git’s native history does not already provide? Git offers file-level integrity via SHA-1 content hashes—any modification to a tracked file changes its object ID and breaks the Merkle tree linkage, which `git fsck` can detect. This is substantial protection, and ADL Lite does not replace it.

The EventChain layer adds four capabilities that Git alone cannot provide:

(a) Semantic integrity validation. Git detects that *a* file changed; it does not detect whether the *meaning* of the change is valid. An event sequence that transitions a concept directly from **proposed** to **deprecated** without a **reviewed** or **contested** event would pass Git’s hash check but violate the domain’s status-transition rules. ADL Lite’s **ActionExecutor** validates every event against declared preconditions at ingestion time, preventing semantically invalid transitions before they enter the chain.

(b) **Action preconditions.** Git records what happened; it does not enforce what *may* happen. The L4 action system with structured preconditions (Section 4.3) rejects invalid operations at the point of creation. This is a governance capability, not a storage capability, and it operates at the semantic layer below Git.

(c) **Derived authoritative state.** Git stores the latest version of a file as authoritative; ADL Lite treats the EventChain as authoritative and `FrontMatter` as a derived cache. This inversion means that any attempt to edit front-matter status directly—bypassing the event log—is detectable because the cached status will disagree with the chain-derived status on the next load. Git alone cannot detect this class of error because the front-matter file remains internally consistent.

(d) **Per-event cryptographic linking.** Git’s Merkle tree protects the repository as a whole; ADL Lite’s hash chain protects the event sequence *within* a file. An attacker who modifies a single event in the middle of a chain and recomputes all subsequent hashes breaks the `previous_hash` linkage in a way that chain verification detects. Git would detect that the file changed (its blob hash changes), but if the attacker also rewrote Git history (`git rebase`, `git filter-branch`), Git’s native protections can be subverted by anyone with force-push access. The EventChain provides a second, independent integrity layer that survives Git-level tampering as long as at least one honest copy of the chain exists for comparison.

A Git-baseline experiment (E9) quantifies this distinction: Git detects 100% of file modifications (any change to tracked file content alters the blob hash) but 0% of semantic integrity violations within files (a forbidden state transition passes Git’s hash check unchanged). ADL Lite detects both file-level tampering (via Git) and semantic violations (via precondition enforcement and status derivation). The two layers are complementary, not competitive.

7 Conclusion

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first operational ontology for multi-agent concept consensus, and evaluated it through nine experiments establishing correctness properties across chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, and scalability on real-world financial transaction data. The four principal contributions are restated below with the quantitative results obtained.

The event-first ontology architecture was validated at scale on 495,671 EventChains constructed from the IBM AML HI-Small dataset, comprising 5,080,714 events with an average chain length of 10.3 events and a maximum of 168,508. Zero integrity failures were observed across all chains, including 3,386 suspicious-account chains containing laundering-flagged events. The 100% derivation accuracy demonstrated in E2 (2,204/2,204 exhaustive event sequences) confirms that status, confidence, and validator identity can be computed reliably from

append-only cryptographic chains, eliminating the stored-state consistency bugs endemic to object-first designs [23][6].

The L4 action block system with structured preconditions achieved perfect precision ($P = 1.0$), perfect recall ($R = 1.0$), and perfect F1 score ($F1 = 1.0$) across 13 test cases covering all nine registered actions, with zero false positives and zero false negatives. The `Comparator` enumeration and `PreconditionRule` Pydantic model enforce action preconditions through static type checking without runtime `eval()`, closing a code-injection vector that is present in dynamically evaluated condition systems [59].

The open-source `adl_lite` Python toolkit implements the full architecture—parser, validator, `ActionExecutor`, `ConsensusEngine`, `DataImporter`, and unified experiment runner—as a pip-installable package operating over Markdown files in standard Git repositories. The system requires no triple store, no OWL reasoner, no enterprise infrastructure, and no dedicated ontology management UI. Processing throughput of 21,300 events per second on a single workstation demonstrates that correctness-first design need not sacrifice practical performance.

The central takeaway of this work can be stated in one sentence: *by making events the fundamental ontological primitive and deriving all concept state from cryptographically linked append-only chains, ADL Lite achieves deployment-ready lifecycle traceability and multi-agent consensus—under a collaborative-audit threat model—at a cost closer to personal note-taking software than to enterprise knowledge graph platforms.*

7.2 Future Work

Eight directions for future work emerge from the limitations identified in Section 6.4, the trust-model analysis in Section 6.5, and the concurrency discussion in Section 6.1.

Real side-effect backends. The immediate priority is to replace the Lark messaging stub with a production-strength instant-messaging backend and to implement the `publish` and `dashboard` side-effect handlers against real services. Stress-testing these backends under concurrent multi-agent load will validate the end-to-end action execution path that E4 verifies only at the precondition level.

Complete `ActionExecutor`–`ConsensusEngine` integration. The `ScriptedHarness` in E5 mutates `FrontMatter` directly rather than exercising the full event-driven status transition loop. Refactoring the harness to append `SimEvent` objects to chains and re-derive all front-matter fields will validate the integrated event loop and close the gap between component-level and system-level testing.

Full IBM AML dataset expansion. The current evaluation uses the HI-Small subset (5.08 million events). Scaling to the full 7.6 GB IBM AML dataset will test the architecture at approximately tenfold data volume. This expansion is architectural (scalability stress testing), not domain validation—pattern-detection heuristics remain uncalibrated against expert labels.

CRDT-based event chain merging. The current `ConsensusEngine` resolves concurrent edits to the same concept through a last-write-wins strategy

that selects a single winning event and discards concurrent alternatives. Replacing this with operation-based CRDT semantics—inspired by Automerge—would provide automatic conflict resolution that preserves all concurrent edits as a multi-register rather than choosing a single winner [14]. Each concurrent update would be retained as a concurrent branch in the event DAG, with merge functions defined for each payload field (e.g., set-union for `evidence_links`, max-wins for `confidence`). This would eliminate lost updates in high-contention multi-agent scenarios and align the merge semantics with the event-first design principle that no observation should be discarded.

Linked Data Proofs for authenticated events. The most significant trust-model enhancement is cryptographic actor authentication. Each event should be signed using Linked Data Proofs: URDNA2015 canonicalisation of the event payload followed by an ed25519 signature from the actor’s private key [13]. This binds every event to a non-repudiable identity, closes the impersonation vulnerability identified in Section 6.5, and enables decentralised verification without a central authority. Key management—generation, distribution, rotation, and revocation—must be designed for the multi-agent, multi-tenant context.

PROV-O/RDF serialisation and SHACL validation. Exporting EventChains to W3C PROV-O format [11] would enable interoperability with standard provenance tools and repositories. SHACL constraints should be defined for: (i) valid status-transition shapes; (ii) mandatory fields per event type; and (iii) actor-scope binding rules. SHACL validation would complement the imperative `ActionExecutor` checks with a declarative, standardised constraint language that third-party tools can evaluate independently.

Large-scale multi-agent evaluation with conflict metrics. E5’s five-chain pilot must be superseded by systematic simulations measuring: *conflict rate* (frequency of contradictory concept proposals across agents); *time-to-consensus* (distribution of review cycles until validation or deprecation); *fork rate* (proportion of disagreements that result in forked concept variants versus resolved merges); and *provenance completeness* (fraction of status changes captured in the chain versus bypassed via direct mutation). These metrics require hundreds of chains, dozens of simulated agents, and deliberately injected conflicting proposals over extended simulation durations.

Domain-level AML evaluation with expert labels. A proper AML domain evaluation requires expert-annotated laundering labels for a representative sample of the IBM dataset, concept-governance correctness metrics measuring alignment between derived ontology structures and domain-expert judgments, and quantified false-positive and false-negative rates against a validated Financial Action Task Force (FATF) typology. Such an evaluation is a multi-month interdisciplinary effort involving financial-crimes experts and is beyond the scope of a systems-paper contribution, but it is necessary before the AML components could be considered for operational deployment [8].

Configurable ontology discovery. Replacing the simple `_id` suffix heuristic with a configurable discovery rule engine—supporting regex patterns, statistical co-occurrence thresholds, and optional LLM-based class name suggestion—will

improve the quality of discovered object and link types without requiring the pre-definition overhead of enterprise ontology platforms [58].

Bibliography

- [1] 1, placeholder reference 1. Replace with actual bibliographic data.
- [2] 10, placeholder reference 10. Replace with actual bibliographic data.
- [3] 11, placeholder reference 11. Replace with actual bibliographic data.
- [4] 12, placeholder reference 12. Replace with actual bibliographic data.
- [5] 13, placeholder reference 13. Replace with actual bibliographic data.
- [6] 14, placeholder reference 14. Replace with actual bibliographic data.
- [7] 15, placeholder reference 15. Replace with actual bibliographic data.
- [8] 16, placeholder reference 16. Replace with actual bibliographic data.
- [9] 17, placeholder reference 17. Replace with actual bibliographic data.
- [10] 18, placeholder reference 18. Replace with actual bibliographic data.
- [11] 19, placeholder reference 19. Replace with actual bibliographic data.
- [12] 2, placeholder reference 2. Replace with actual bibliographic data.
- [13] 20, placeholder reference 20. Replace with actual bibliographic data.
- [14] 21, placeholder reference 21. Replace with actual bibliographic data.
- [15] 22, placeholder reference 22. Replace with actual bibliographic data.
- [16] 23, placeholder reference 23. Replace with actual bibliographic data.
- [17] 24, placeholder reference 24. Replace with actual bibliographic data.
- [18] 25, placeholder reference 25. Replace with actual bibliographic data.
- [19] 26, placeholder reference 26. Replace with actual bibliographic data.
- [20] 27, placeholder reference 27. Replace with actual bibliographic data.
- [21] 28, placeholder reference 28. Replace with actual bibliographic data.
- [22] 29, placeholder reference 29. Replace with actual bibliographic data.
- [23] 3, placeholder reference 3. Replace with actual bibliographic data.
- [24] 30, placeholder reference 30. Replace with actual bibliographic data.
- [25] 31, placeholder reference 31. Replace with actual bibliographic data.
- [26] 32, placeholder reference 32. Replace with actual bibliographic data.
- [27] 33, placeholder reference 33. Replace with actual bibliographic data.
- [28] 34, placeholder reference 34. Replace with actual bibliographic data.
- [29] 35, placeholder reference 35. Replace with actual bibliographic data.
- [30] 36, placeholder reference 36. Replace with actual bibliographic data.
- [31] 37, placeholder reference 37. Replace with actual bibliographic data.
- [32] 38, placeholder reference 38. Replace with actual bibliographic data.
- [33] 39, placeholder reference 39. Replace with actual bibliographic data.
- [34] 4, placeholder reference 4. Replace with actual bibliographic data.
- [35] 40, placeholder reference 40. Replace with actual bibliographic data.
- [36] 41, placeholder reference 41. Replace with actual bibliographic data.
- [37] 42, placeholder reference 42. Replace with actual bibliographic data.
- [38] 43, placeholder reference 43. Replace with actual bibliographic data.
- [39] 44, placeholder reference 44. Replace with actual bibliographic data.
- [40] 45, placeholder reference 45. Replace with actual bibliographic data.
- [41] 46, placeholder reference 46. Replace with actual bibliographic data.
- [42] 47, placeholder reference 47. Replace with actual bibliographic data.

- [43] 48, placeholder reference 48. Replace with actual bibliographic data.
- [44] 49, placeholder reference 49. Replace with actual bibliographic data.
- [45] 5, placeholder reference 5. Replace with actual bibliographic data.
- [46] 50, placeholder reference 50. Replace with actual bibliographic data.
- [47] 51, placeholder reference 51. Replace with actual bibliographic data.
- [48] 52, placeholder reference 52. Replace with actual bibliographic data.
- [49] 53, placeholder reference 53. Replace with actual bibliographic data.
- [50] 54, placeholder reference 54. Replace with actual bibliographic data.
- [51] 55, placeholder reference 55. Replace with actual bibliographic data.
- [52] 56, placeholder reference 56. Replace with actual bibliographic data.
- [53] 57, placeholder reference 57. Replace with actual bibliographic data.
- [54] 58, placeholder reference 58. Replace with actual bibliographic data.
- [55] 59, placeholder reference 59. Replace with actual bibliographic data.
- [56] 6, placeholder reference 6. Replace with actual bibliographic data.
- [57] 60, placeholder reference 60. Replace with actual bibliographic data.
- [58] 7, placeholder reference 7. Replace with actual bibliographic data.
- [59] 8, placeholder reference 8. Replace with actual bibliographic data.
- [60] 9, placeholder reference 9. Replace with actual bibliographic data.

A PROV-O Export Example

This appendix provides a complete, syntactically valid Turtle serialization of an ADL Lite `EventChain` exported to the W3C PROV-O ontology ³. The example uses the `disc-capital-trap` concept (Scenario 1 from the E5 multi-agent pilot study, Section 4.7), which undergoes the full lifecycle: registration, validation, challenge, and fork. The serialization demonstrates how each ADL event maps to a `prov:Activity`, how agent roles map to `prov:Agent`, and how cryptographic hashes and causal links map to PROV-O's provenance relations.

A.1 A.1 Prefix Declarations

```

1 @prefix prov: \textless{http://www.w3.org/ns/prov\#\
   textgreater{} .}
2 @prefix adl: \textless{https://adl{-}lite.org/ns/\
   textgreater{} .}
3 @prefix xsd: \textless{http://www.w3.org/2001/XMLSchema\#\
   textgreater{} .}
4 @prefix rdfs: \textless{http://www.w3.org/2000/01/rdf{-}
   schema\#\textgreater{} .}
5 @prefix foaf: \textless{http://xmlns.com/foaf/0.1/\
   textgreater{} .}

```

³ W3C PROV-O: <https://www.w3.org/TR/prov-o/>.

The `adl:` namespace is a placeholder for the ADL Lite vocabulary URI. In a production deployment, this would resolve to the canonical namespace document (e.g., <https://adl-lite.org/ns/1.0/>). All PROV-O core terms (`prov:Entity`, `prov:Activity`, `prov:Agent`, `prov:wasGeneratedBy`, `prov:wasAssociatedWith`, `prov:wasInformedBy`, `prov:startedAtTime`) use the standard W3C namespace.

A.2 A.2 Concept Entity

The concept itself is modelled as a `prov:Entity` that is generated by the activities (events) in its chain. The entity carries ADL-specific properties (confidence, status, namespace) as datatype annotations.

```

1 adl:disc{-capital{-}trap}
2   a prov:Entity ;
3   rdfs:label "Capital Trap"@en ;
4   adl:conceptId "disc{-capital{-}trap" ;}
5   adl:namespace "disc" ;
6   adl:confidence "0.85"^^{xsd:float ;}
7   adl:status "provisional" ;
8   prov:wasGeneratedBy
9     adl:evt{-register{-}001 ,}
10    adl:evt{-validate{-}001 ,}
11    adl:evt{-challenge{-}001 ,}
12    adl:evt{-fork{-}001 ;}
13   prov:wasAttributedTo adl:actor{-discoverer .}
```

The `prov:wasGeneratedBy` property lists every activity that contributed to the concept's current state. The order of generation is implicit in the activity timestamps (Section A.3) and explicit in the `adl:previousEvent` links that form the cryptographic chain. The `prov:wasAttributedTo` property identifies the Discoverer as the original author.

A.3 A.3 Event Activities

Each event in the chain is modelled as a `prov:Activity` with its associated agent, timestamp, causal predecessors, and ADL-specific cryptographic metadata.

Registration Event The first event in the chain has no cryptographic predecessor; its `adl:previousEvent` is the literal string "genesis", indicating the start of the chain.

```

1 adl:evt{-register{-}001}
2   a prov:Activity , adl:RegisterEvent ;
3   rdfs:label "Register disc{-capital{-}trap"@en ;}
```

```

4   prov:wasAssociatedWith adl:actor{-discoverer ;}
5   prov:startedAtTime "2024{-01{-}15T09:00:00Z"\^{ }\^{ }xsd:
      dateTime ;}
6   adl:eventType "REGISTER" ;
7   adl:eventIndex "0"\^{ }\^{ }xsd:integer ;}
8   adl:previousEvent "genesis" ;
9   adl:eventHash "sha256:
      a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef123456
      " ;
10  adl:payloadConfidence "0.75"\^{ }\^{ }xsd:float ;}
11  adl:payloadSummary "Discoverer proposes Capital Trap
      pattern for review" .

```

Validation Event The validation event is associated with the Reviewer agent. Its `prov:wasInformedBy` property references the registration event, establishing the causal link in PROV-O terms. The `adl:previousEvent` property contains the SHA-256 hash of the registration event, enabling cryptographic chain verification.

```

1  adl:evt{-validate{-}001}
2    a prov:Activity , adl:ValidateEvent ;
3    rdfs:label "Validate disc{-capital{-}trap"@en ;}
4    prov:wasAssociatedWith adl:actor{-reviewer ;}
5    prov:wasInformedBy adl:evt{-register{-}001 ;}
6    prov:startedAtTime "2024{-01{-}15T14:30:00Z"\^{ }\^{ }xsd:
      dateTime ;}
7    adl:eventType "VALIDATE" ;
8    adl:eventIndex "1"\^{ }\^{ }xsd:integer ;}
9    adl:previousEvent "sha256:
      a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef123456
      " ;
10   adl:eventHash "sha256:
      e5f6a7b8c9d0123456789012345678901234fedcba0987654321fedcba098765
      " ;
11   adl:payloadConfidence "0.85"\^{ }\^{ }xsd:float ;}
12   adl:payloadValidator "reviewer{-agent{-}7" ;}
13   adl:payloadSummary "Reviewer validates with confidence
      0.85" .

```

Challenge Event The challenge event is initiated by the Skeptic agent. It represents a contested validation and triggers the fork workflow in Scenario 1.

```

1  adl:evt{-challenge{-}001}
2    a prov:Activity , adl:ChallengeEvent ;
3    rdfs:label "Challenge disc{-capital{-}trap validation"@en
      ;}
4    prov:wasAssociatedWith adl:actor{-skeptical ;}
5    prov:wasInformedBy adl:evt{-validate{-}001 ;}

```

```

6   prov:startedAtTime "2024{-01{-}16T10:15:00Z"\^{\^}}xsd:
    dateTime ;}
7   adl:eventType "CHALLENGE" ;
8   adl:eventIndex "2"\^{\^}}xsd:integer ;}
9   adl:previousEvent "sha256:
    e5f6a7b8c9d0123456789012345678901234fedcba0987654321fedcba098765
    " ;
10  adl:eventHash "sha256:1
    a2b3c4d5e6f789012345678901234567890abcde1234567890abcde1234567
    " ;
11  adl:payloadReason "Confidence threshold insufficient for
    production use" ;
12  adl:payloadSummary "Skeptic challenges validation;
    proposes fork" .

```

Fork Event The fork event is initiated by the Merger agent. It creates a new concept (`disc-capital-trap-v2`) and marks the original concept as forked. The `adl:forkTarget` property references the new concept entity.

```

1   adl:evt{-fork{-}001}
2   a prov:Activity , adl:ForkEvent ;
3   rdfs:label "Fork disc{-capital{-}trap"@en ;}
4   prov:wasAssociatedWith adl:actor{-merger ;}
5   prov:wasInformedBy adl:evt{-challenge{-}001 ;}
6   prov:startedAtTime "2024{-01{-}16T16:00:00Z"\^{\^}}xsd:
    dateTime ;}
7   adl:eventType "FORK" ;
8   adl:eventIndex "3"\^{\^}}xsd:integer ;}
9   adl:previousEvent "sha256:1
    a2b3c4d5e6f789012345678901234567890abcde1234567890abcde1234567
    " ;
10  adl:eventHash "sha256:9
    a8b7c6d5e4f321098765432109876543210fedcb0987654321fedcb0987654
    " ;
11  adl:forkTarget adl:disc{-capital{-}trap{-}v2 ;}
12  adl:payloadSummary "Merger resolves challenge by forking
    to disc{-capital{-}trap{-}v2" .}

```

A.4 A.4 Agent Definitions

Each agent role in the 5-agent harness is modelled as a `prov:Agent`. The `foaf:name` property provides a human-readable label; the `adl:role` property identifies the agent's function in the ADL workflow.

```

1   adl:actor{-discoverer}

```

```

2      a prov:Agent ;
3      foaf:name "Discoverer" ;
4      adl:role "discoverer" ;
5      rdfs:comment "Proposes new concepts for inclusion in the
        ADL corpus"@en .
6
7  adl:actor{-reviewer}
8      a prov:Agent ;
9      foaf:name "Reviewer" ;
10     adl:role "reviewer" ;
11     rdfs:comment "Validates proposed concepts against quality
        criteria"@en .
12
13  adl:actor{-skeptic}
14      a prov:Agent ;
15      foaf:name "Skeptic" ;
16      adl:role "skeptic" ;
17      rdfs:comment "Challenges validations and proposes forks
        when criteria are unmet"@en .
18
19  adl:actor{-merger}
20      a prov:Agent ;
21      foaf:name "Merger" ;
22      adl:role "merger" ;
23      rdfs:comment "Resolves challenges by merging or forking
        contested concepts"@en .
24
25  adl:actor{-librarian}
26      a prov:Agent ;
27      foaf:name "Librarian" ;
28      adl:role "librarian" ;
29      rdfs:comment "Archives deprecated or superseded concepts
        with evidence"@en .

```

A.5 A.5 Fork Target Entity

The fork event creates a new concept entity. This entity starts its own event chain at provisional status and is independently validated (or not) by the Reviewer agent.

```

1  adl:disc{-capital{-}trap{-}v2}
2      a prov:Entity ;
3      rdfs:label "Capital Trap v2"@en ;
4      adl:conceptId "disc{-capital{-}trap{-}v2" ;}
5      adl:namespace "disc" ;
6      adl:confidence "0.0"^^{xsd:float ;}
7      adl:status "provisional" ;

```

```

8   prov:wasDerivedFrom adl:disc{-capital{-}trap ;}
9   prov:wasGeneratedBy adl:evt{-fork{-}001 ;}
10  prov:wasAttributedTo adl:actor{-merger .}

```

The `prov:wasDerivedFrom` property records the provenance relationship between the forked concept and its parent. The `prov:wasGeneratedBy` property references the fork activity that created it. The initial confidence is 0.0 because the forked concept has not yet been independently validated.

A.6 A.6 Cryptographic Chain Verification

The PROV-O export preserves the cryptographic integrity of the original ADL event chain. An external verifier can reconstruct the chain as follows:

1. Sort the activities by `adl:eventIndex` to recover event order.
2. For each activity, verify that `adl:previousEvent` equals the `adl:eventHash` of the preceding activity (or "genesis" for index 0).
3. Recompute the SHA-256 digest of each activity's canonical content and confirm it matches `adl:eventHash`.
4. Verify that `prov:wasInformedBy` relations align with `adl:previousEvent` links (causal consistency).

This procedure ensures that the PROV-O serialization is not merely a passive export but a verifiable provenance record. Any tampering with the exported Turtle file—e.g., altering an event hash, changing a timestamp, or inserting a spurious `wasInformedBy` link—can be detected by recomputing the chain digests and comparing them against the stored `adl:eventHash` values.

A.7 A.7 Complete Document

The following listing presents the complete Turtle document in a single block suitable for loading into an RDF triplestore or passing to a SHACL validator.

```

1  @prefix prov: \textless{http://www.w3.org/ns/prov\#\
   textgreater{} .}
2  @prefix adl: \textless{https://adl{-}lite.org/ns/\
   textgreater{} .}
3  @prefix xsd: \textless{http://www.w3.org/2001/XMLSchema\#\
   textgreater{} .}
4  @prefix rdfs: \textless{http://www.w3.org/2000/01/rdf{-}
   schema\#\textgreater{} .}
5  @prefix foaf: \textless{http://xmlns.com/foaf/0.1/\
   textgreater{} .}
6

```


[illegible]

[illegible]


```

8      rdfs:range adl:Concept .
9
10     adl:specialisation{-of}
11       a rdfs:Property ;
12       rdfs:label "specialisation of"@en ;
13       rdfs:comment "The subject concept is a specialisation of
14         the object concept"@en ;
15       rdfs:domain adl:RelationBlock ;
16       rdfs:range adl:Concept .
17
18     adl:co{-occurs{-}with}
19       a rdfs:Property ;
20       rdfs:label "co{-occurs with"@en ;}
21       rdfs:comment "Concepts that appear together in the same
22         context or corpus"@en ;
23       rdfs:domain adl:RelationBlock ;
24       rdfs:range adl:Concept .
25
26     adl:related{-to}
27       a rdfs:Property ;
28       rdfs:label "related to"@en ;
29       rdfs:comment "Generic relatedness without further
30         specificity"@en ;
31       rdfs:domain adl:RelationBlock ;
32       rdfs:range adl:Concept .
33
34     adl:analogical{-to}
35       a rdfs:Property ;
36       rdfs:label "analogical to"@en ;
37       rdfs:comment "Analogy{-based relationship between
38         concepts from different domains"@en ;}
39       rdfs:domain adl:RelationBlock ;
40       rdfs:range adl:Concept .
41
42     adl:dual{-of}
43       a rdfs:Property ;
44       rdfs:label "dual of"@en ;
45       rdfs:comment "Duality relationship where each concept is
46         the complement of the other"@en ;
47       rdfs:domain adl:RelationBlock ;
48       rdfs:range adl:Concept .
49
50     adl:fork{-of}
51       a rdfs:Property ;
52       rdfs:label "fork of"@en ;
53       rdfs:comment "The subject concept is a fork (variant) of
54         the object concept"@en ;
55       rdfs:domain adl:RelationBlock ;
56       rdfs:range adl:Concept .

```



```

60 \# {-}{-}{-} Constraint 4: Confidence (optional, bounded)
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}

61 sh:property [
62     sh:path adl:confidence ;
63     sh:name "relation confidence" ;
64     sh:description "Confidence score for the relation
        assertion" ;
65     sh:maxCount 1 ;
66     sh:datatype xsd:float ;
67     sh:minInclusive 0.0 ;
68     sh:maxInclusive 1.0 ;
69     sh:message
70         "Confidence must be a float in the range [0.0,
            1.0]" ;
71     sh:severity sh:Violation
72 ] ;
73
74 \# {-}{-}{-} Constraint 5: Directionality
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}

75 sh:property [
76     sh:path adl:bidirectional ;
77     sh:name "bidirectional flag" ;
78     sh:description "Whether the relation is bidirectional
        " ;
79     sh:maxCount 1 ;
80     sh:datatype xsd:boolean ;
81     sh:message
82         "Bidirectional flag must be a boolean (true or
            false)" ;
83     sh:severity sh:Violation
84 ] ;
85
86 \# {-}{-}{-} Constraint 6: Evidence reference (optional)
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}

87 sh:property [
88     sh:path adl:evidence ;
89     sh:name "evidence reference" ;
90     sh:description "URI or citation supporting the
        relation" ;
91     sh:maxCount 1 ;
92     sh:datatype xsd:anyURI ;
93     sh:message
94         "Evidence reference must be a valid URI" ;
95     sh:severity sh:Warning
96 ] .

```

Constraint Design Rationale Closed predicate set (Constraint 2). The `sh:in` constraint on `adl:predicate` enforces the closed-world assumption of the ADL Core ontology. This prevents ontology drift: authors cannot introduce ad-hoc predicates (e.g., `similar-to`, `like`, `connected-to`) that would fragment the concept graph and compromise interoperability. If a new relationship type is needed, it must be added to the ADL Core ontology (via the governance process described in Section 3.2) and to the `sh:in` list in the shape definition before it can be used in relation blocks.

Concept ID pattern (Constraints 1 and 3). The sh:pattern "[a-z0-9_-]+\$" ensures that source and target values are valid ADL concept identifiers: lowercase alphanumeric strings with hyphens and underscores. This prevents malformed references, empty strings, and identifiers containing spaces or special characters that would break URL construction or file-system operations.

Confidence bounding (Constraint 4). The `sh:minInclusive` and `sh:maxInclusive` constraints on `adl:confidence` ensure that confidence scores are interpretable probabilities. Values outside $[0, 1]$ indicate a data-entry error or a miscalibrated confidence model and are rejected at validation time.

Severity levels. Violations of structural constraints (source, predicate, target, confidence) are reported as **sh:Violation** (the default), meaning the data graph does not conform to the shape. The evidence reference constraint is reported as **sh:Warning**, meaning the relation block is structurally valid but lacks supporting evidence, which is permissible but not recommended.

A.12 B.4 Worked Validation Example

Consider the following data graph containing two relation blocks: one valid and one invalid.

[illegible]

```

14 \# {-}{-}{-} Invalid relation block (predicate not in closed
15 set) {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}
16
17 adl:relation{-002}
18   a adl:RelationBlock ;
19   adl:source "disc{-attention{-}residual" ;}
20   adl:predicate "similar{-to" ;                \# \textless
        {-}{-}{-} NOT in ADL Core set}
21   adl:target "concept{-gradient{-}explosion" ;}
22   adl:confidence "1.5"^^{\^{\^}}xsd:float .    \# \
        textless{-}{-}{-} Out of bounds}

```

Validation Report Executing the SHACL validator (e.g., Apache Jena `shacl` `validate` or `pyshacl`) against the data graph produces the following validation report:

```

=====
Validation Report
=====
Conforms: False

Results (3):
-----

Result 1:
Severity:      sh:Violation
Source Shape:  adl:RelationShape > adl:predicate
Focus Node:    adl:relation-002
Result Path:   adl:predicate
Value:         "similar-to"
Message:       Predicate must be from the closed ADL Core set:
               isomorphic-to, specialisation-of, co-occurs-with,
               related-to, analogical-to, dual-of, or fork-of
Debug Info:    The predicate "similar-to" is not registered in the
               ADL Core ontology. To use this predicate, submit a
               proposal to the ADL governance process (Section 3.2)
               to add it to the closed set.

-----

Result 2:
Severity:      sh:Violation
Source Shape:  adl:RelationShape > adl:confidence
Focus Node:    adl:relation-002
Result Path:   adl:confidence
Value:         "1.5"^^xsd:float
Message:       Confidence must be a float in the range [0.0, 1.0]

```



```

Debug Info:   Received confidence value 1.5 which exceeds the
               maximum inclusive bound of 1.0. Confidence scores
               must be normalised to the [0.0, 1.0] interval.

```

Result 3:

```

Severity:      sh:Warning
Source Shape:  adl:RelationShape > adl:evidence
Focus Node:    adl:relation-002
Result Path:   adl:evidence
Message:       Evidence reference is recommended for relation assertions
Debug Info:    Relation adl:relation-002 lacks an evidence reference.
               While not required, providing a URI or citation improves
               auditability and supports confidence calibration.

```

Interpretation The validator rejects `adl:relation-002` for two structural violations and issues one warning:

- **Violation 1 (predicate):** The string `"similar-to"` is not a member of the closed `sh:in` set. This is the most common validation failure in practice: authors frequently use natural-language predicate names that have not been admitted to the ADL Core ontology. The fix is either to map `"similar-to"` to the closest registered predicate (e.g., `adl:related-to`) or to propose the new predicate through the governance process.
- **Violation 2 (confidence):** The value 1.5 exceeds the `sh:maxInclusive` 1.0 bound. This typically indicates a bug in the confidence computation pipeline (e.g., unnormalised raw scores passed directly to the relation block) or a data-entry error. The fix is to normalise the confidence score to `[0, 1]` before serialisation.
- **Warning (evidence):** The absence of an `adl:evidence` property is reported as a `sh:Warning`, not a violation, because evidence is recommended but not mandatory. This allows relation blocks to pass validation during early-stage authoring while encouraging authors to add citations before final publication.

The valid relation block (`adl:relation-001`) produces no validation results: all constraints are satisfied, the predicate `adl:specialisation-of` is in the closed set, the confidence 0.85 is within bounds, and the evidence URI is well-formed.

A.13 B.5 Integrating SHACL Validation into the ADL Pipeline

The SHACL shape is executed at two points in the ADL Lite pipeline:

- The shape graph is versioned alongside the ADL Core ontology. When a new predicate is admitted through governance, it is added to both the ontology file (`adl_core_ontology.yaml`) and the SHACL shape (`adl_relation_shape.ttl`), ensuring that the structural constraints remain synchronised with the semantic vocabulary.

The following listing presents the complete SHACL shape graph and ADL Core predicate definitions in a single Turtle document suitable for loading into any SHACL-compliant validator.

[illegible]

```

18 adl:co{-occurs{-}with}
19   a rdfs:Property ;
20   rdfs:label "co{-occurs with"@en ;}
21   rdfs:comment "Concepts that appear together in the same
      context or corpus"@en .
22
23 adl:related{-to}
24   a rdfs:Property ;
25   rdfs:label "related to"@en ;
26   rdfs:comment "Generic relatedness without further
      specificity"@en .
27
28 adl:analogical{-to}
29   a rdfs:Property ;
30   rdfs:label "analogical to"@en ;
31   rdfs:comment "Analogy{-based relationship between
      concepts from different domains"@en .}
32
33 adl:dual{-of}
34   a rdfs:Property ;
35   rdfs:label "dual of"@en ;
36   rdfs:comment "Duality relationship where each concept is
      the complement of the other"@en .
37
38 adl:fork{-of}
39   a rdfs:Property ;
40   rdfs:label "fork of"@en ;
41   rdfs:comment "The subject concept is a fork (variant) of
      the object concept"@en .
42
43 \# {-}{-}{-} Relation Shape Definition
    {-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}{-}
44
45 adl:RelationShape
46   a sh:NodeShape ;
47   rdfs:label "ADL L3 Relation Block Shape"@en ;
48   rdfs:comment "Validates relation blocks in ADL Lite Level
       3 (L3) concept definitions"@en ;
49   sh:targetClass adl:RelationBlock ;
50
51   sh:property [
52     sh:path adl:source ;
53     sh:name "source concept" ;
54     sh:minCount 1 ;
55     sh:maxCount 1 ;
56     sh:datatype xsd:string ;
57     sh:pattern "\~{[a{-}z0{-}9\_{-}]+\}$" ;}
58     sh:message
```

```

59         "Relation source must be a valid concept ID: non
60         {-empty lowercase "}
61         "alphanumeric string with hyphens and underscores
62         only" ;
63     sh:severity sh:Violation
64 ] ;
65
66 sh:property [
67     sh:path adl:predicate ;
68     sh:name "relation predicate" ;
69     sh:minCount 1 ;
70     sh:maxCount 1 ;
71     sh:in (
72         adl:isomorphic{-to}
73         adl:specialisation{-of}
74         adl:co{-occurs{-}with}
75         adl:related{-to}
76         adl:analogical{-to}
77         adl:dual{-of}
78         adl:fork{-of}
79     ) ;
80     sh:message
81     "Predicate must be from the closed ADL Core set:
82     "
83     "isomorphic{-to, specialisation{-}of, co{-}occurs
84     {-}with, "
85     "related{-to, analogical{-}to, dual{-}of, or fork
86     {-}of" ;}
87     sh:severity sh:Violation
88 ] ;
89
90 sh:property [
91     sh:path adl:target ;
92     sh:name "target concept" ;
93     sh:minCount 1 ;
94     sh:maxCount 1 ;
95     sh:datatype xsd:string ;
96     sh:pattern "\\^[a{-}z0{-}9\\_{-}]+\\$" ;}
97     sh:message
98     "Relation target must be a valid concept ID: non
99     {-empty lowercase "}
100     "alphanumeric string with hyphens and underscores
101     only" ;
102     sh:severity sh:Violation
103 ] ;
104
105 sh:property [
106     sh:path adl:confidence ;
107     sh:name "relation confidence" ;
108     sh:maxCount 1 ;

```

```
102         sh:datatype xsd:float ;
103         sh:minInclusive 0.0 ;
104         sh:maxInclusive 1.0 ;
105         sh:message "Confidence must be a float in the range
106             [0.0, 1.0]" ;
107         sh:severity sh:Violation
108     ] ;
109     sh:property [
110         sh:path adl:bidirectional ;
111         sh:name "bidirectional flag" ;
112         sh:maxCount 1 ;
113         sh:datatype xsd:boolean ;
114         sh:message "Bidirectional flag must be a boolean (
115             true or false)" ;
116         sh:severity sh:Violation
117     ] ;
118     sh:property [
119         sh:path adl:evidence ;
120         sh:name "evidence reference" ;
121         sh:maxCount 1 ;
122         sh:datatype xsd:anyURI ;
123         sh:message "Evidence reference must be a valid URI" ;
124         sh:severity sh:Warning
125     ] .
```